

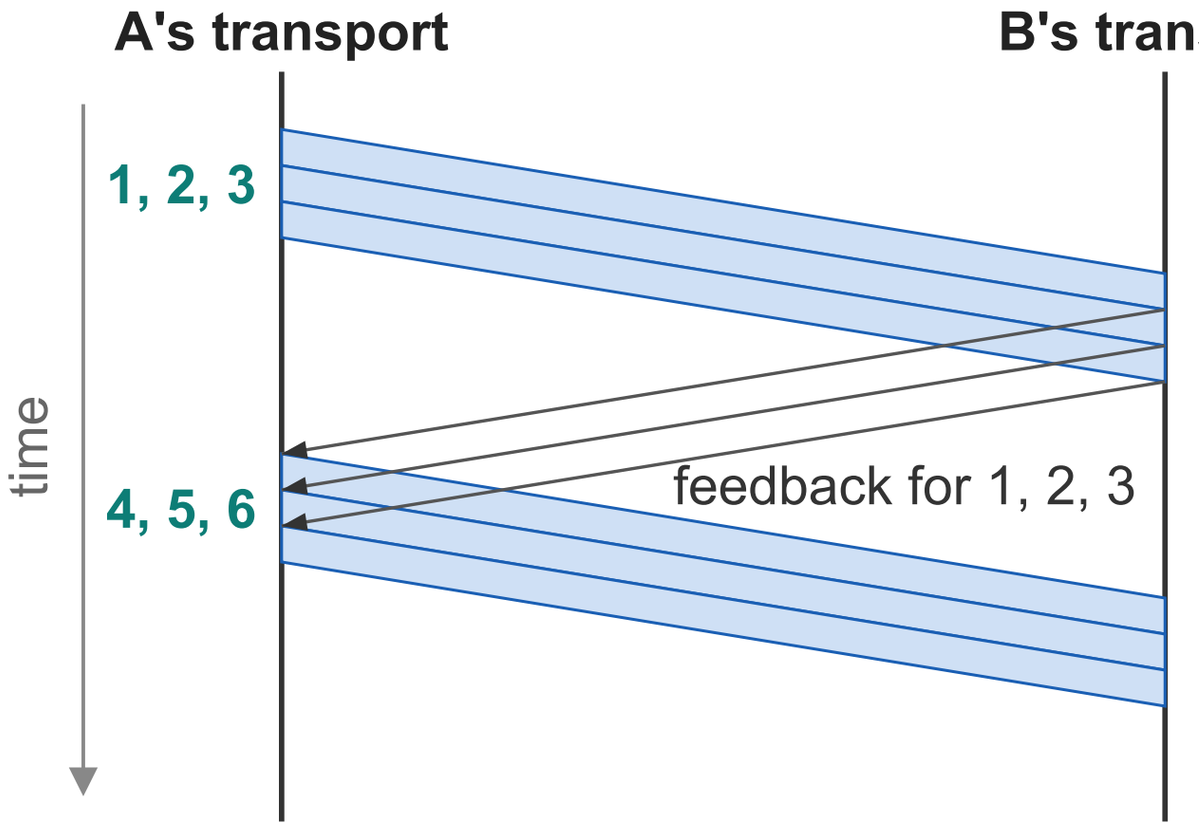
UNIVERSITY OF WATERLOO

Congestion control - Part I

CS 456/656, Fall 2026

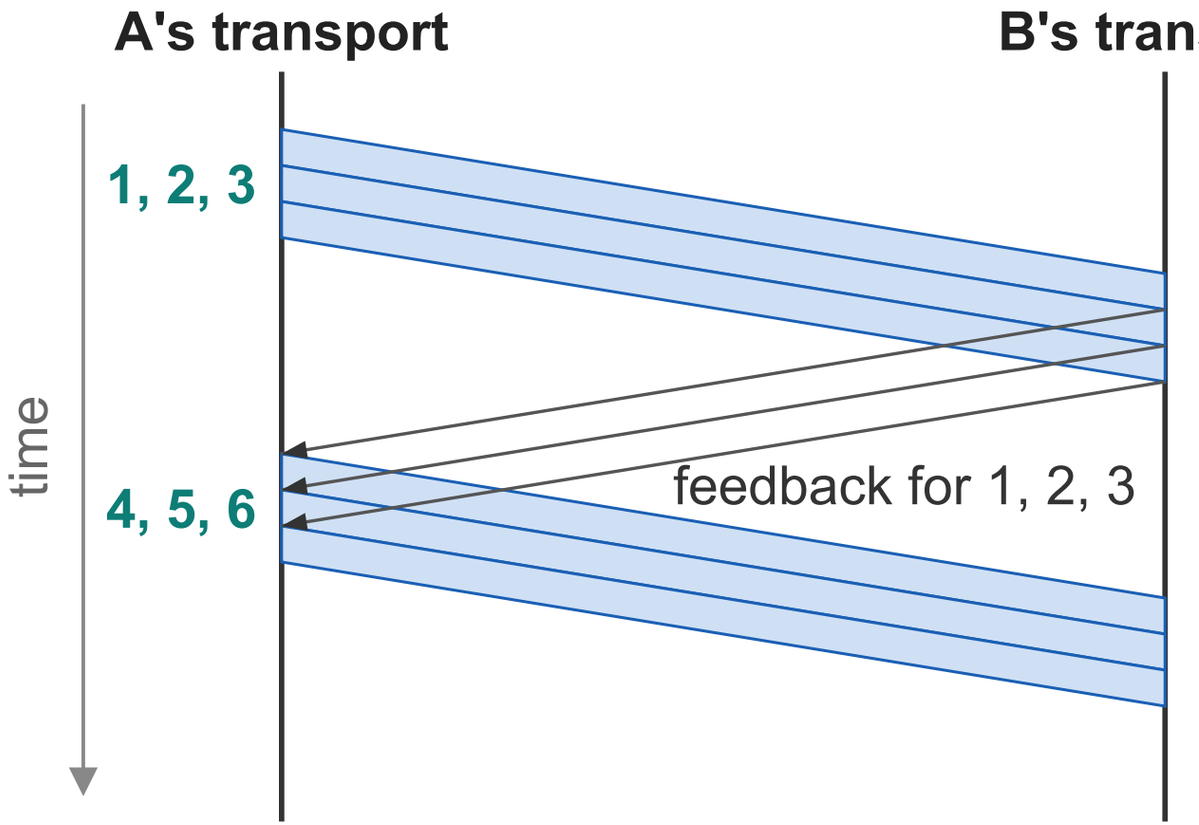
Mina Tahmasbi Arashloo

Recap: a batch of three packets at a time



A sends 3 packets back to back,
without waiting for feedback
in between.

Recap: a batch of three packets at a time



A sends 3 packets back to back, without waiting for feedback in between.

3 packets every round trip:
3 times the utilization

0.024 / (30 + 0.024) = 0.0008

Recap: what is a good batch size?

- **Too small:** we are underutilizing the network, and it takes longer to get the data delivered
- **Too big:**
  - The network is shared. If everyone sends everything they have at full speed, it will create congestion that can result in increasing drops and delays
  - The sender may “overwhelm” the receiver

Recap: what is a good batch size?

- **Too small:** we are underutilizing the network, and it takes longer to get the data delivered
- **Too big:**
  - The network is shared. If everyone sends everything they have at full speed, it will create congestion that can result in increasing drops and delays
  - The sender may “overwhelm” the receiver

✓ **flow control with a sliding window**

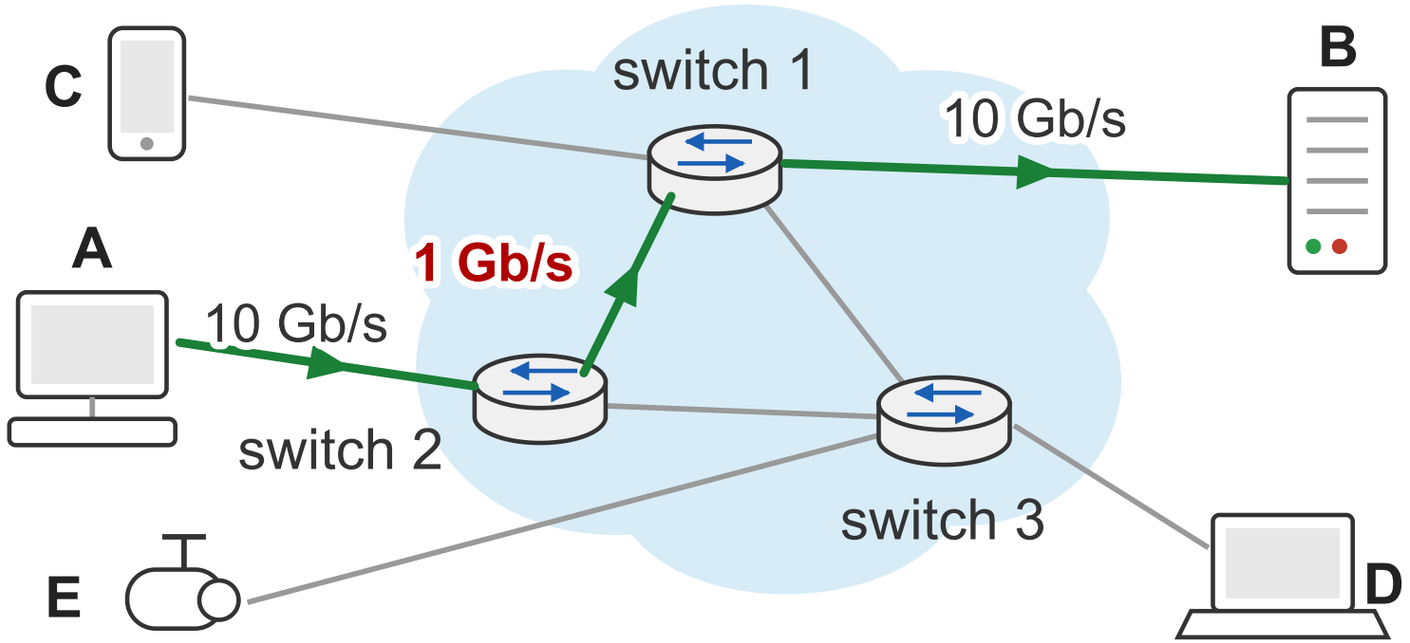
Recap: what is a good batch size?

Part 2

- **Too small:** we are underutilizing the network, and it takes longer to get the data delivered
- **Too big:**
  - The network is shared. If everyone sends everything they have at full speed, it will create congestion that can result in increasing drops and delays
  - The sender may “overwhelm” the receiver

✓ flow control with a sliding window

How fast should the sender send?



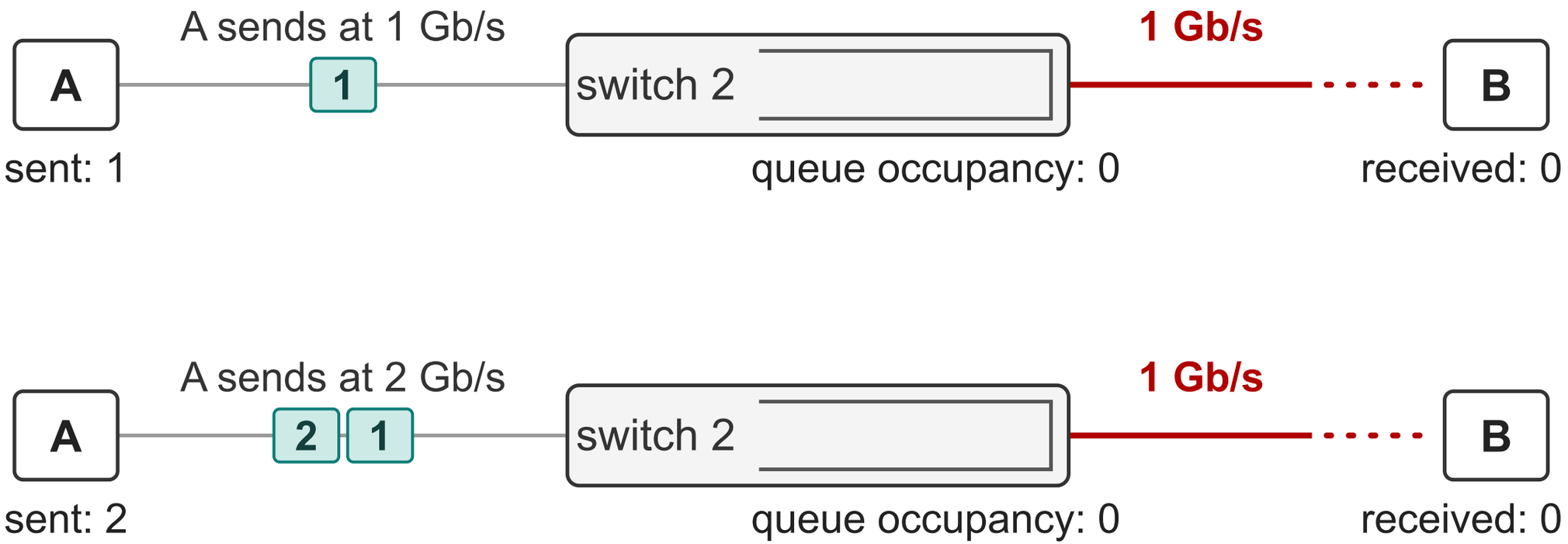
For intuition, we first talk about **sending rate**, not batch size (rate ≈ batch size / RTT).

Consider a sender and a receiver:

- The sender knows exactly what path its traffic will take to the destination, and the properties (rate, propagation delay, etc.) of each link
- The sender always has more data to send

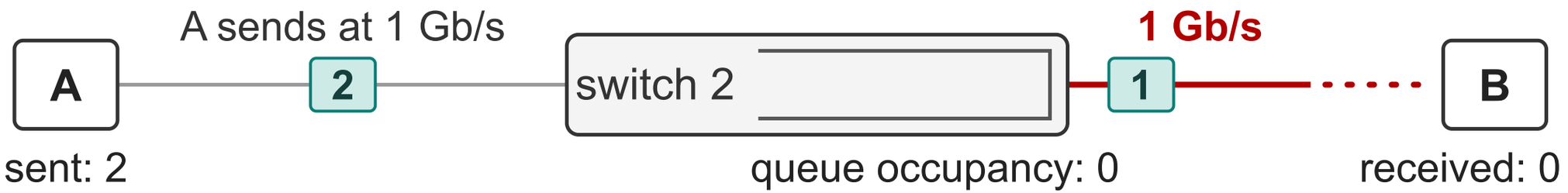
What if A sends faster than the slowest link?

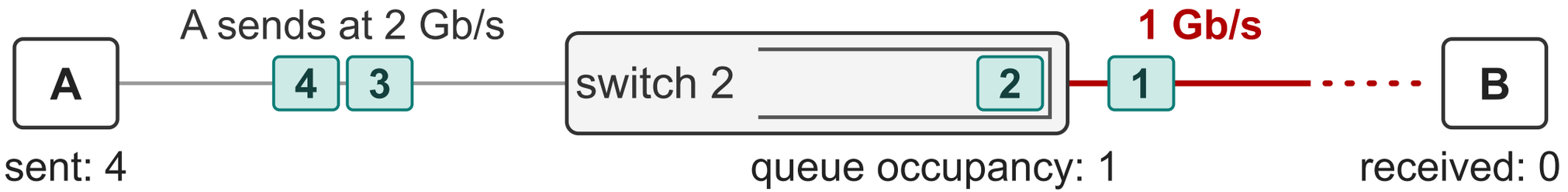
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



What if A sends faster than the slowest link?

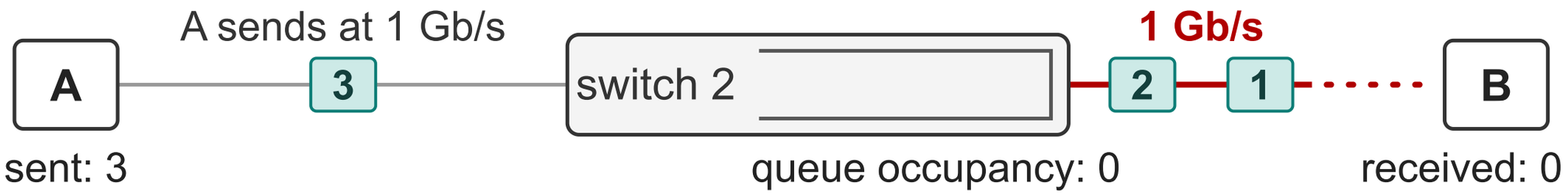
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).

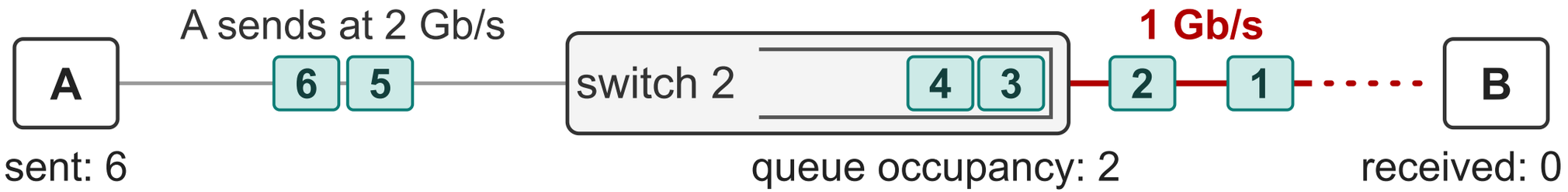




What if A sends faster than the slowest link?

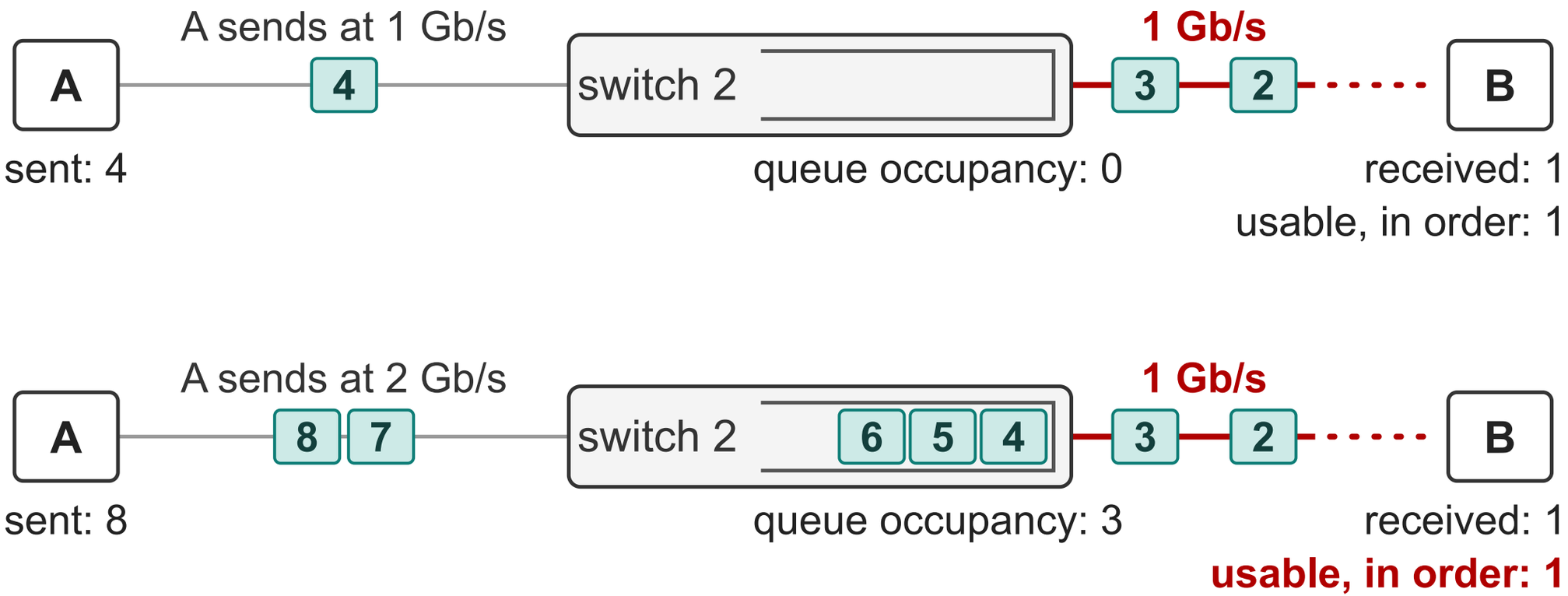
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).





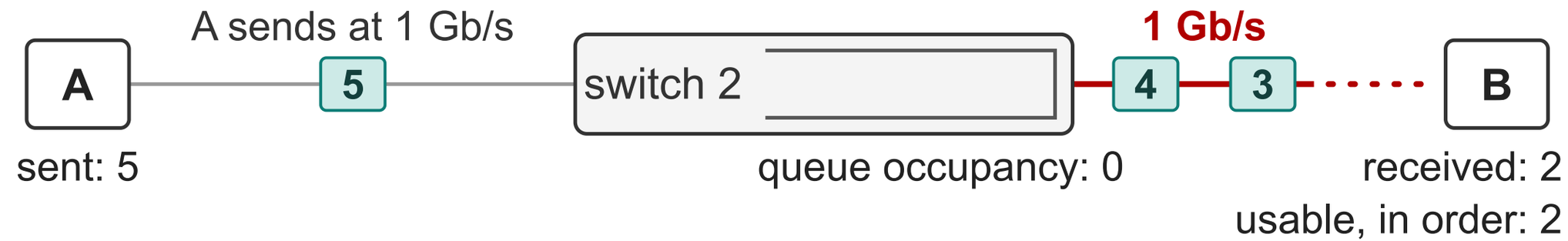
What if A sends faster than the slowest link?

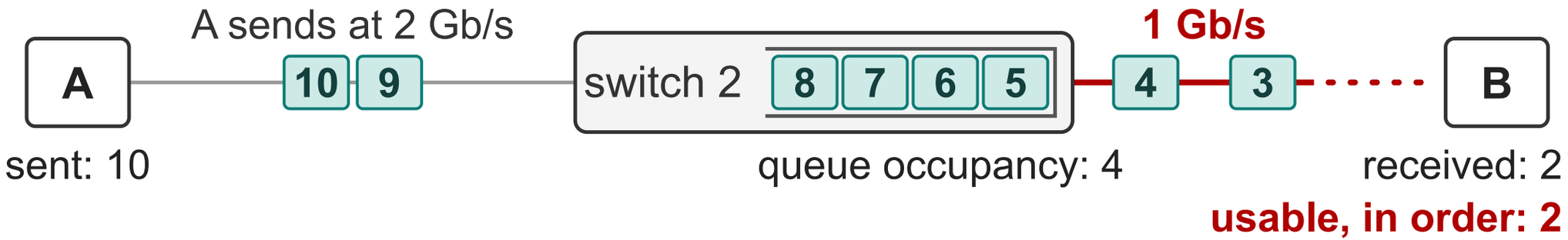
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



What if A sends faster than the slowest link?

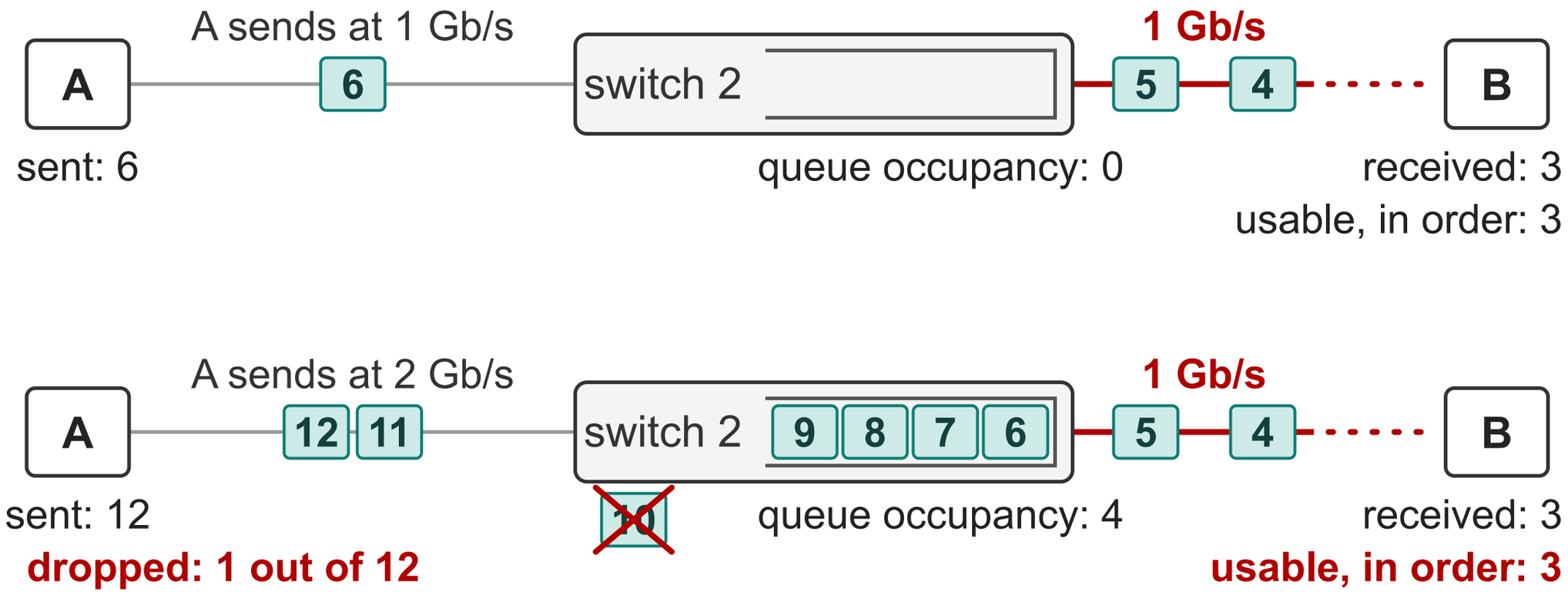
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).





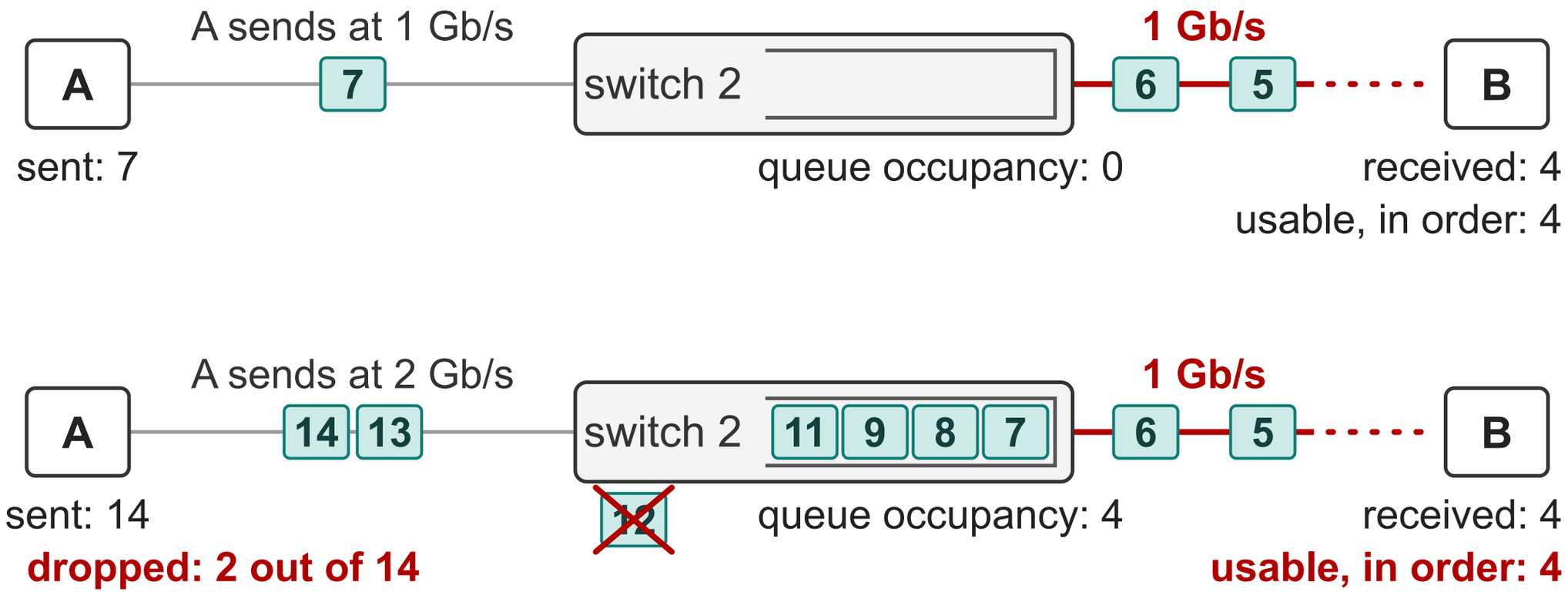
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



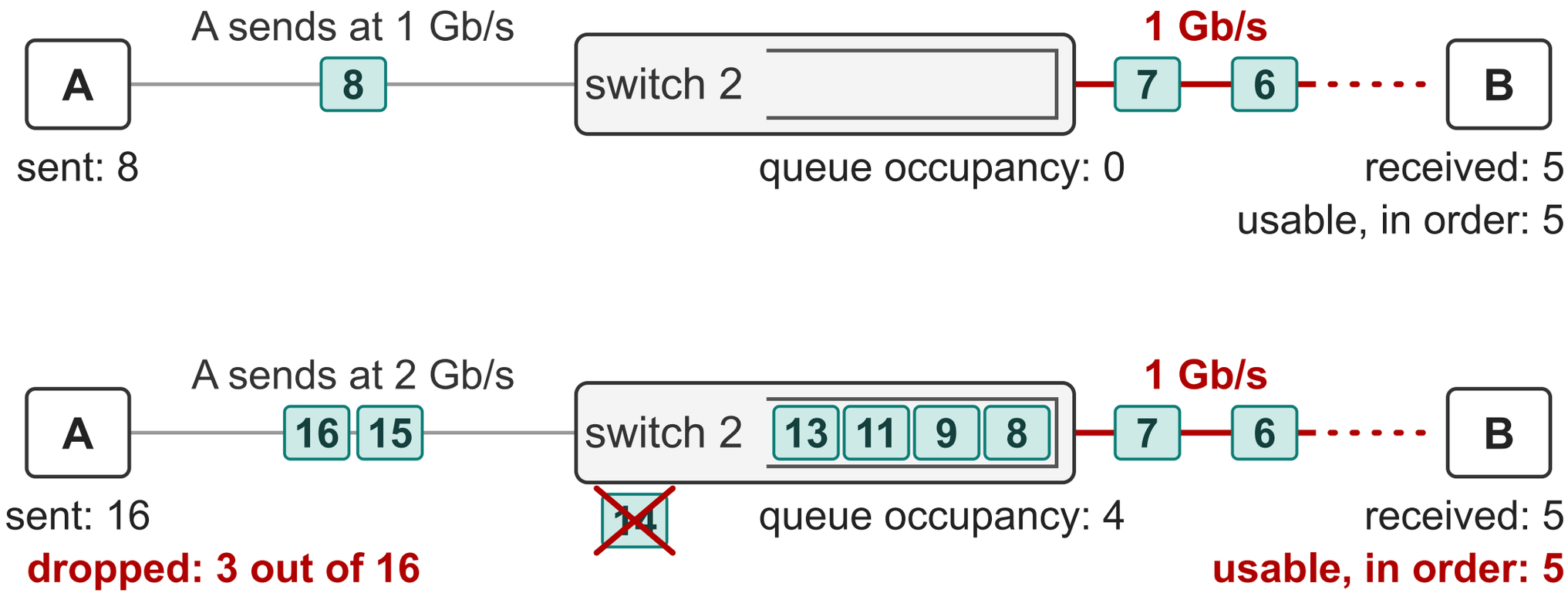
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



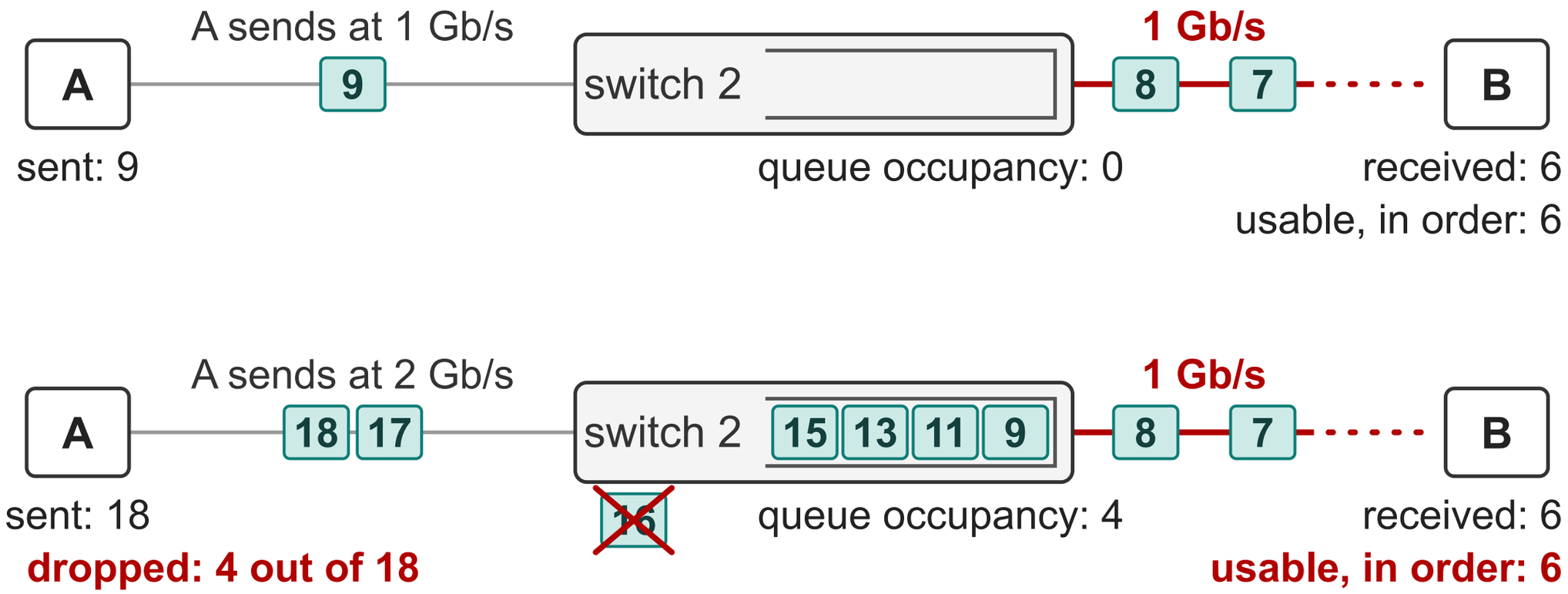
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



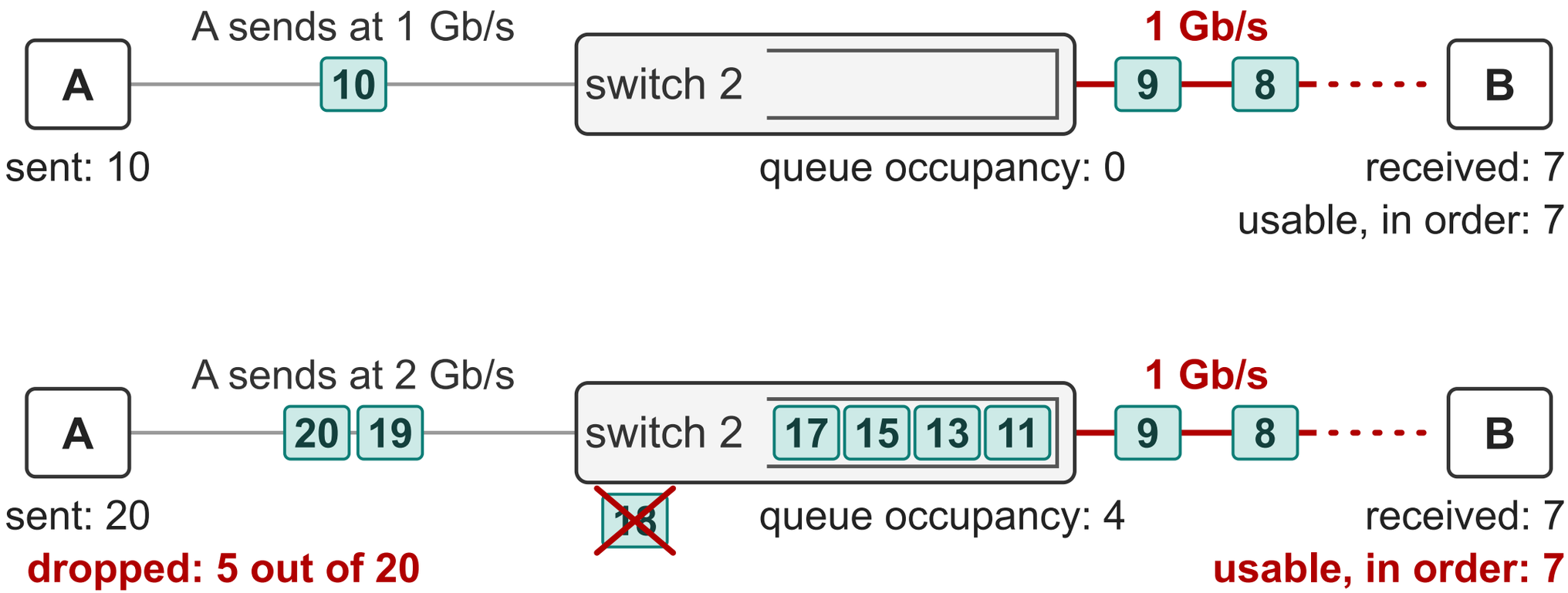
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



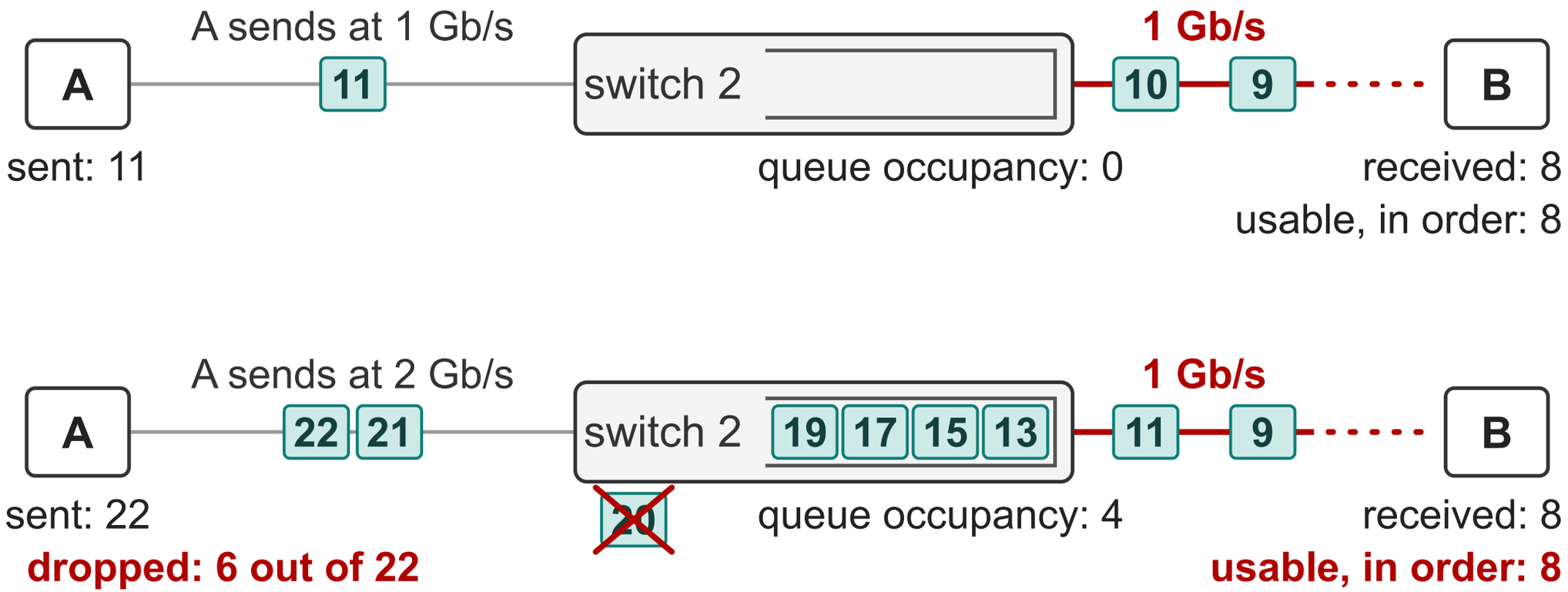
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



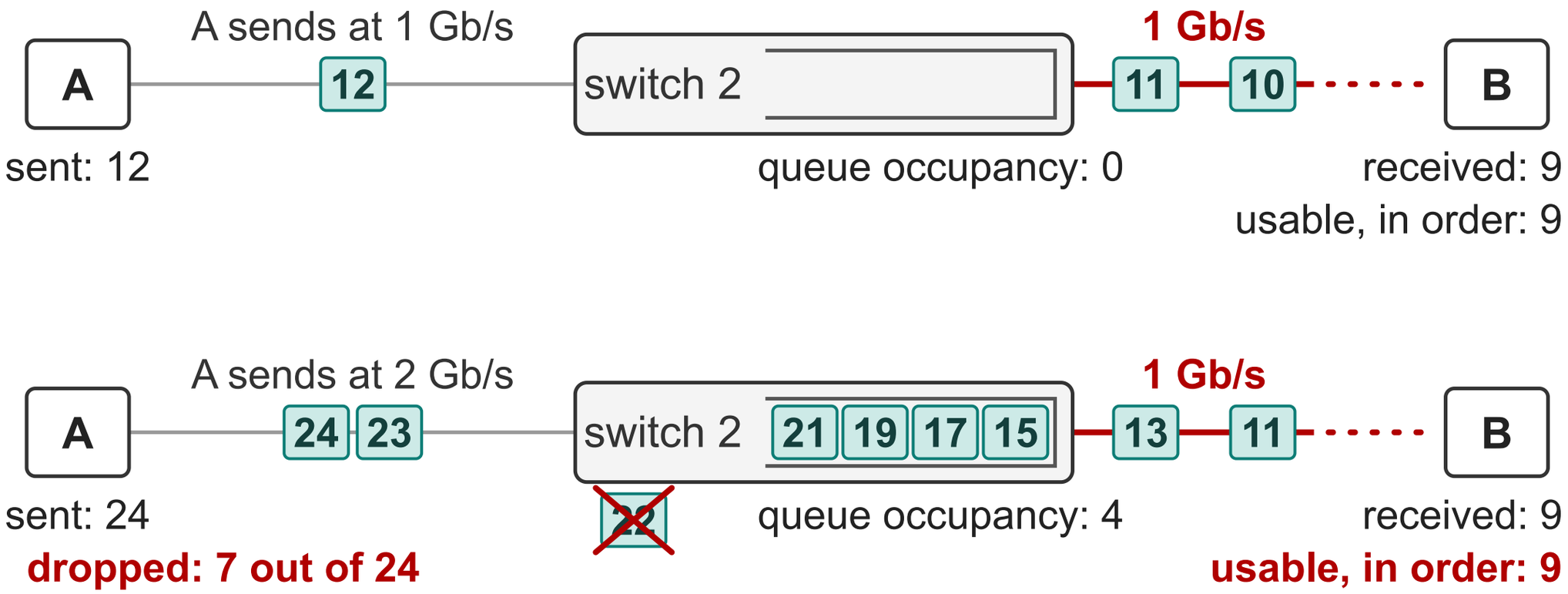
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



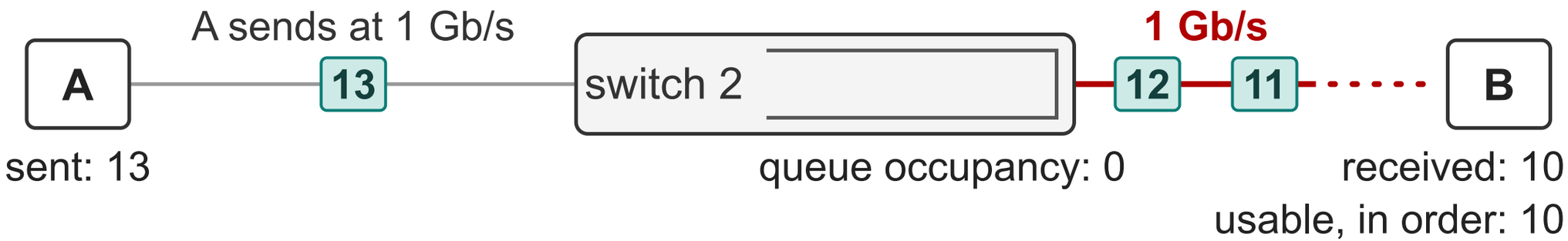
What if A sends faster than the slowest link?

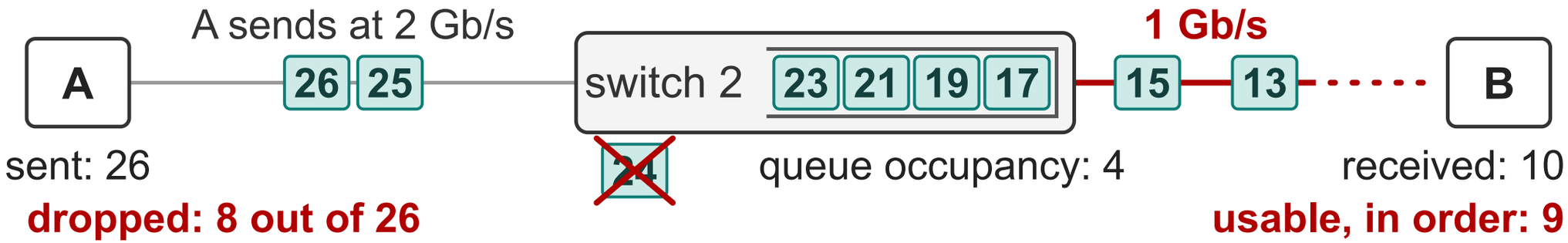
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



What if A sends faster than the slowest link?

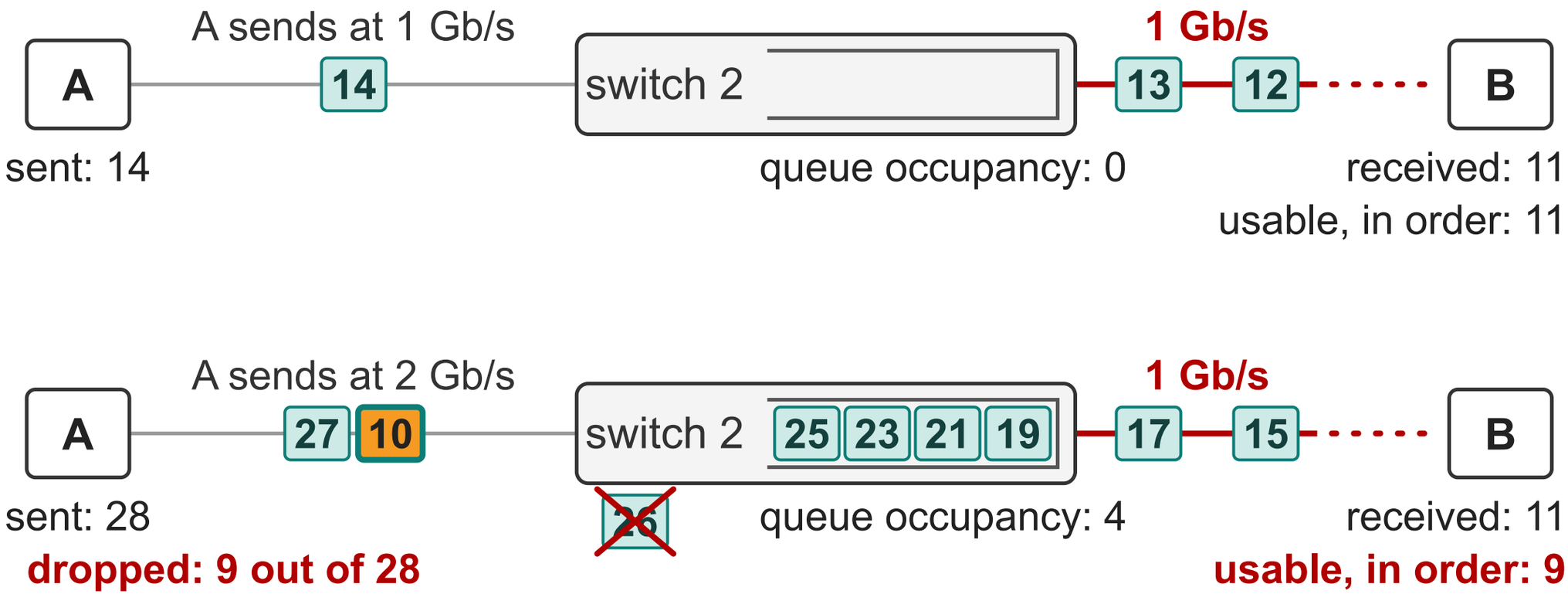
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).





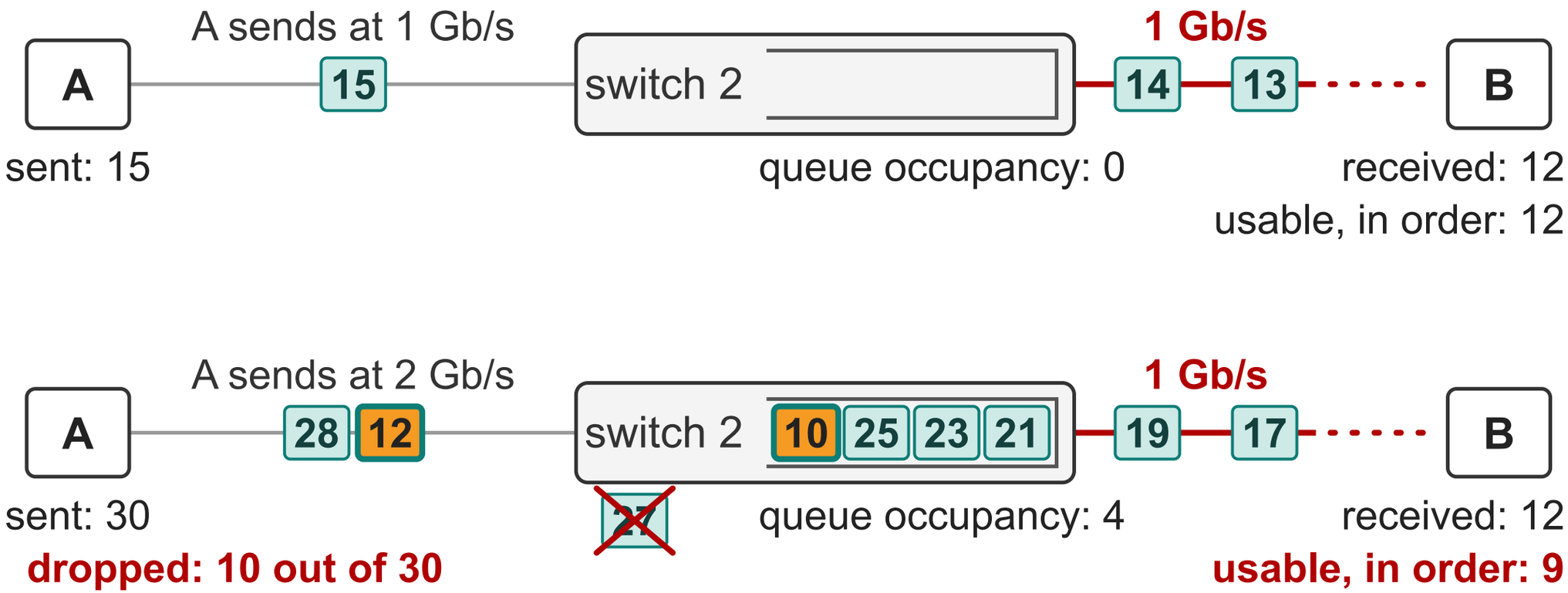
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



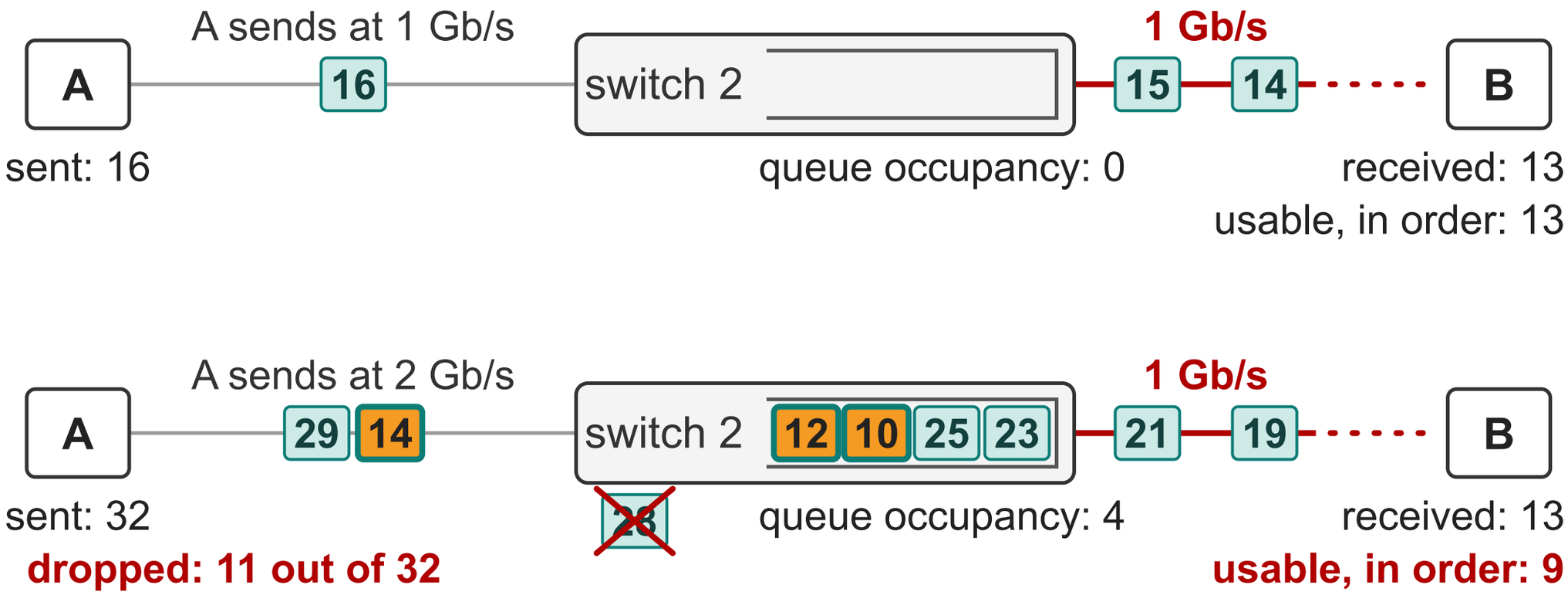
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



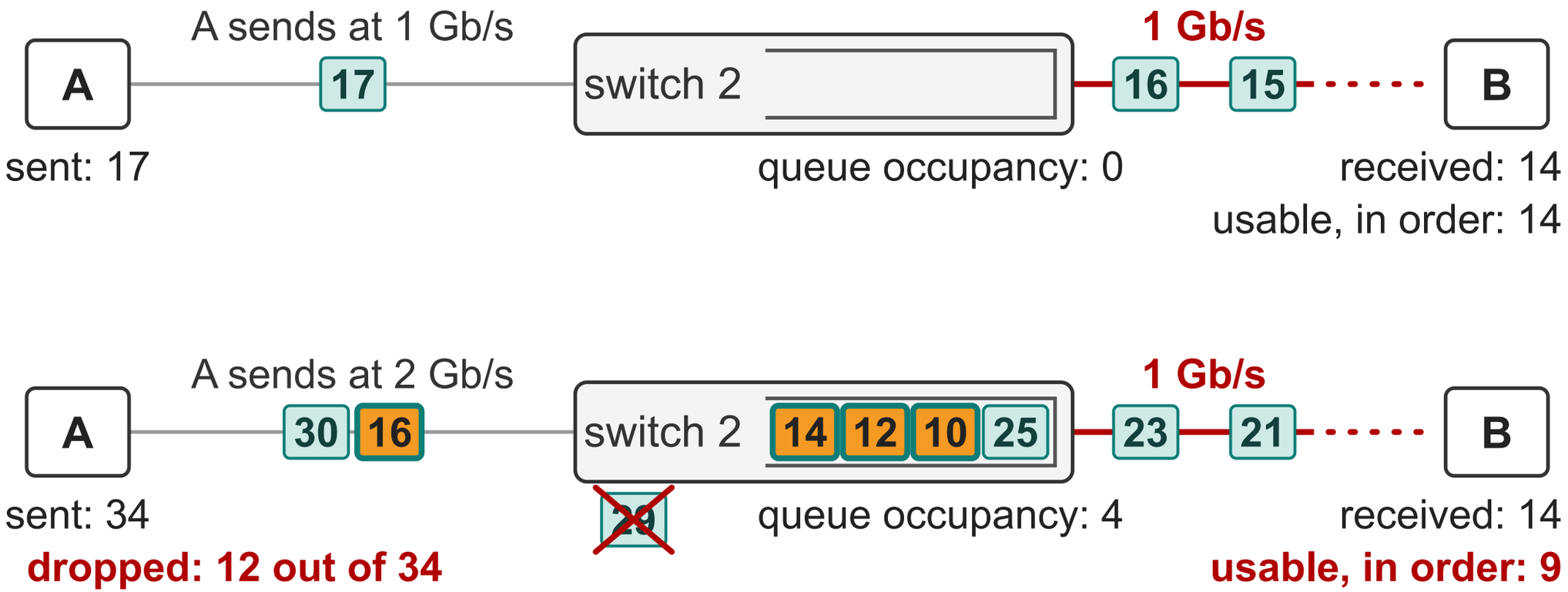
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



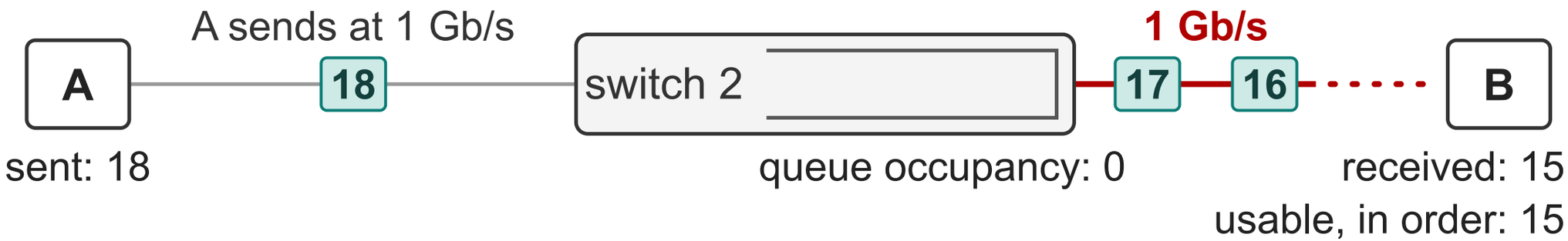
What if A sends faster than the slowest link?

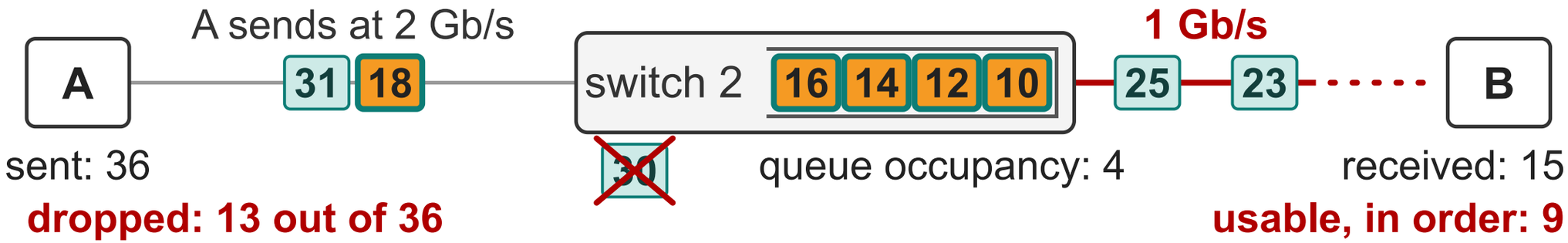
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



What if A sends faster than the slowest link?

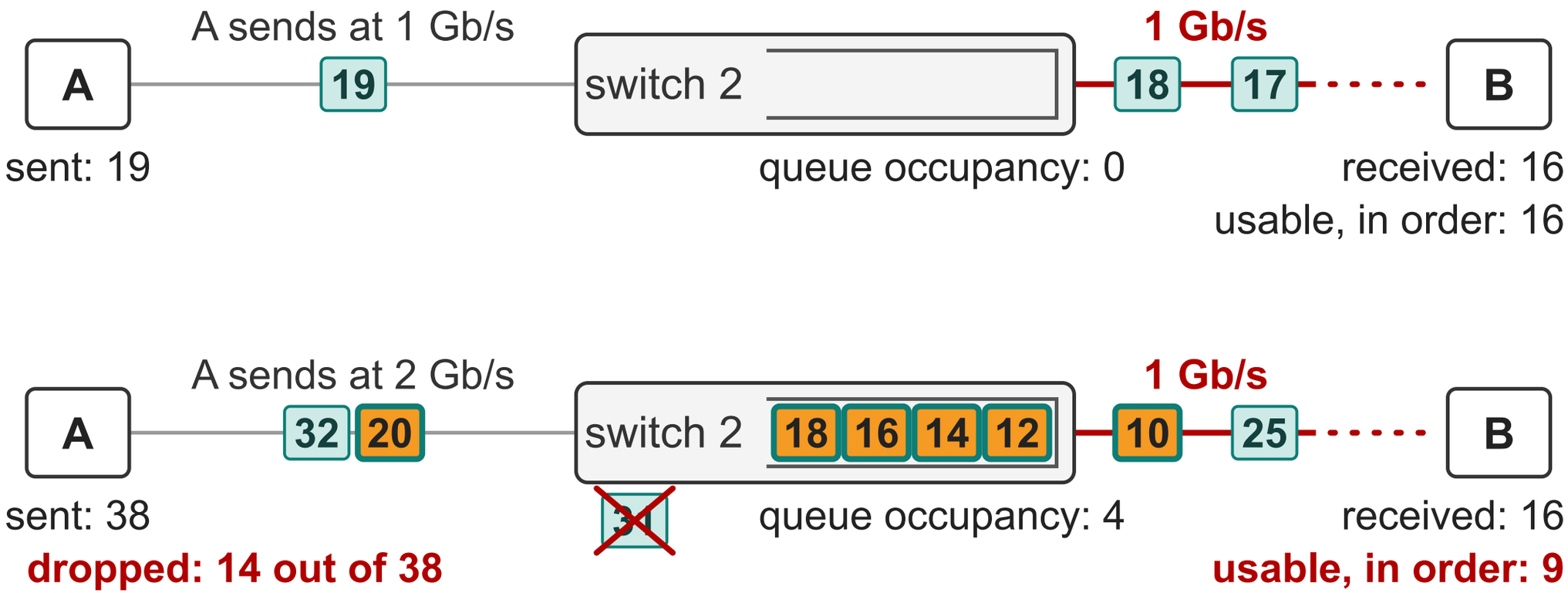
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).





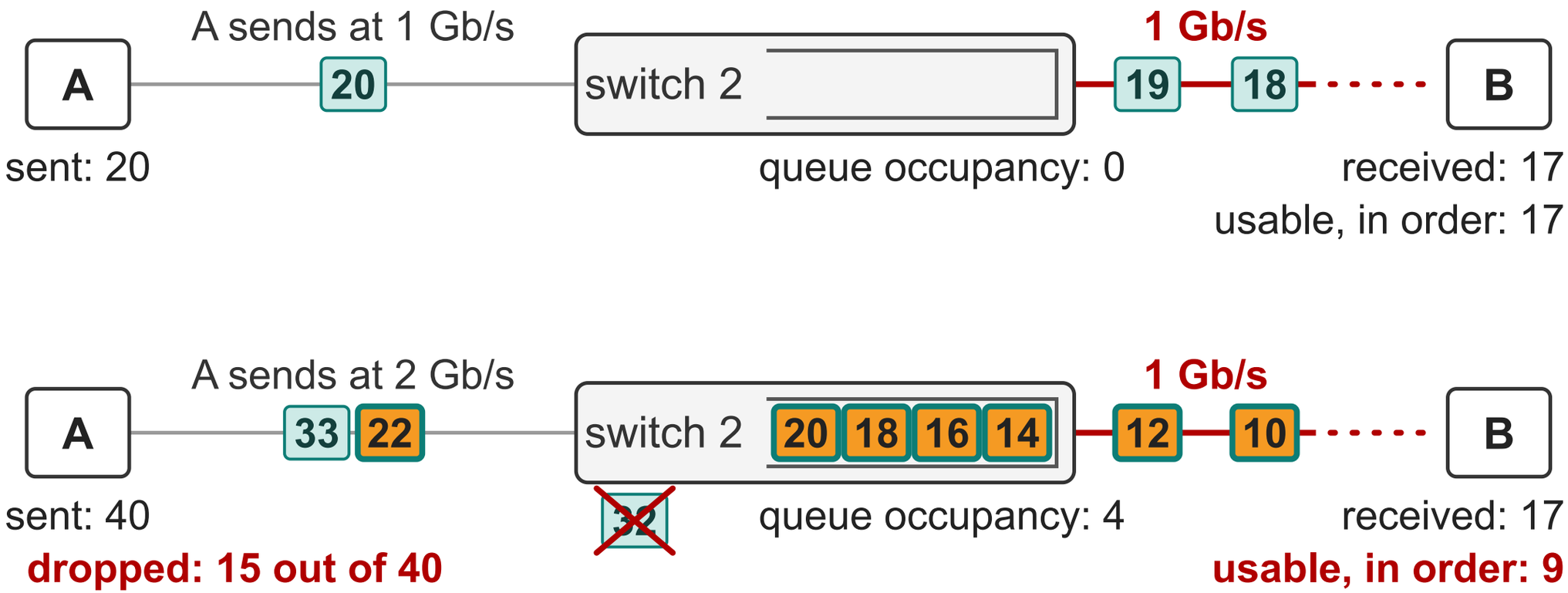
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



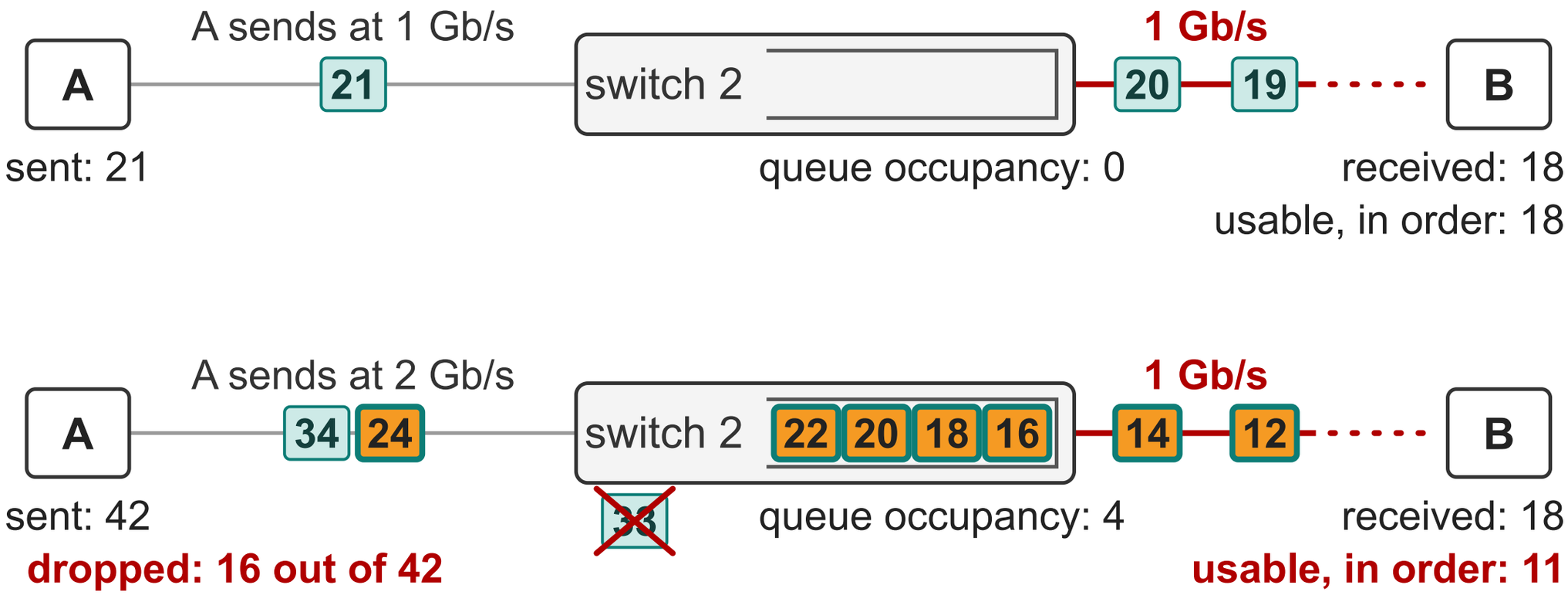
What if A sends faster than the slowest link?

When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



What if A sends faster than the slowest link?

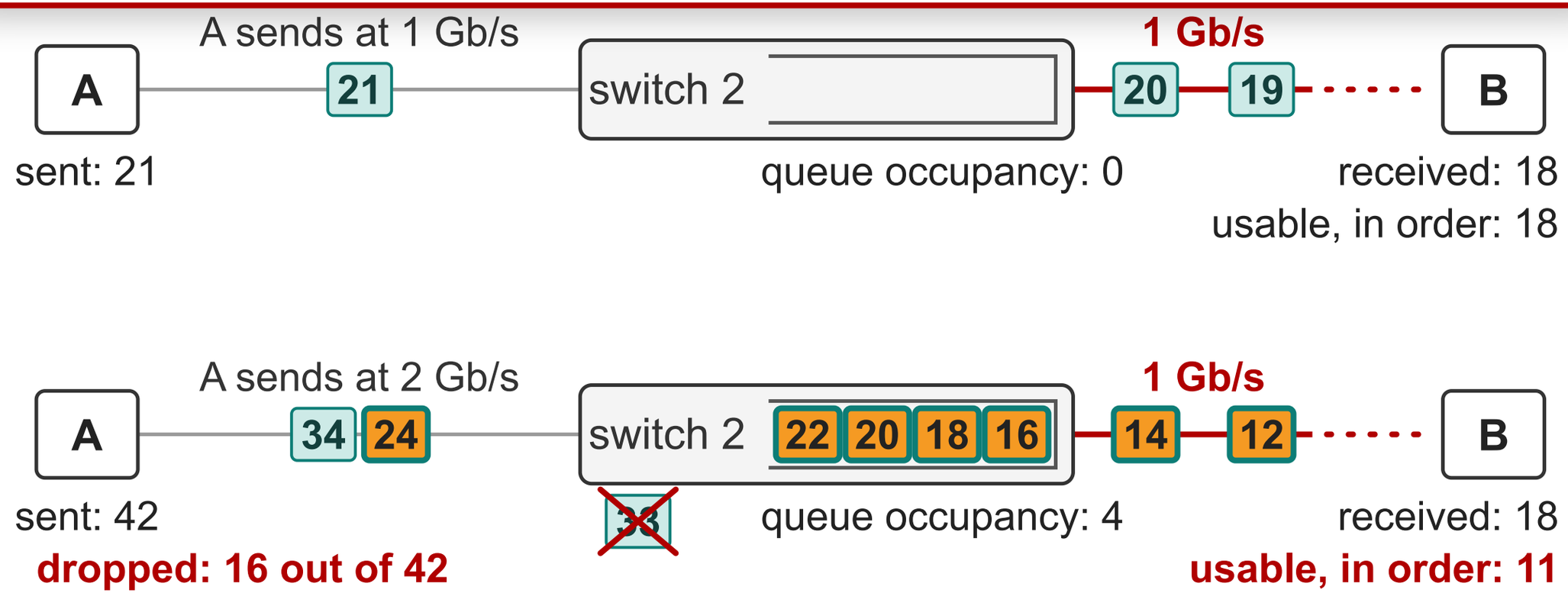
When A finds out a packet was lost (over an RTT later, using info in the acks), A sends it again (orange).



What if A sends faster than the slowest link?

When
(orar
gain

Sending faster does not get more data through. If anything, it adds losses and wasted sends.

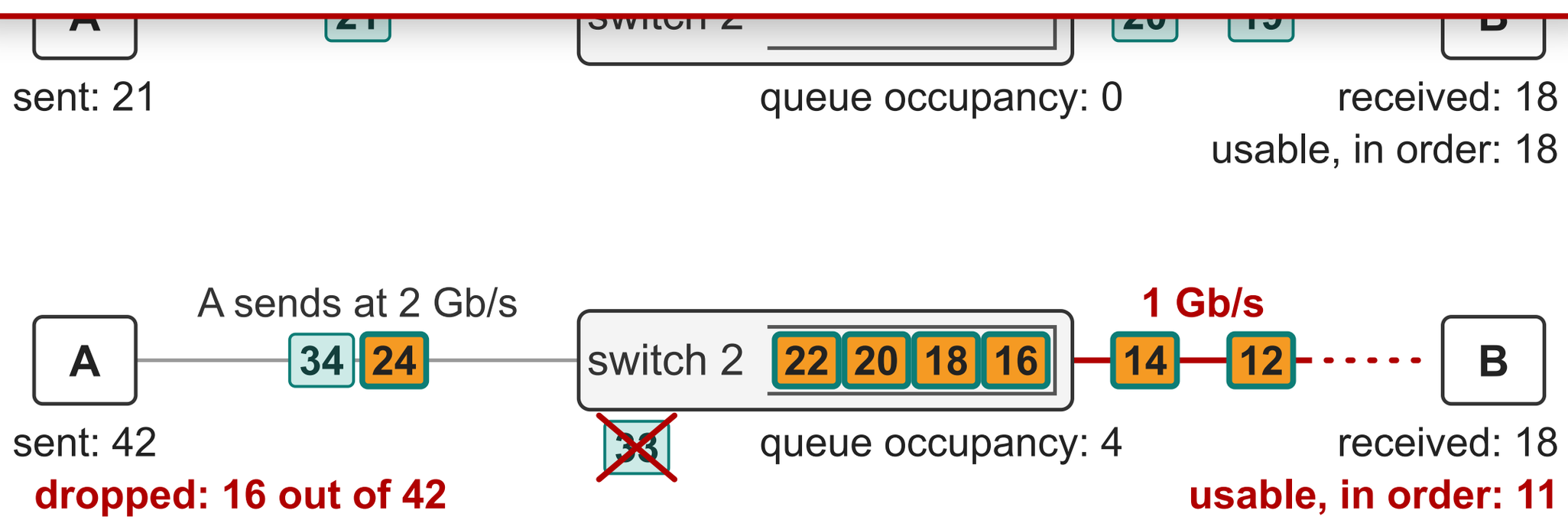


What if A sends faster than the slowest link?

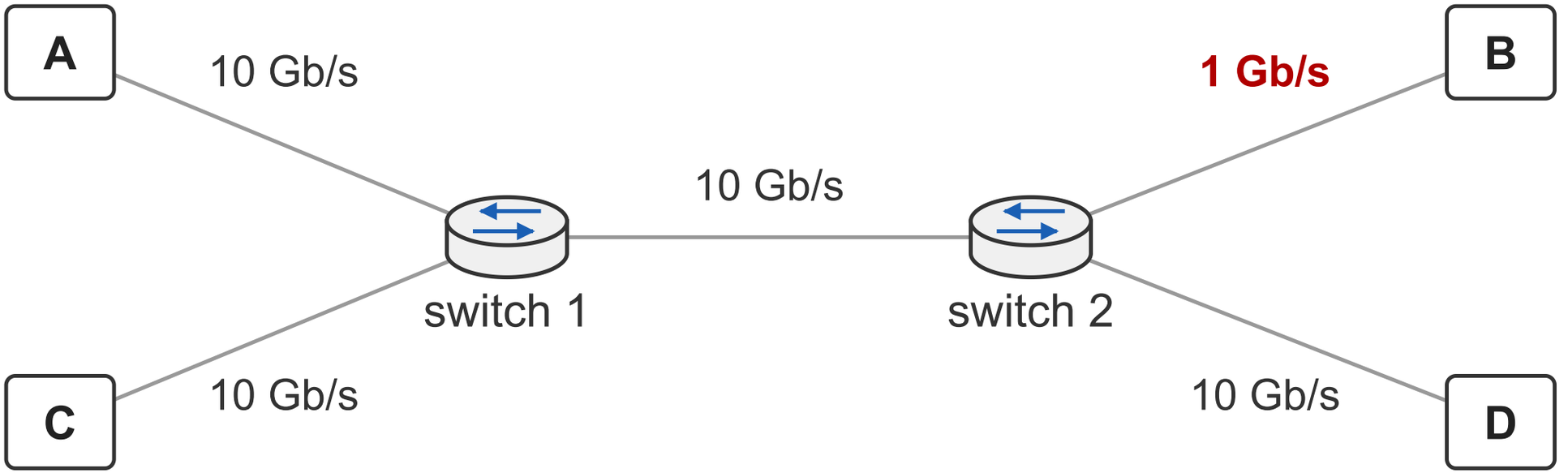
When
(orar
gain

Sending faster does not get more data through. If anything, it adds losses and wasted sends.

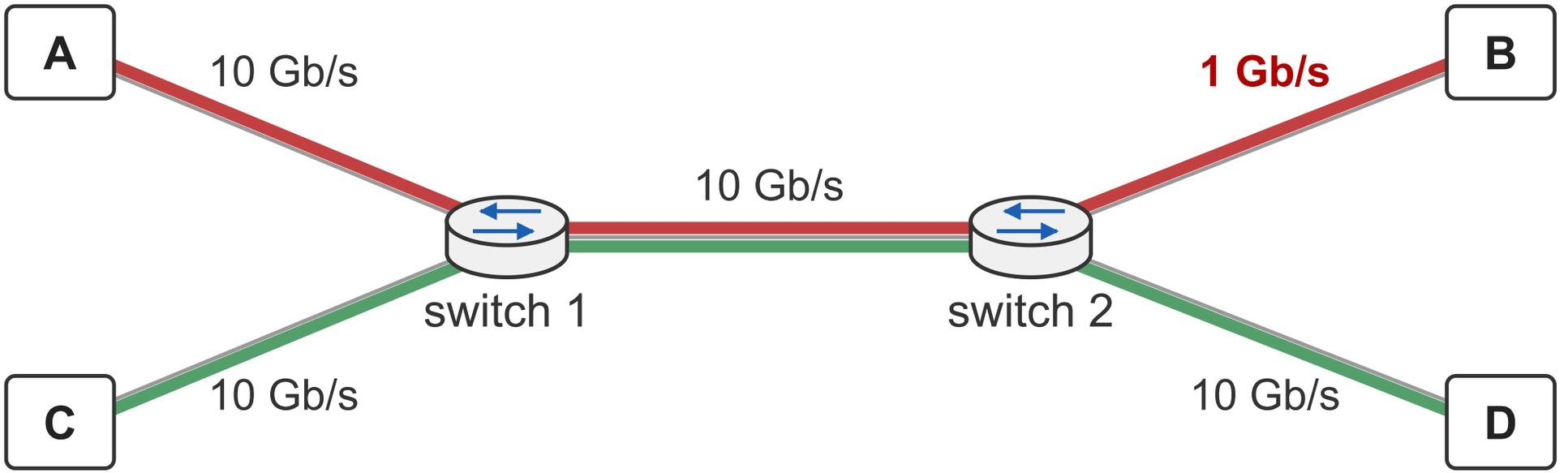
It can have severe consequences on a shared network.



Losses downstream waste upstream work

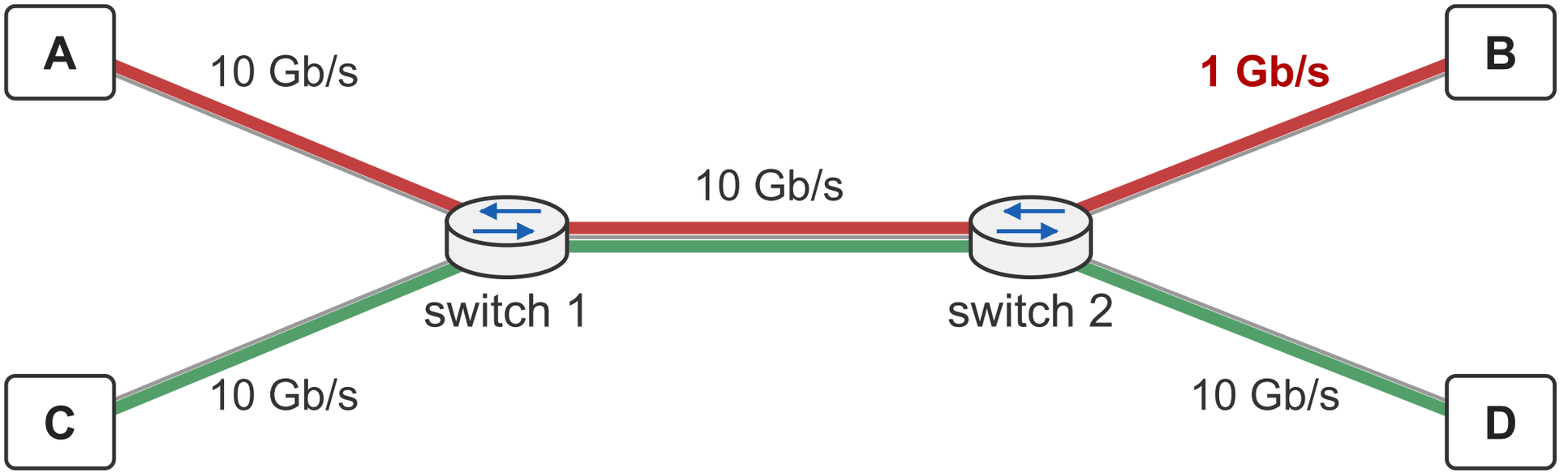


Losses downstream waste upstream work



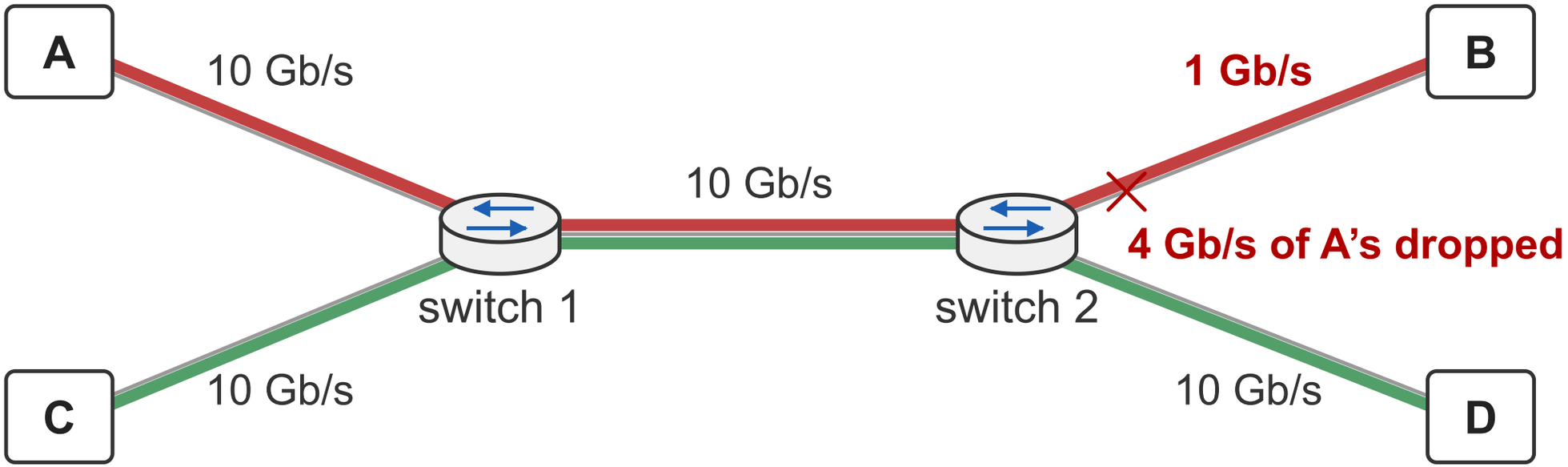
A and C each send as fast as their own link allows: 10 Gb/s

Losses downstream waste upstream work



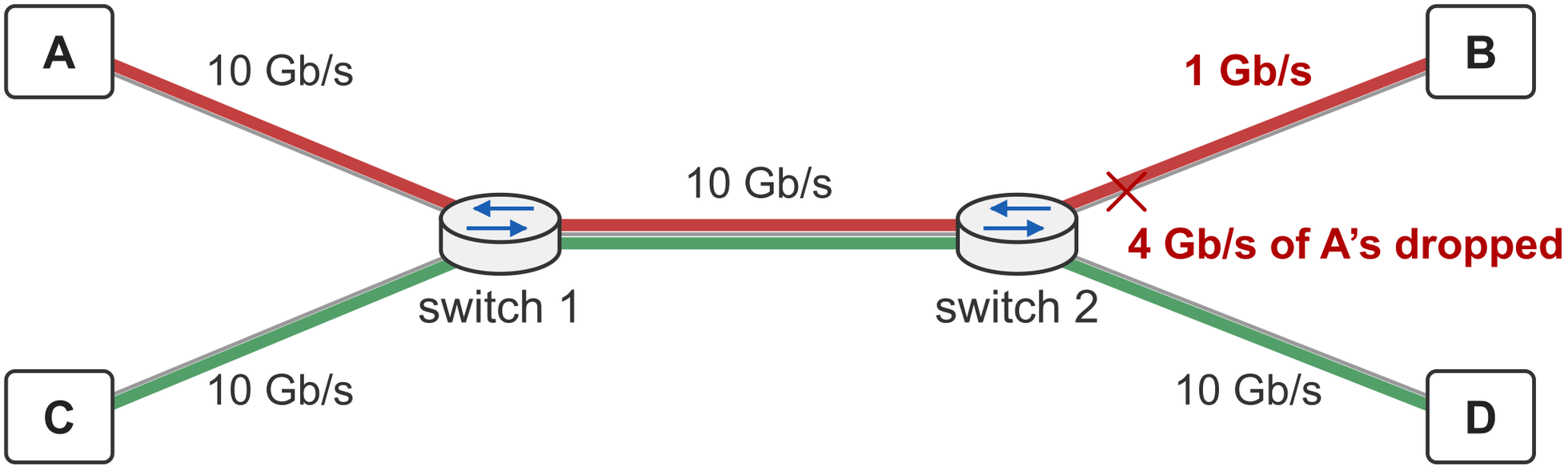
A and C each send as fast as their own link allows: 10 Gb/s
Switch 1 to switch 2 is 10 Gb/s. About 5 Gb/s of each flow gets through

Losses downstream waste upstream work



A and C each send as fast as their own link allows: 10 Gb/s
Switch 1 to switch 2 is 10 Gb/s. About 5 Gb/s of each flow gets through
Switch 2 to B is 1 Gb/s. Out of A's 5 Gb/s, 4 Gb/s is dropped

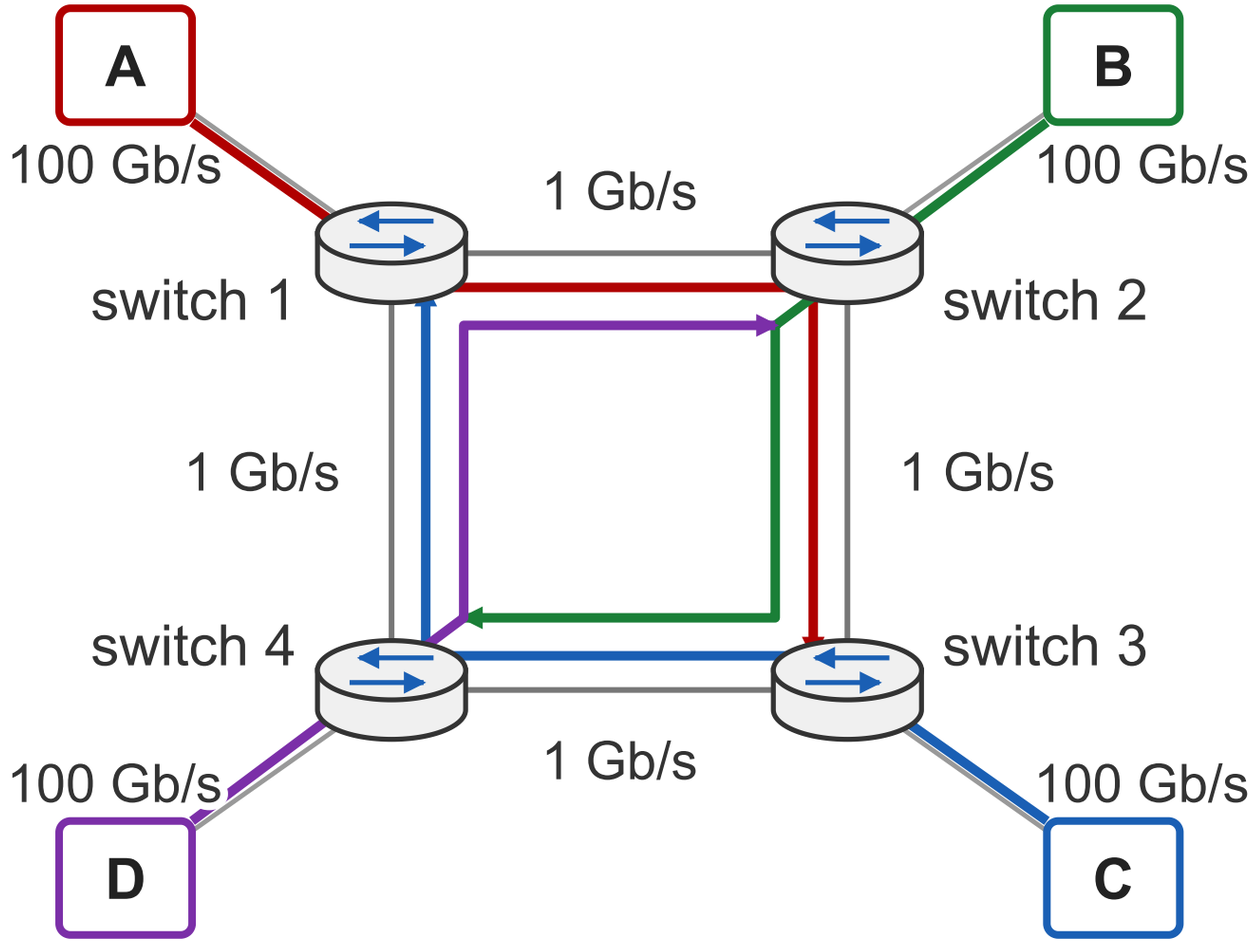
Losses downstream waste upstream work



A and C each send as fast as their own link allows: 10 Gb/s
Switch 1 to switch 2 is 10 Gb/s. About 5 Gb/s of each flow gets through
Switch 2 to B is 1 Gb/s. Out of A's 5 Gb/s, 4 Gb/s is dropped
B gets 1 Gb/s and D gets 5 Gb/s: 6 Gb/s total
Had A sent 1 Gb/s from the beginning, the total could have been 1 + 9 = 10 Gb/s

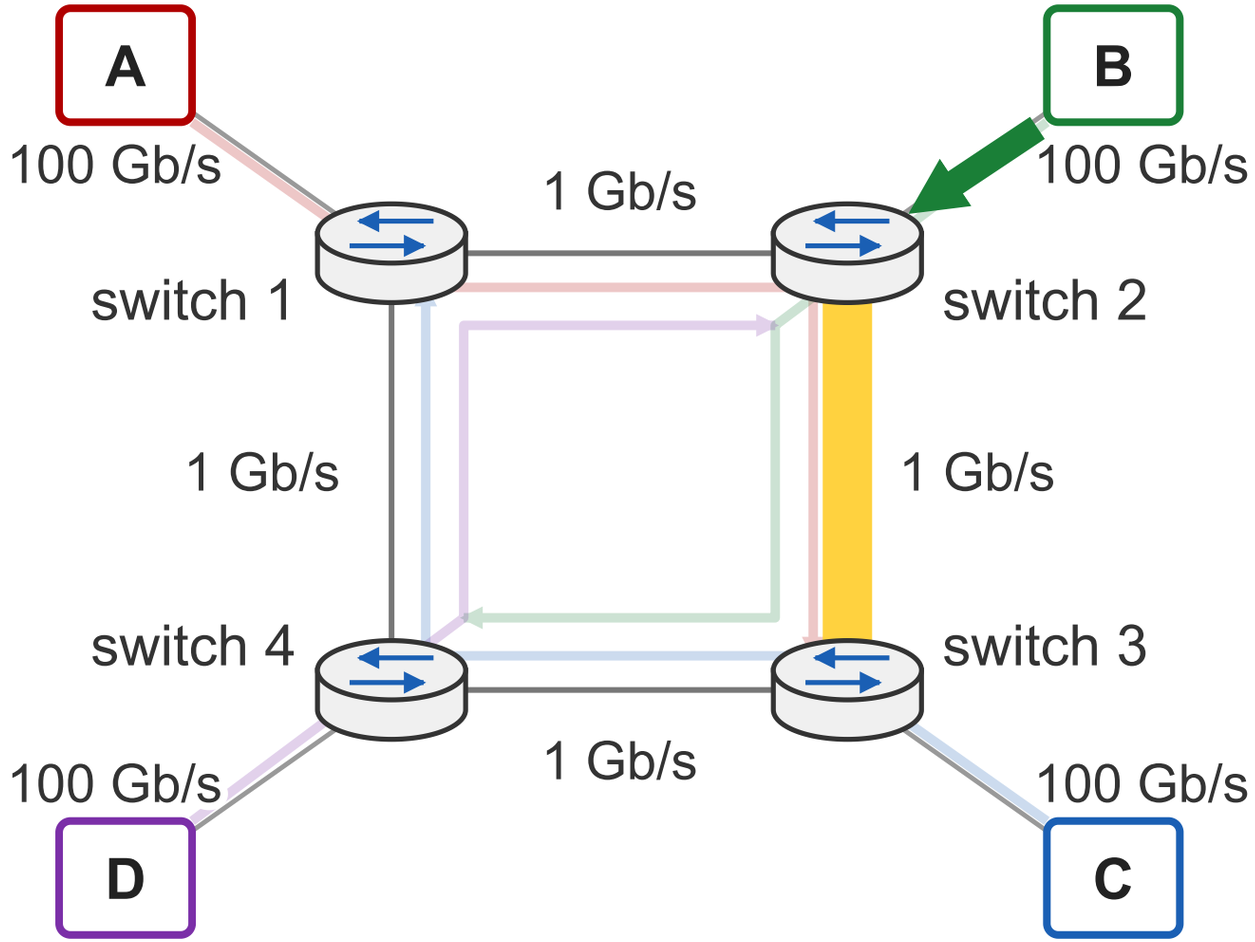
Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.



Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.

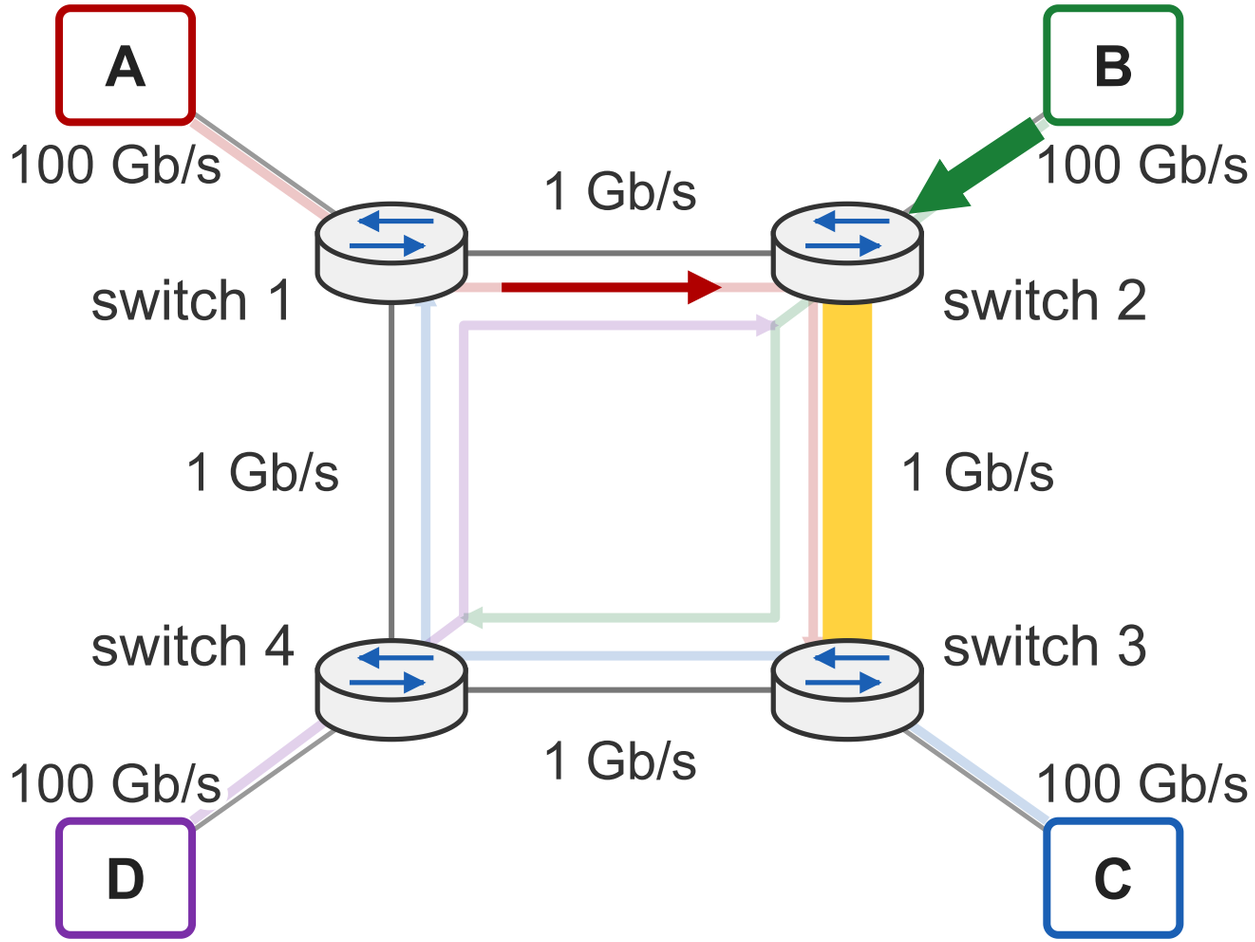


Consider switch 2's queue for the link to switch 3:

- B is sending 100 Gb/s into the queue

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.

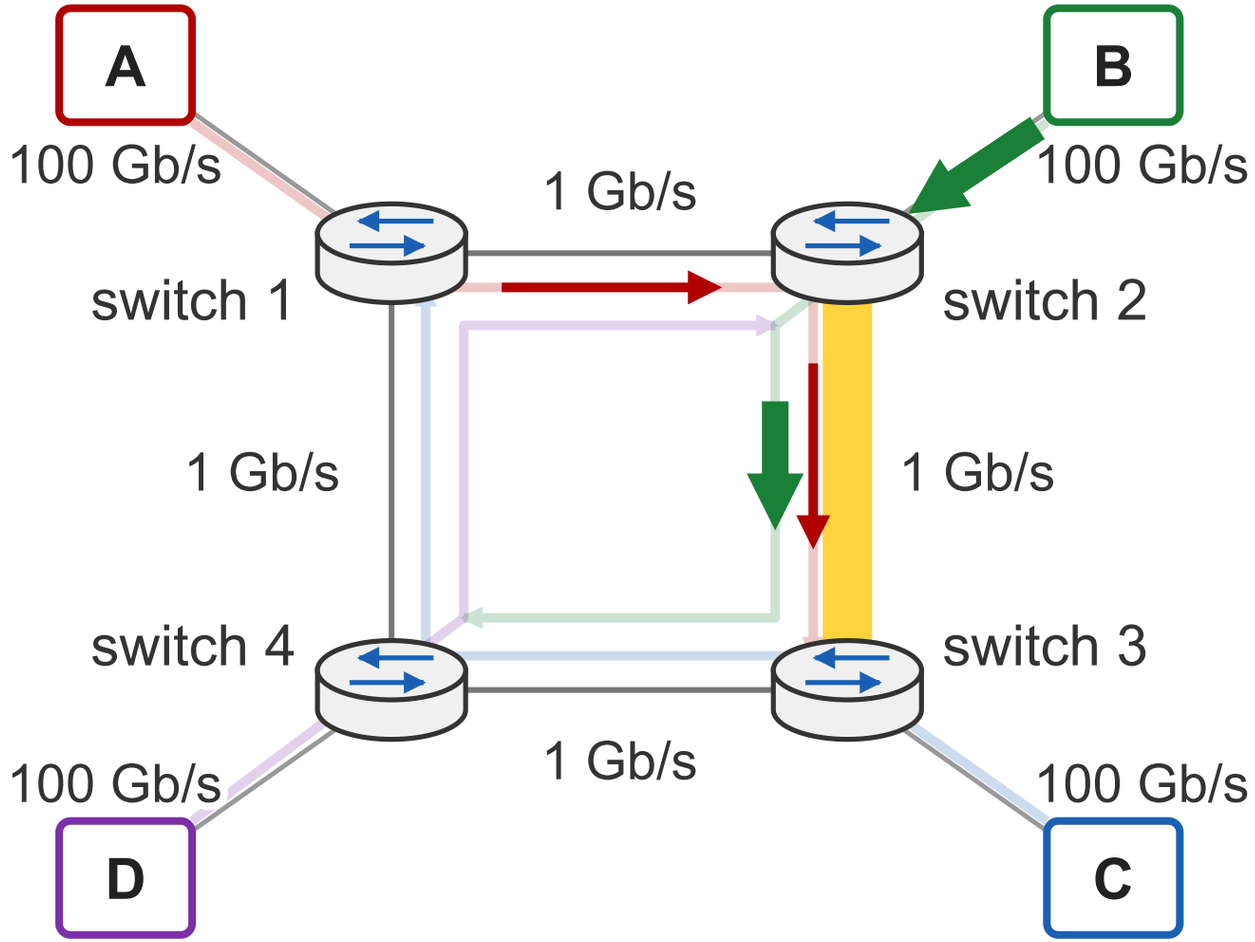


Consider switch 2's queue for the link to switch 3:

- B is sending 100 Gb/s into the queue
- A is sending $X \leq 1$ Gb/s into the queue

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.

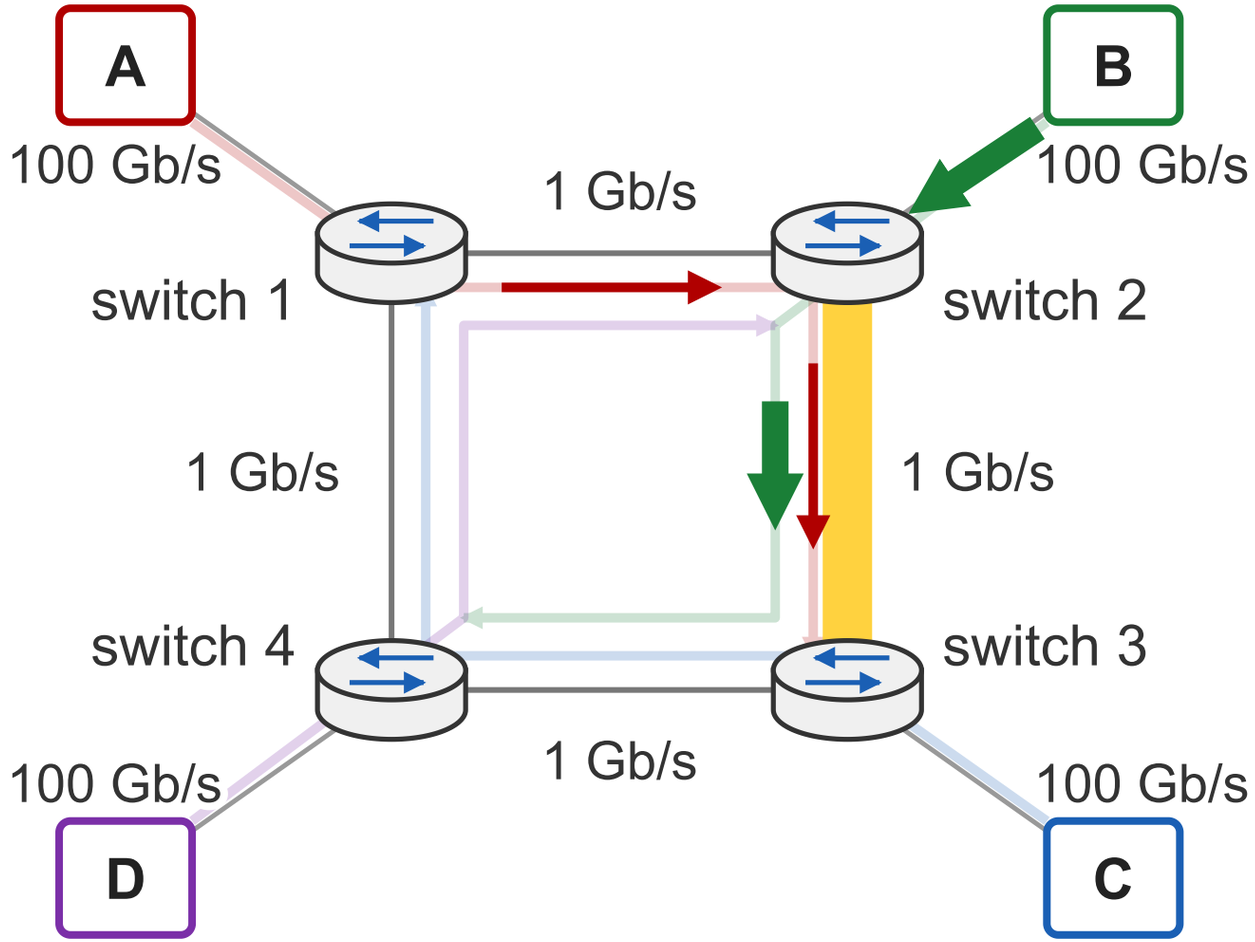


Consider switch 2's queue for the link to switch 3:

- B is sending 100 Gb/s into the queue
- A is sending $X \leq 1$ Gb/s into the queue
- A's share = $X / (X + 100)$
- B's share = $100 / (X + 100)$

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.

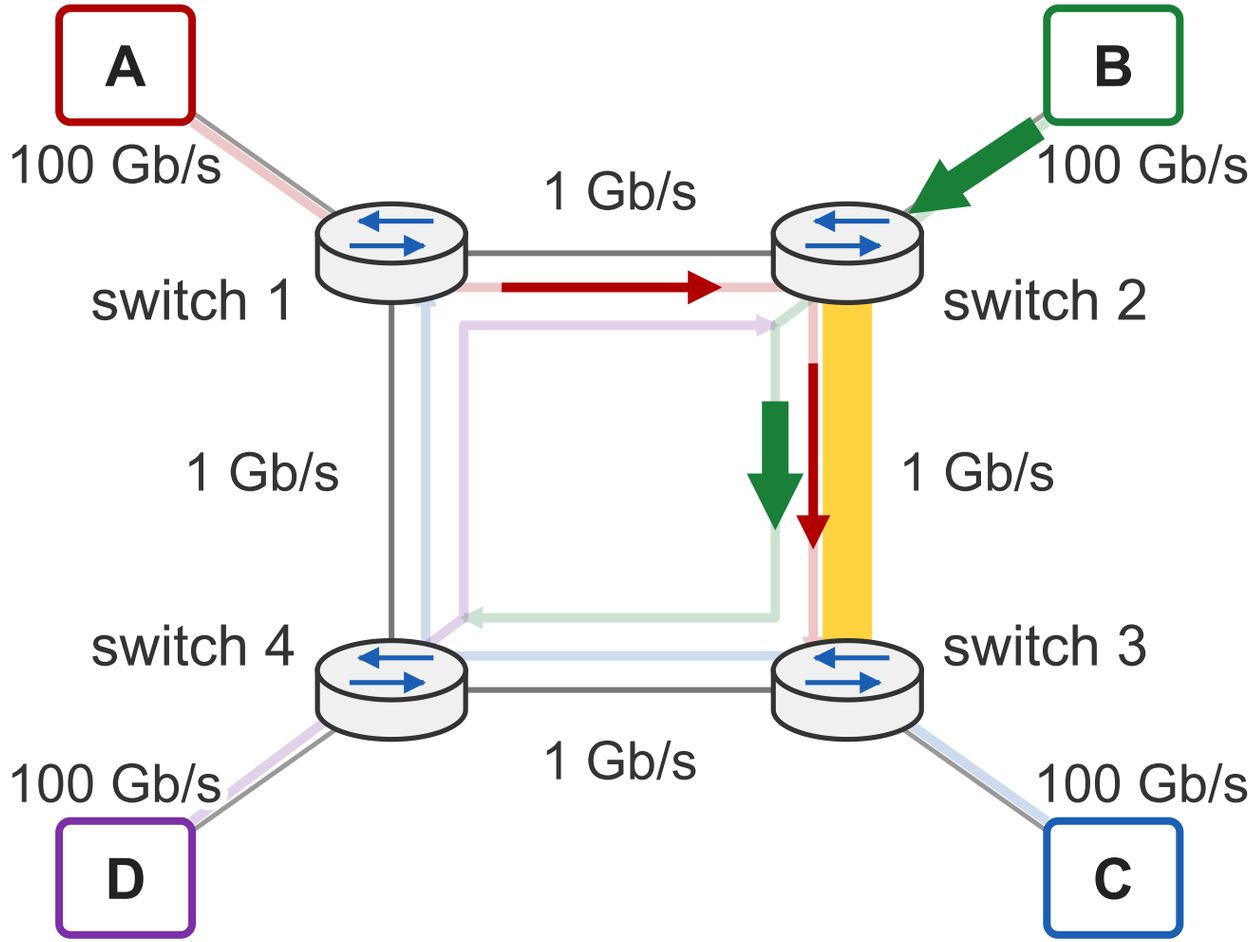


Consider switch 2's queue for the link to switch 3:

- B is sending 100 Gb/s into the queue
- A is sending $X \leq 1$ Gb/s into the queue
- A's share = $X / (X + 100)$
- B's share = $100 / (X + 100)$
- B is getting most of the 1 Gb/s

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.



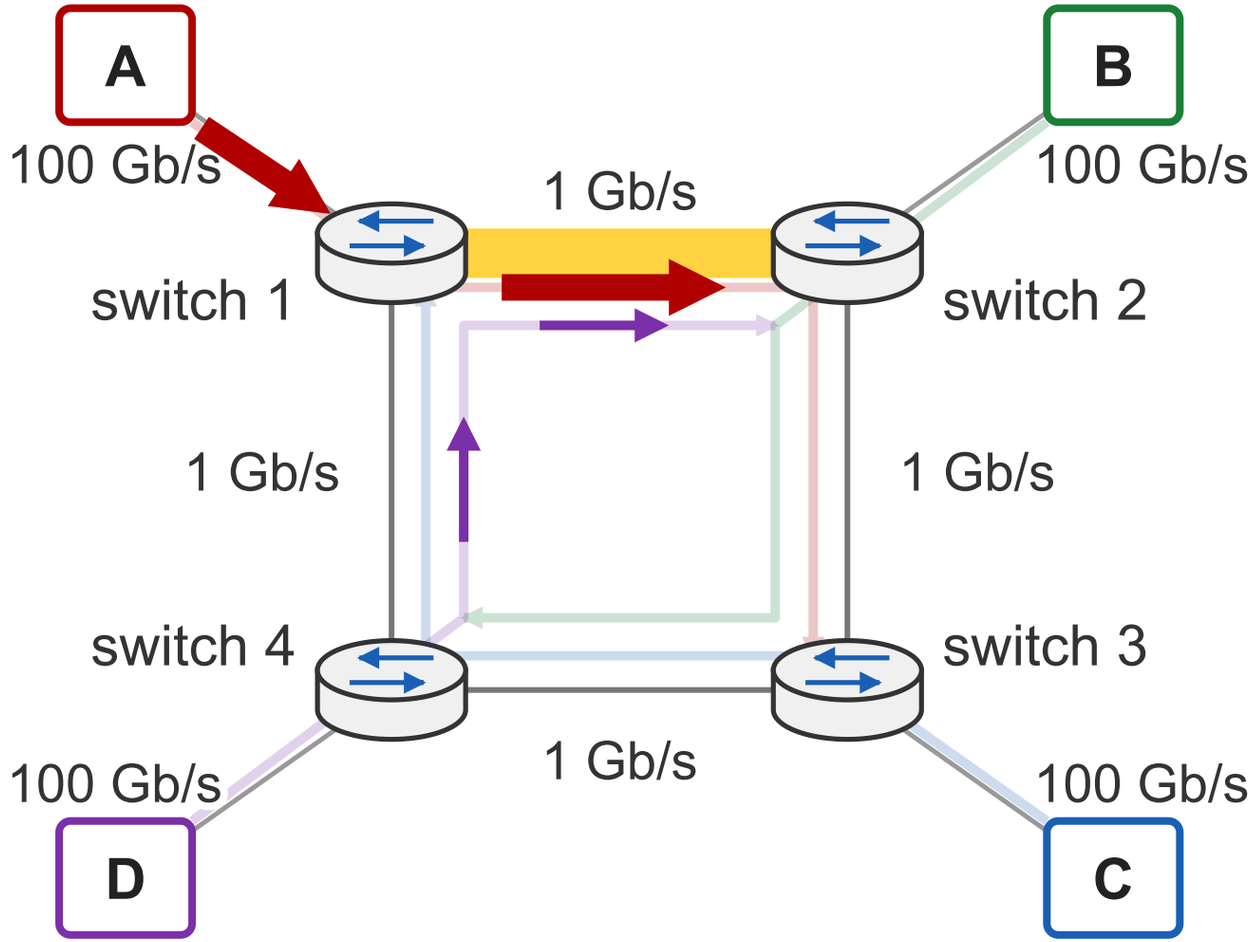
Consider switch 2's queue for the link to switch 3:

- B is sending 100 Gb/s into the queue
- A is sending X ≤ 1 Gb/s into the queue
- A's share = X / (X + 100)
- B's share = 100 / (X + 100)
- B is getting most of the 1 Gb/s

What is X?

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.

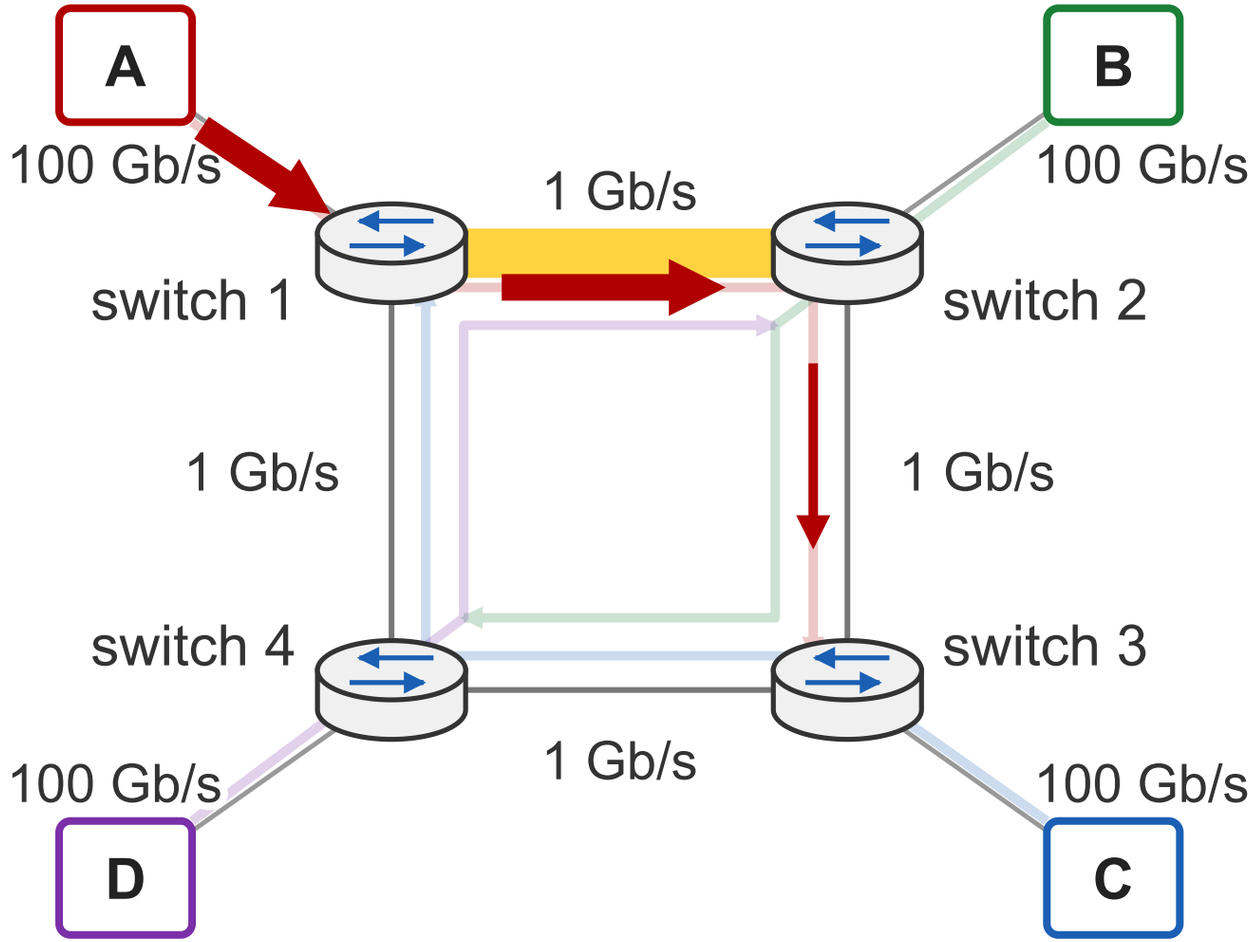


At switch 1, A is in B's place:

- A sends 100 Gb/s
- D's packets at most 1 Gb/s
- A's traffic is getting most of the 1 Gb/s
- So, X is almost 1 Gb/s

Congestion collapse

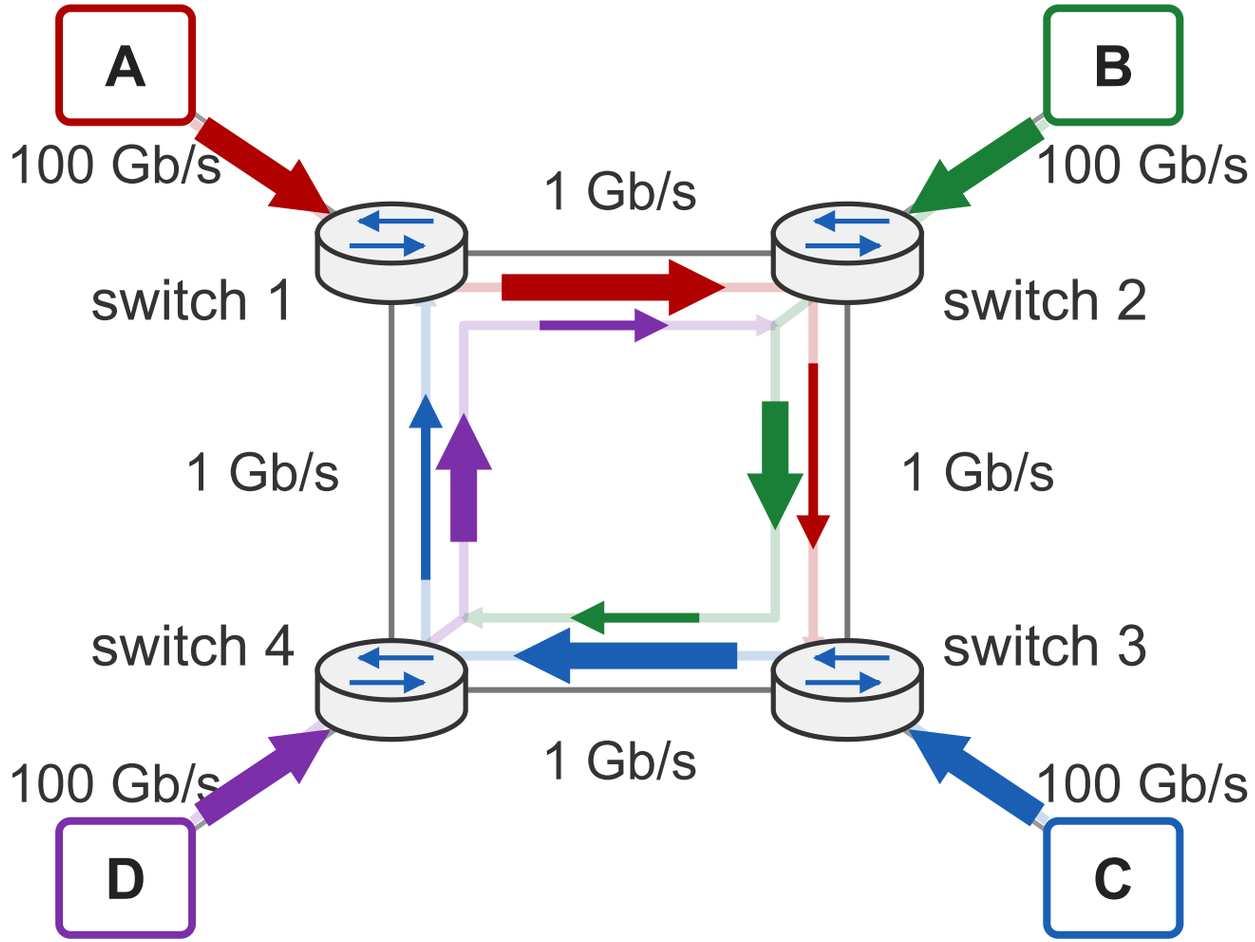
Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.



- So, A is taking most of the link from switch 1 to switch 2
- It lets almost nothing of D's packets through
- But then, almost all of its own packets are dropped at switch 2
  - Because B doesn't let A's packets through, similar to how A didn't let D's packets through

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.

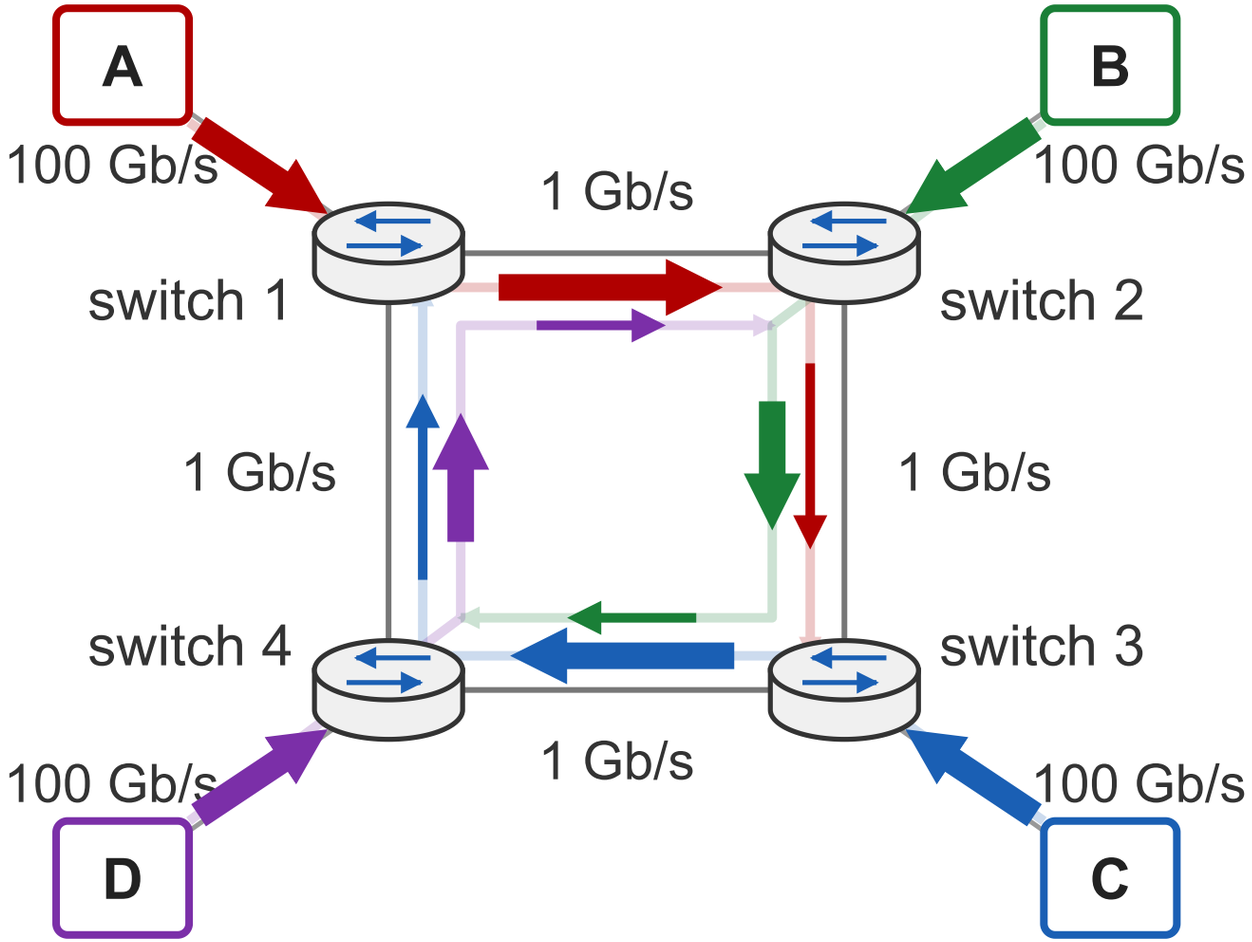


This is the same for every flow:

- Takes most of the bottleneck at its first hop
- Crowds out the other flow on that link
- Then, it is mostly dropped at the next hop

Congestion collapse

Each host sends to the opposite corner (A → C, B → D, C → A, D → B), as fast as it can.



This is the same for every flow:

- Takes most of the bottleneck at its first hop
- Crowds out the other flow on that link
- Then, it is mostly dropped at the next hop

The network links are always busy, but they are not actually delivering data to the end points

How could this have been better? (in-class)

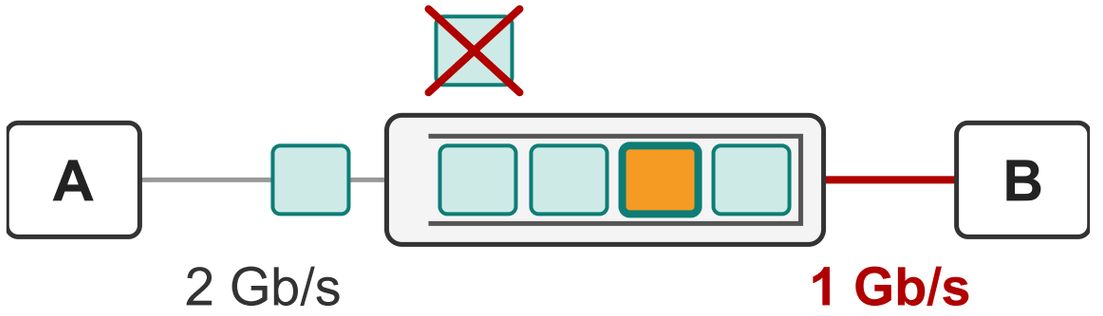
(in-class discussion)

- How much would you have each host send instead?

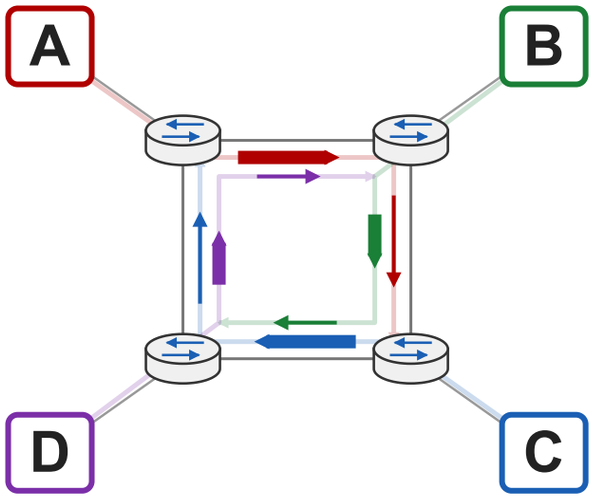
Reset

Recap so far: send only as much as your share of the bottleneck link allows

Only flow on the path: sending faster than the bottleneck link does not get more data through. If anything, it adds losses and wasted sends.

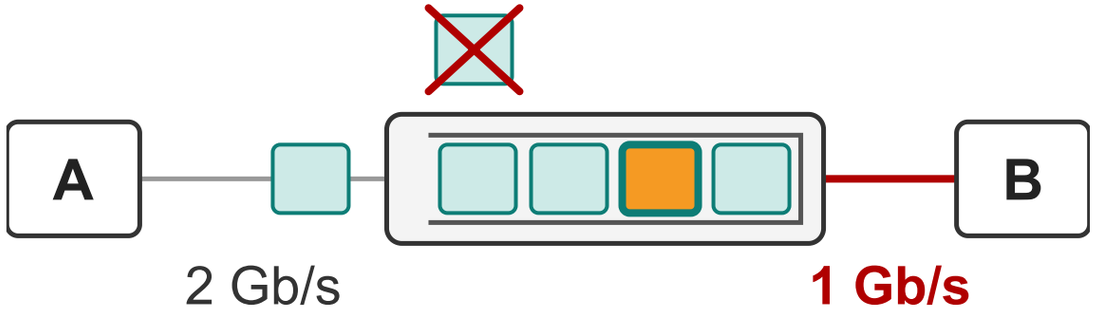


Multiple flows sharing bottleneck links: the network can be busy and still deliver almost nothing (congestion collapse) if flows do not adjust to their share.



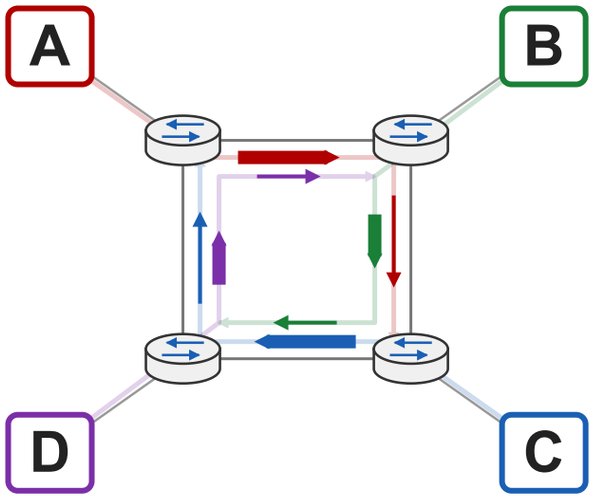
Recap so far: send only as much as your share of the bottleneck link allows

Only flow on the path: sending faster than the bottleneck link does not get more data through. If anything, it adds losses and wasted sends.

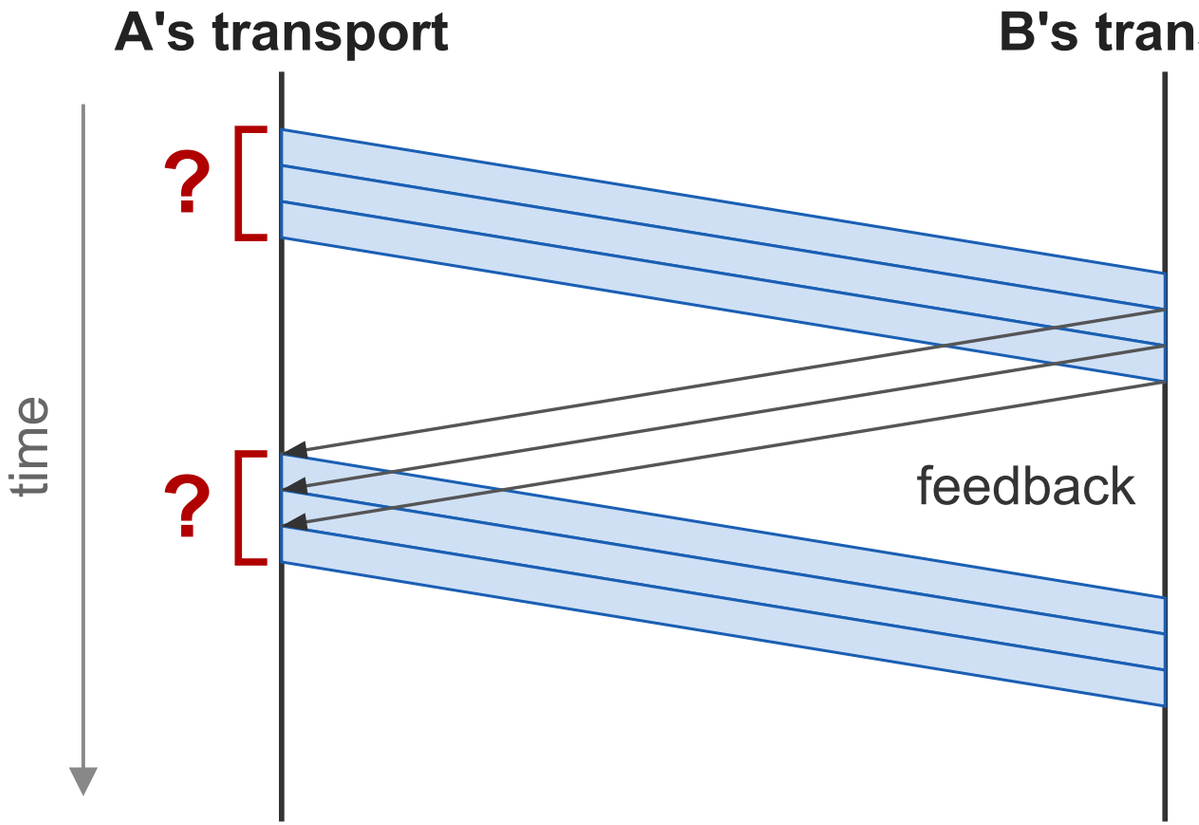


Multiple flows sharing bottleneck links: the network can be busy and still deliver almost nothing (congestion collapse) if flows do not adjust to their share.

One common way: bound the number of in-flight bytes

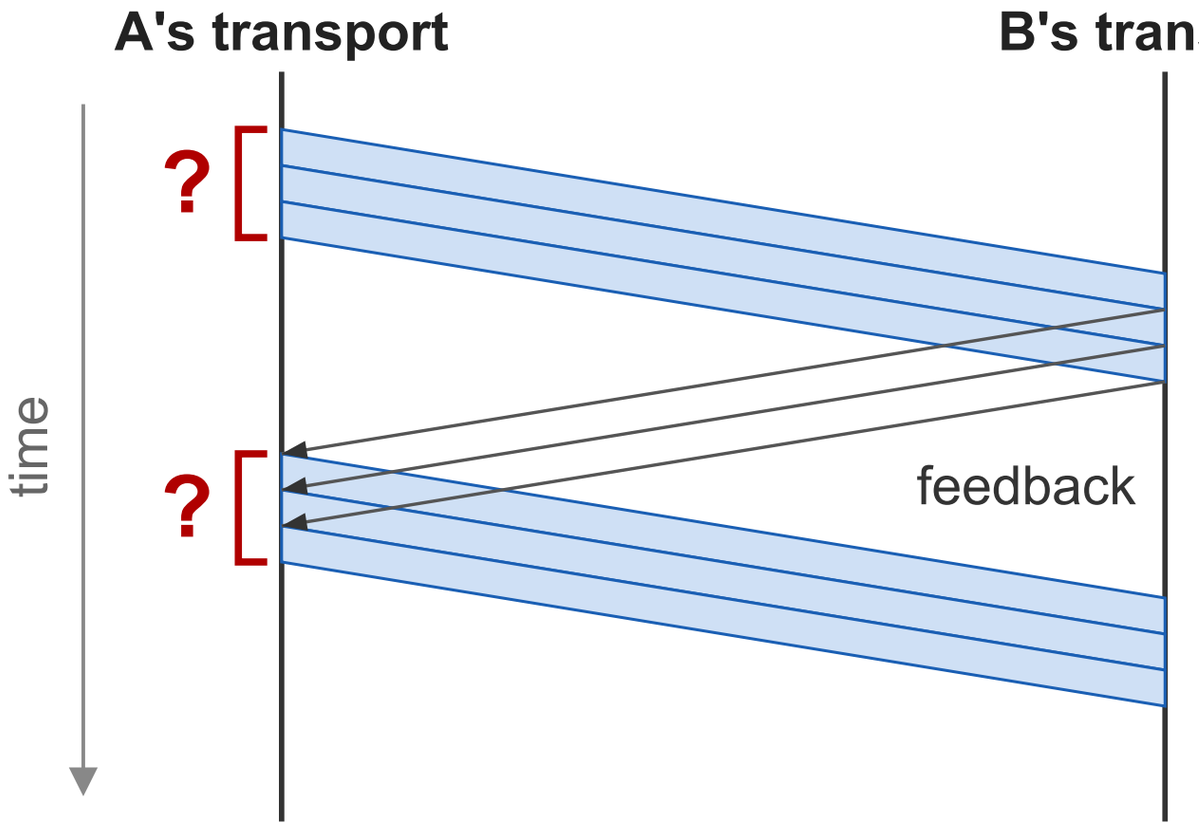


Back to the batch size



A sends a batch back to back, without waiting for feedback in between: one batch every round trip.

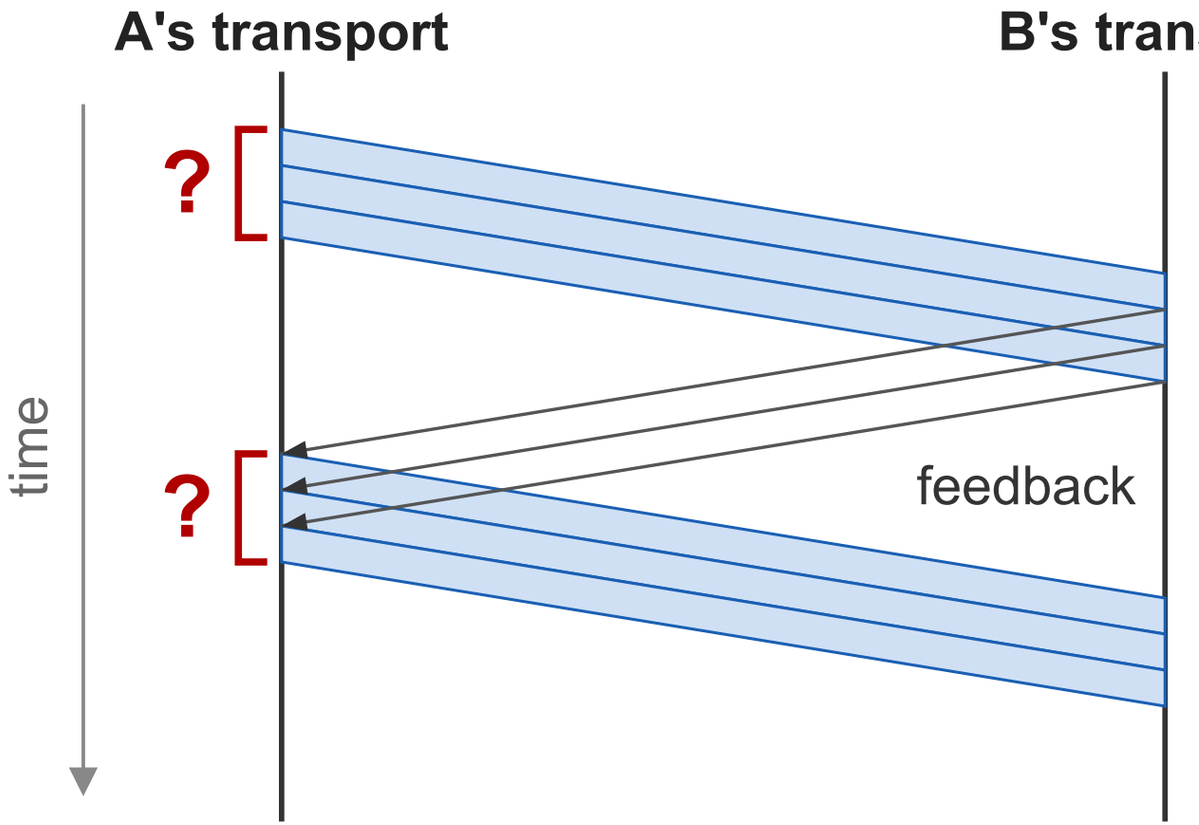
Back to the batch size



A sends a batch back to back, without waiting for feedback in between: one batch every round trip.

Sending rate ≈ batch size / RTT

Back to the batch size

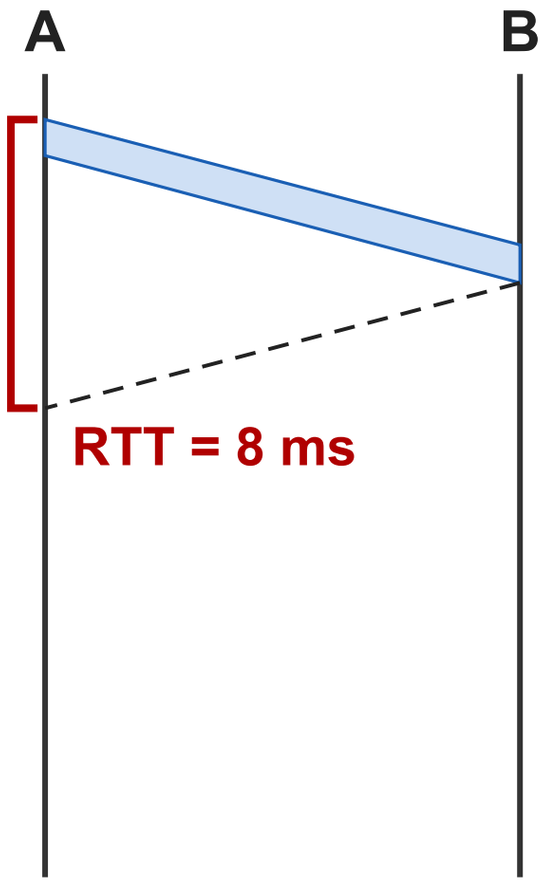


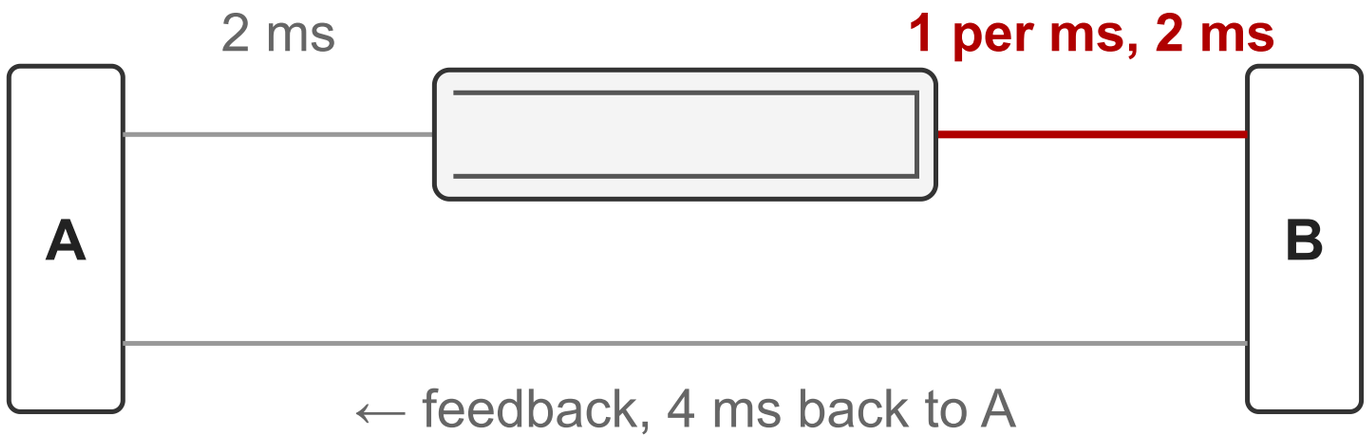
A sends a batch back to back, without waiting for feedback in between: one batch every round trip.

Sending rate ≈ batch size / RTT

So what is the right batch size?

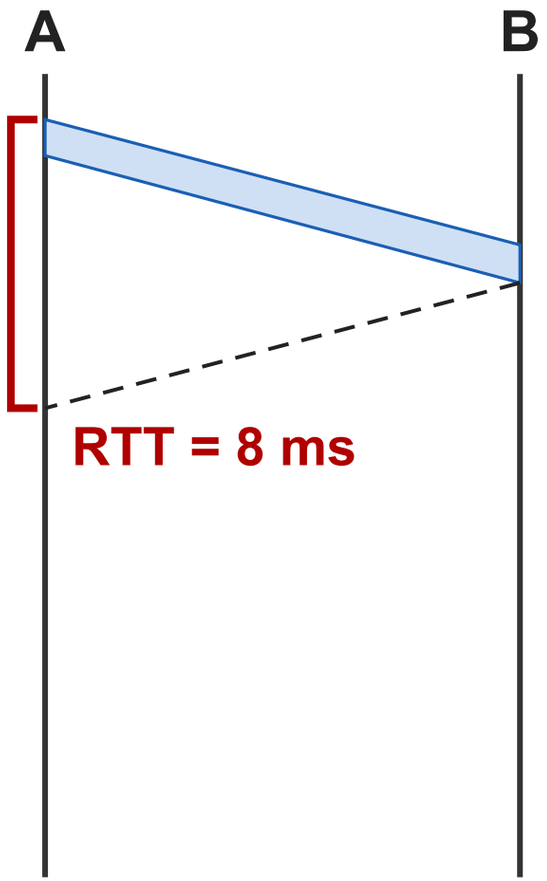
Bounding the number of in-flight bytes

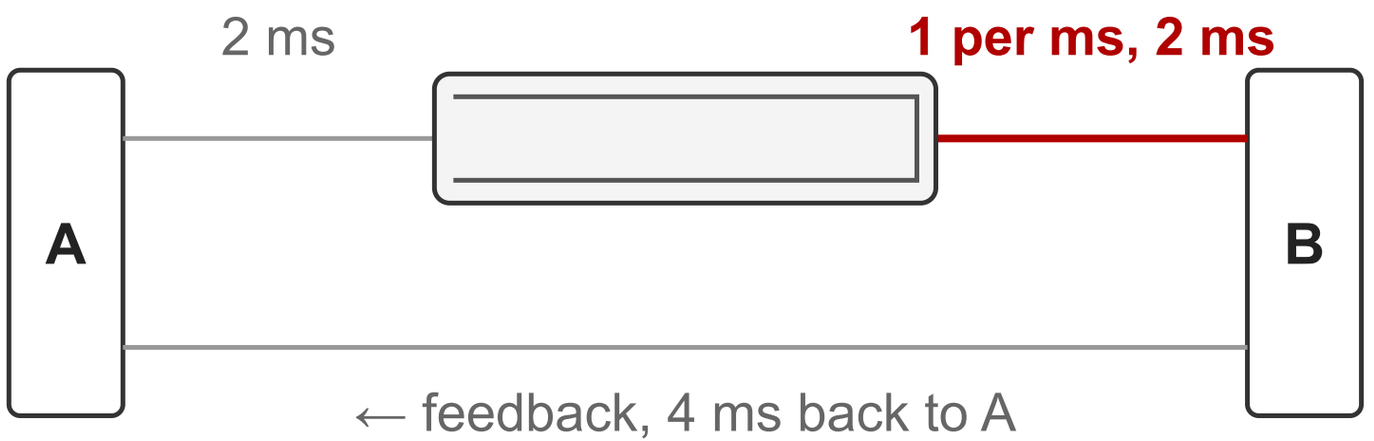




Each packet is 1,500 bytes. One packet and its feedback take one RTT: 8 ms, with nothing waiting in the queue.

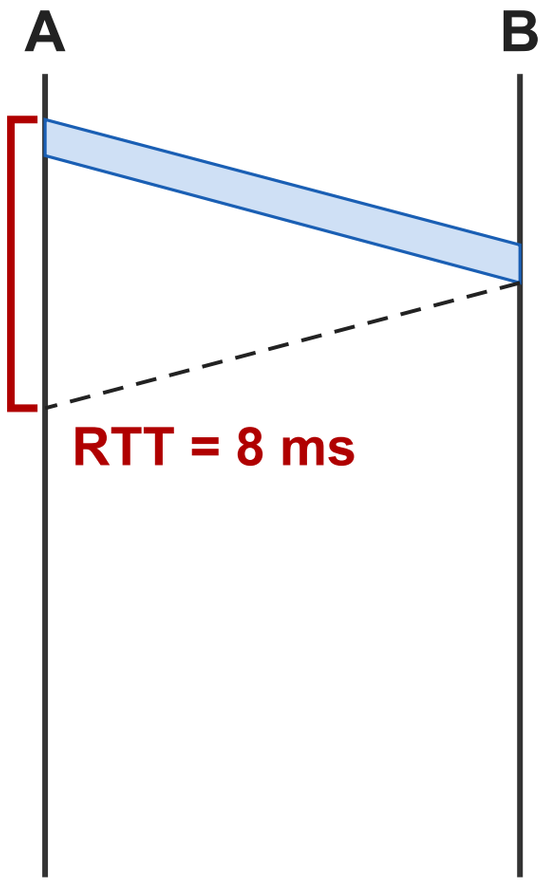
Bounding the number of in-flight bytes

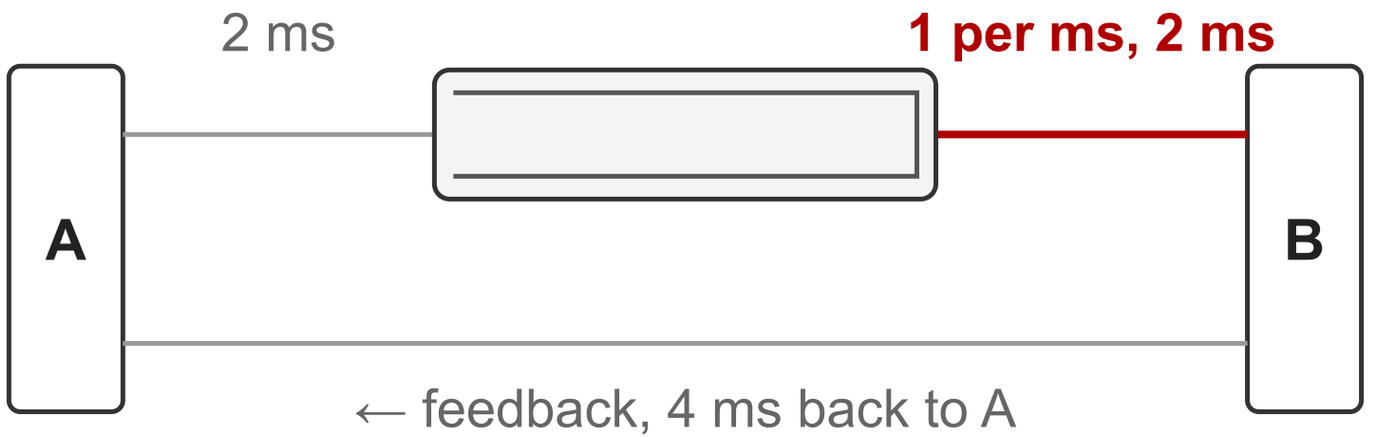




How many bytes could the bottleneck send in one RTT?

Bounding the number of in-flight bytes

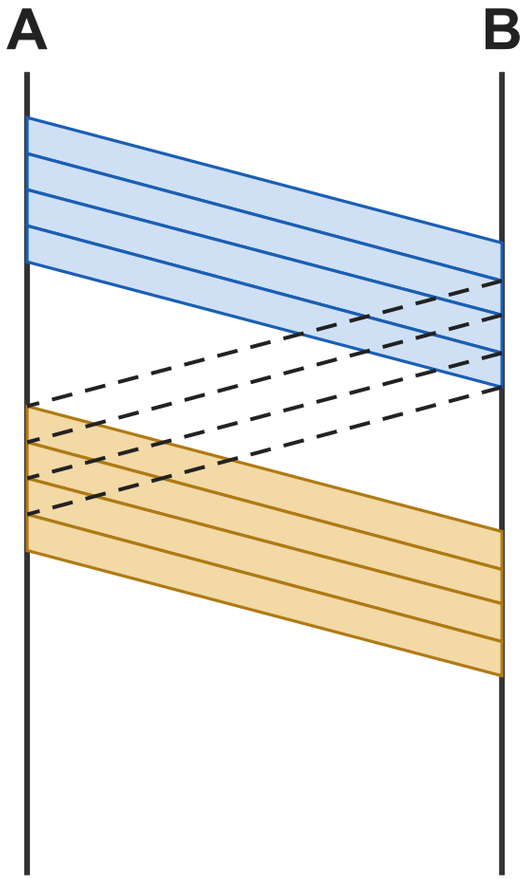


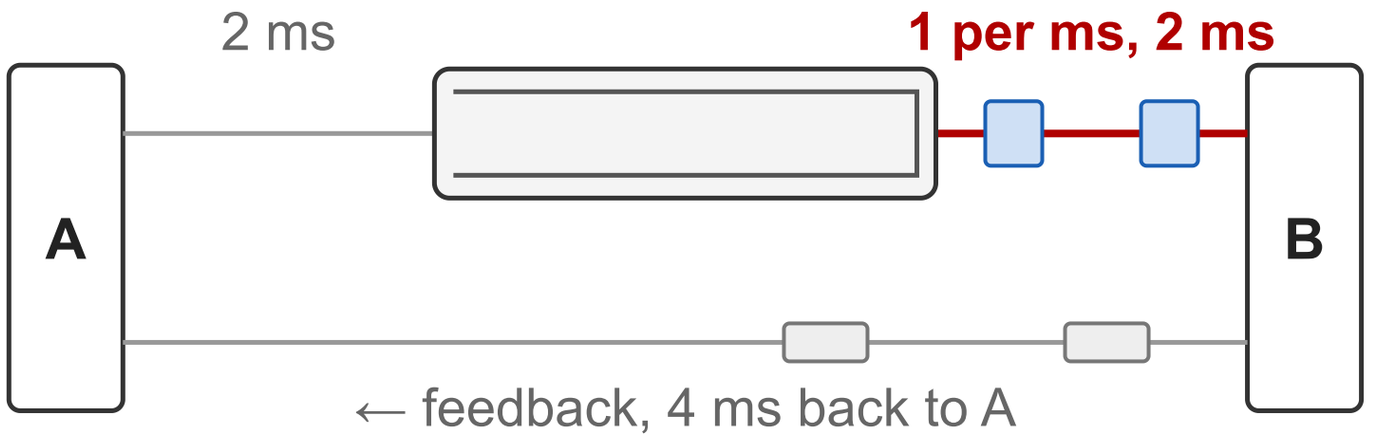


How many bytes could the bottleneck send in one RTT?

1 packet per ms × 8 ms = 8 packets = 12,000 bytes: the bandwidth-delay product (BDP)

Bounding the number of in-flight bytes

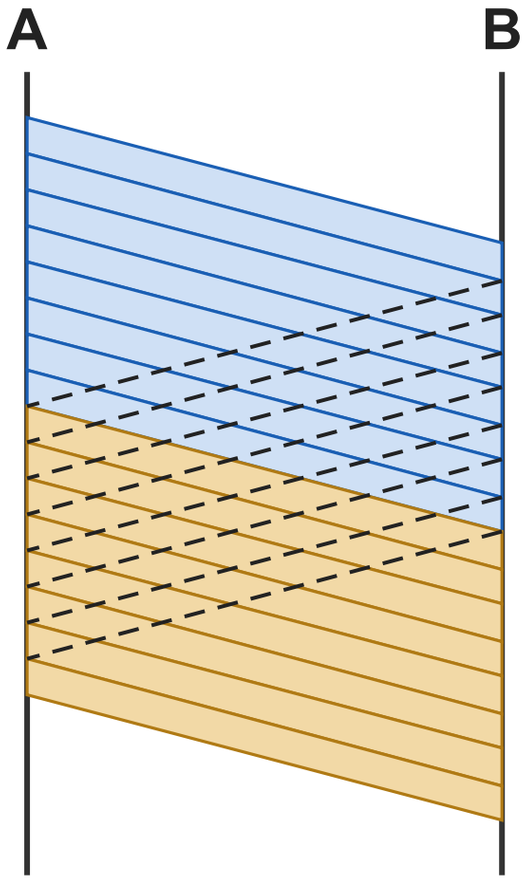


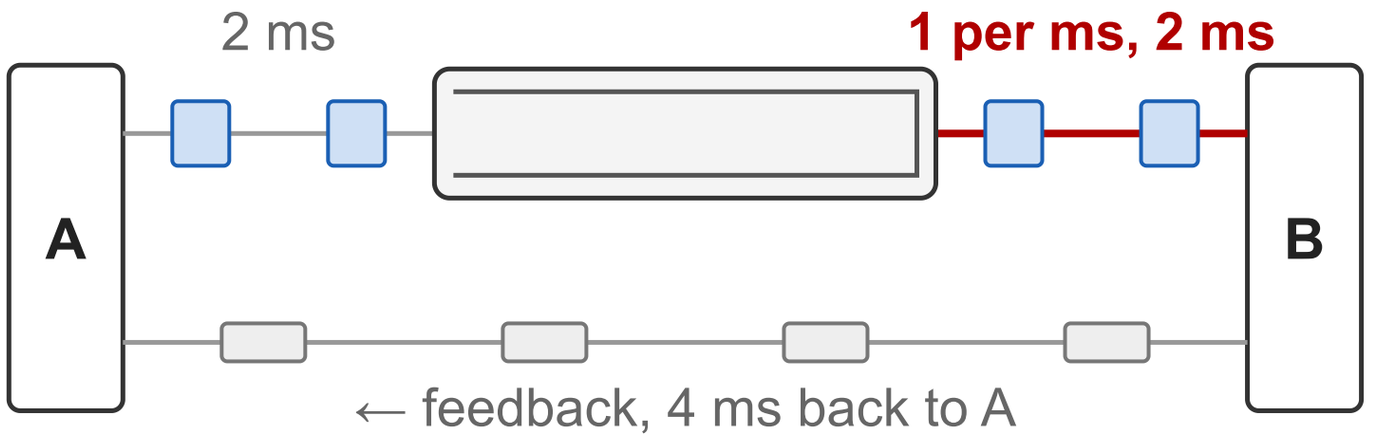


Bound 6,000 bytes (4 packets): below the BDP

A sends 4 back to back, then waits for the first feedback. B receives 4 packets, then sits idle for 4 ms.

Bounding the number of in-flight bytes

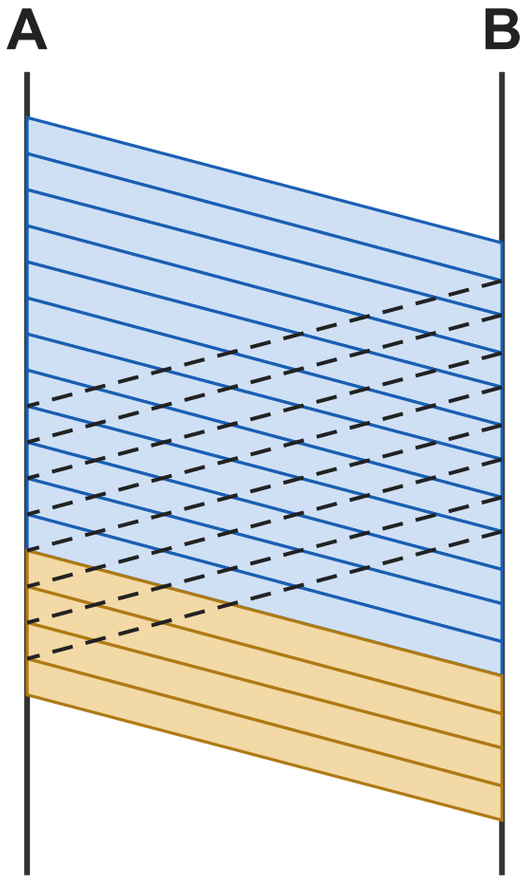


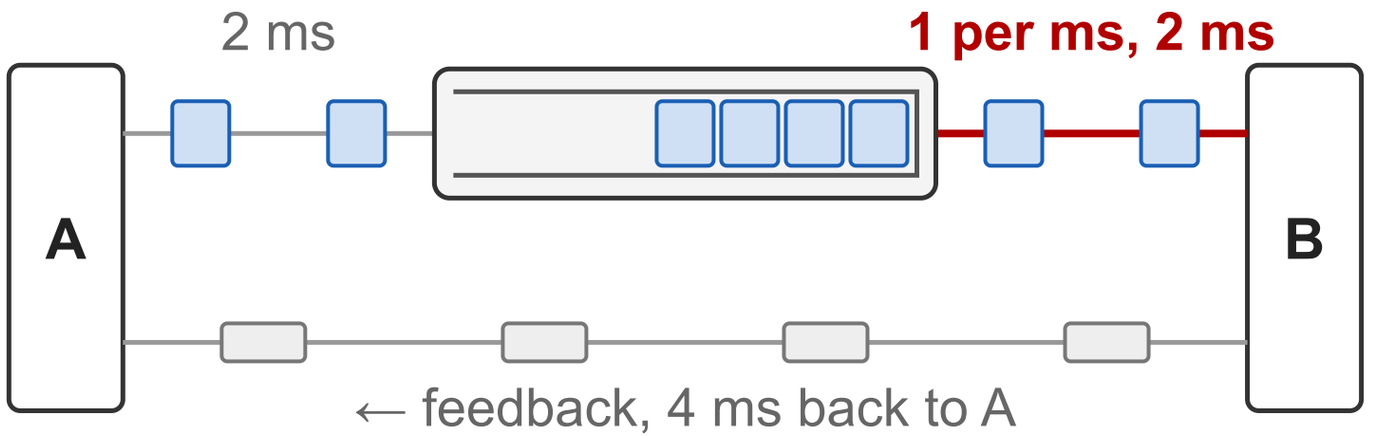


Bound 12,000 bytes (8 packets): the BDP

The round trip is full, the queue empty.
B continuously receives packets, and
there is no idle time.

Bounding the number of in-flight bytes

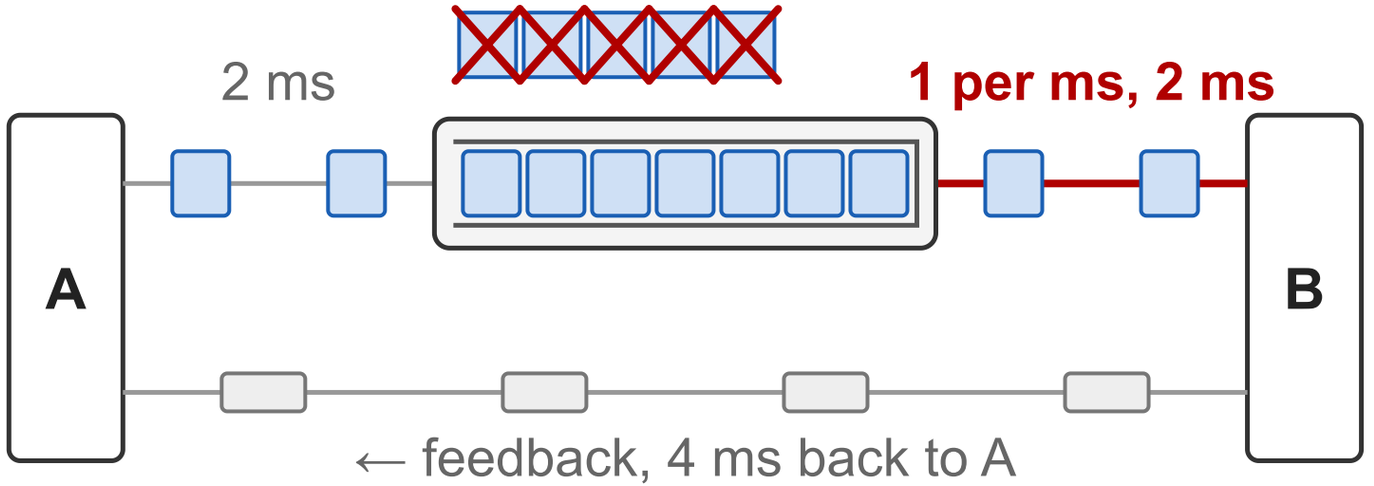




Bound 18,000 bytes (12 packets): above the BDP

The round trip is full **and** four packets always wait in the queue. B still gets one every 1 ms, with no idle time. But the round-trip time is now 12 ms.

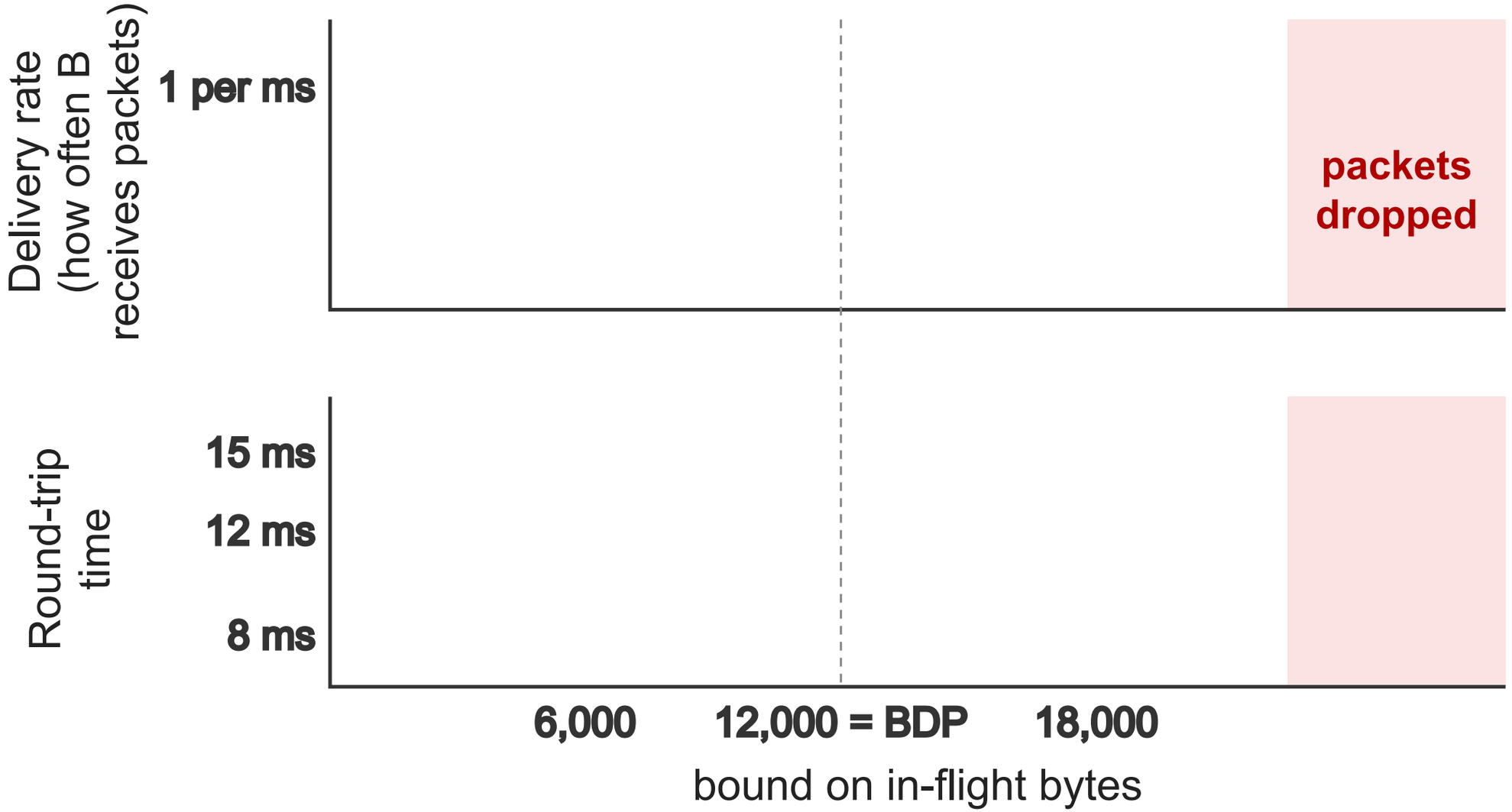
Bounding the number of in-flight bytes



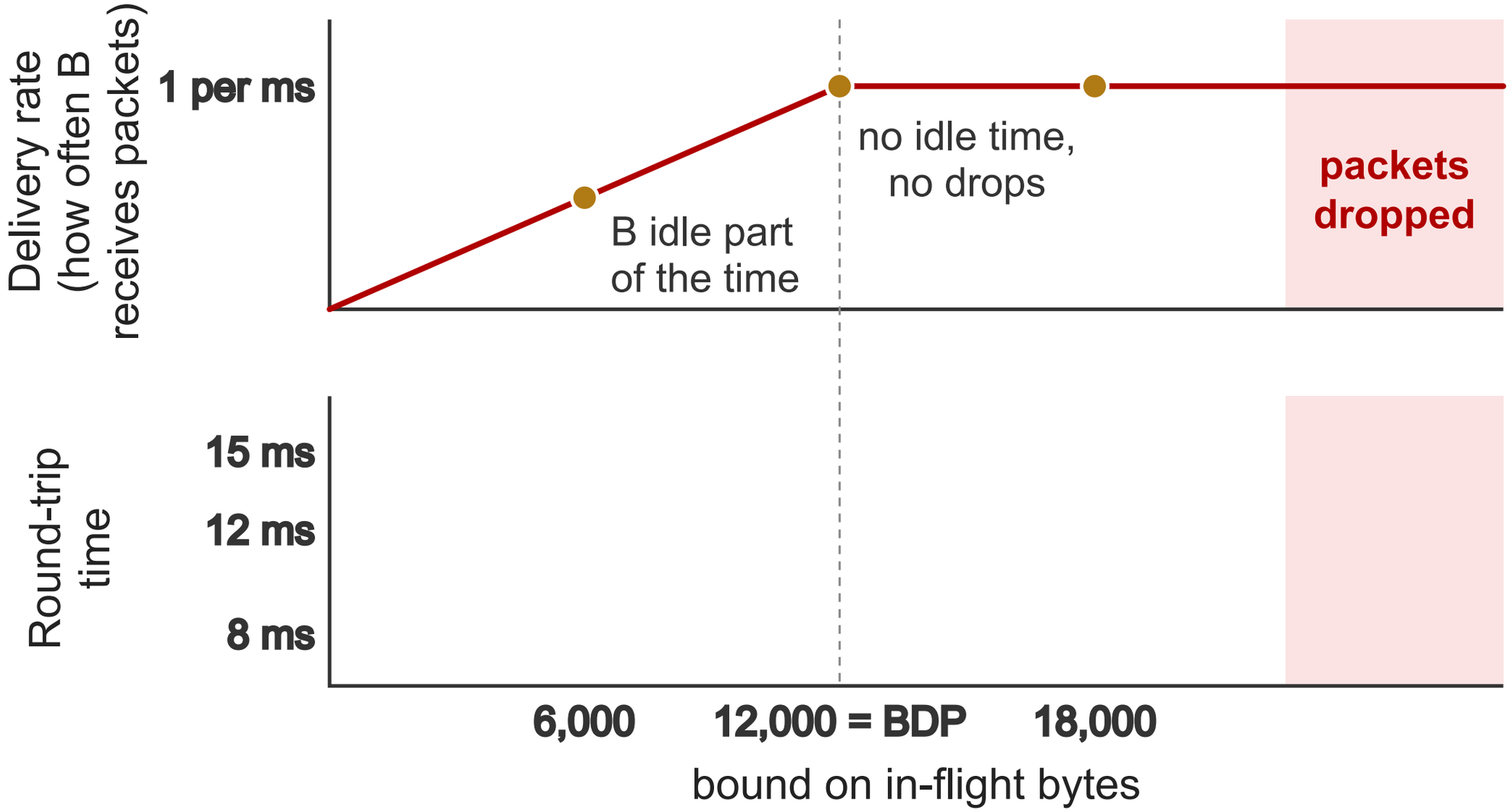
Bound 30,000 bytes (20 packets): far above the BDP

Round trip full, queue full (7), 5 dropped every round trip. B still gets one every 1 ms, but some are retransmissions. The round-trip time is now 15 ms.

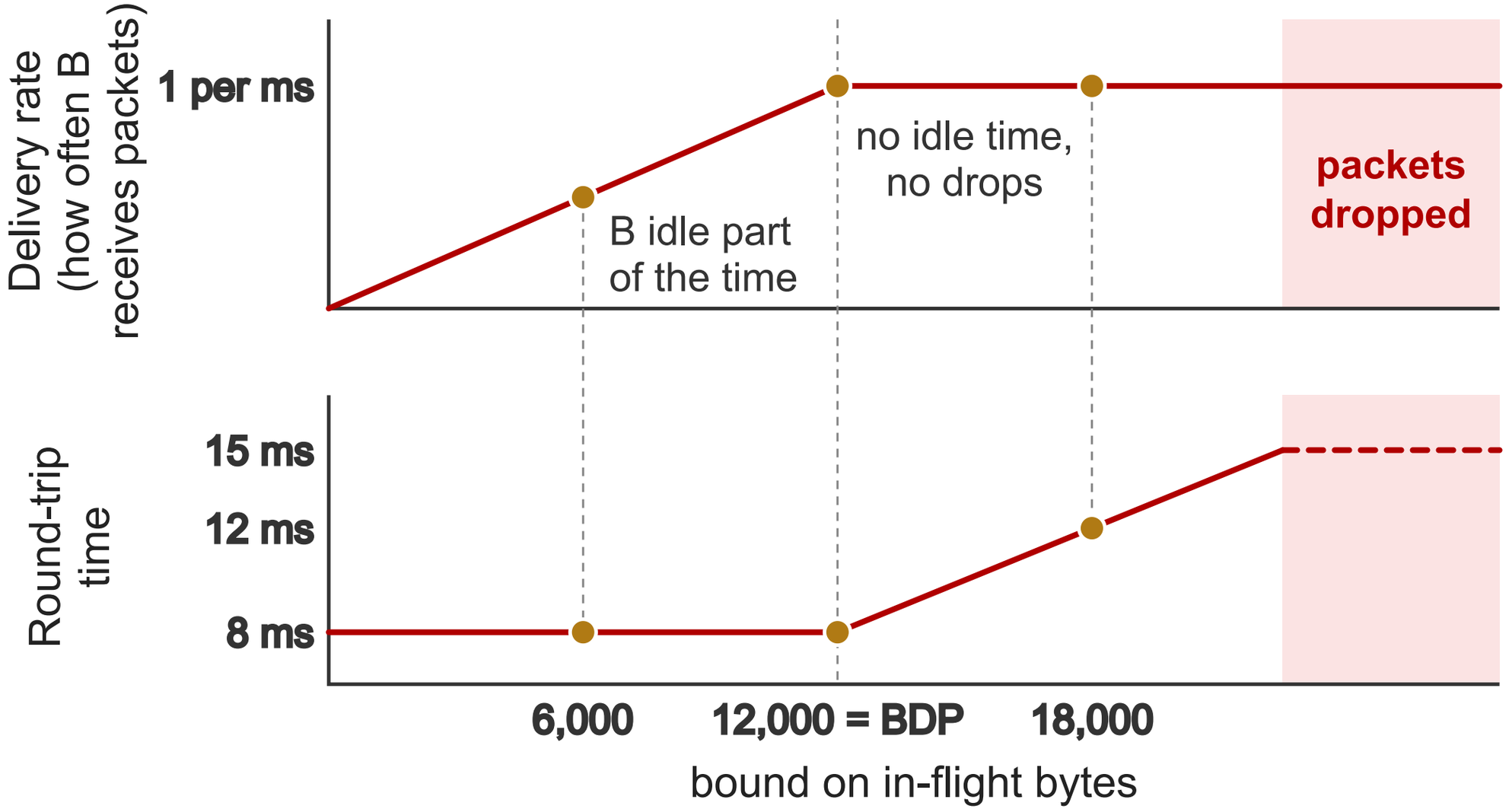
Bounding the number of in-flight bytes



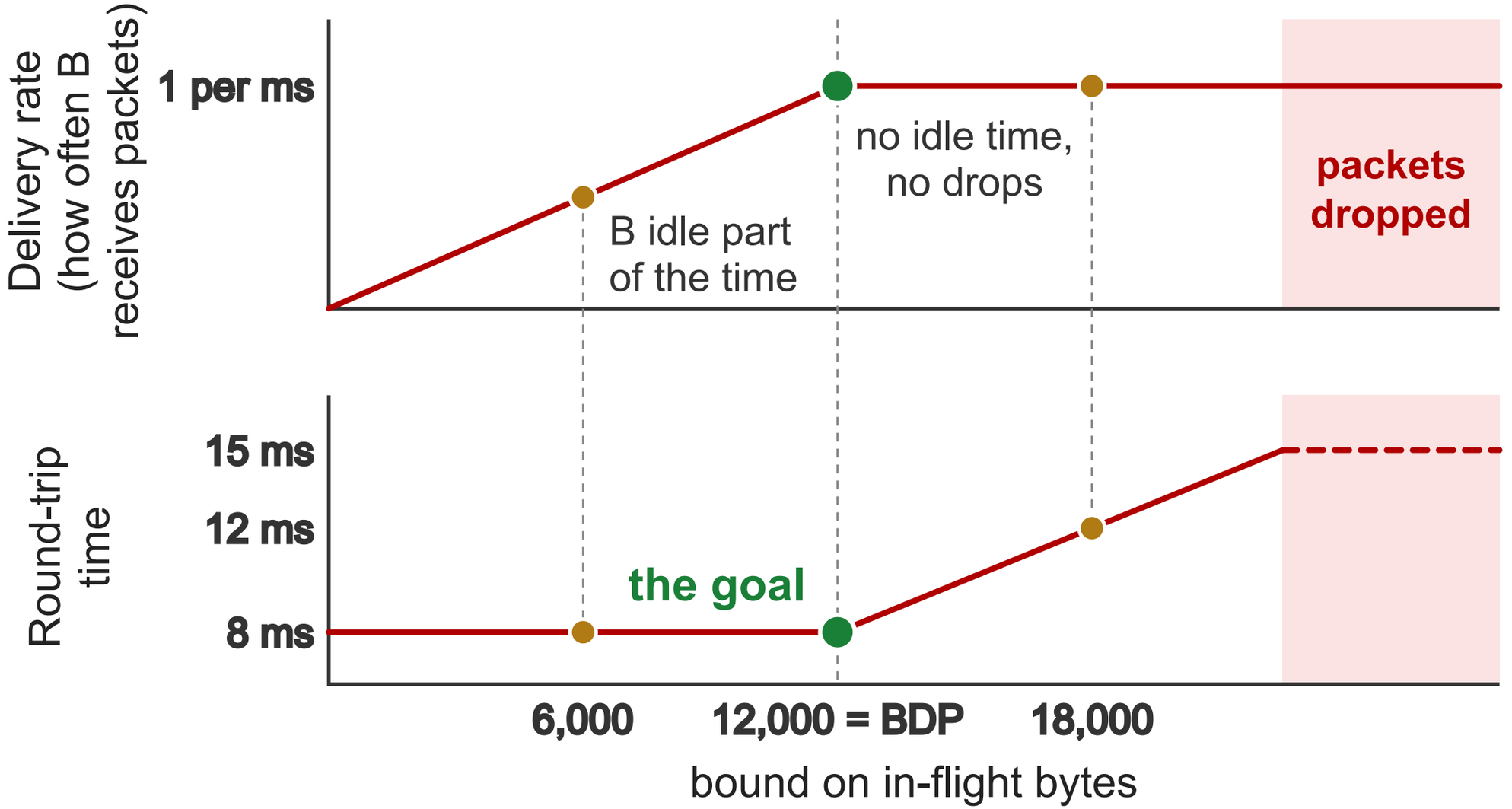
Bounding the number of in-flight bytes



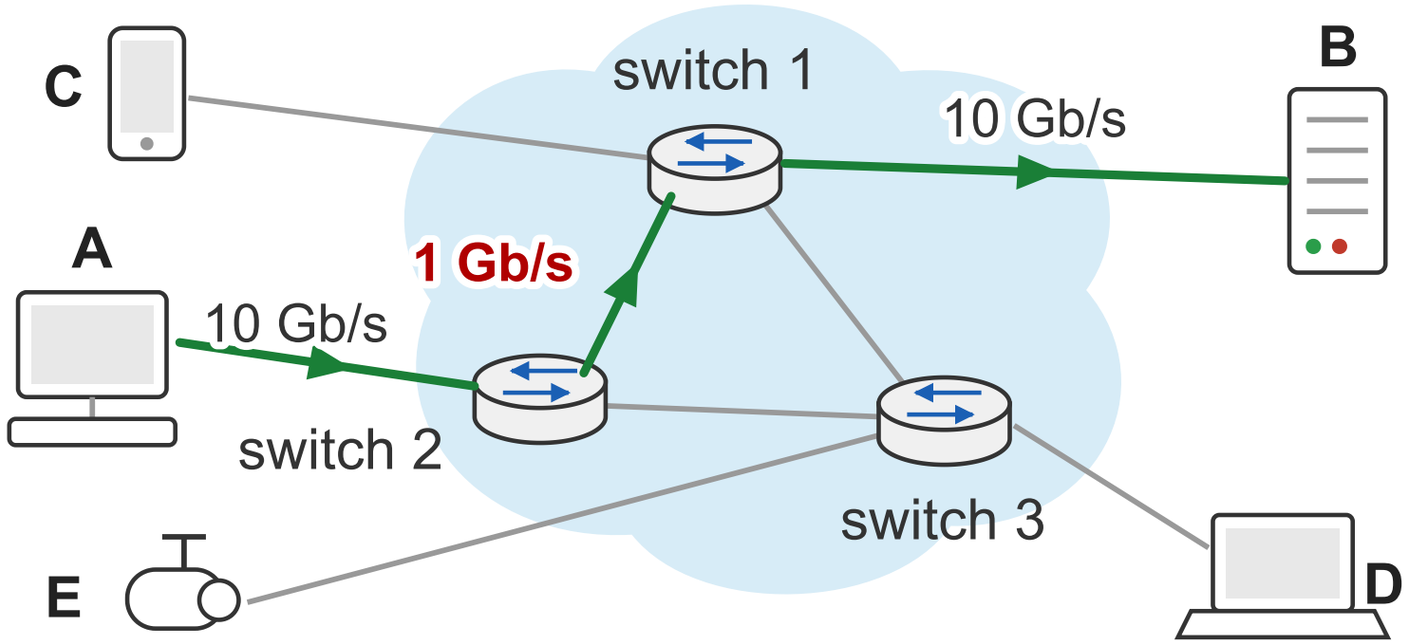
Bounding the number of in-flight bytes



Bounding the number of in-flight bytes



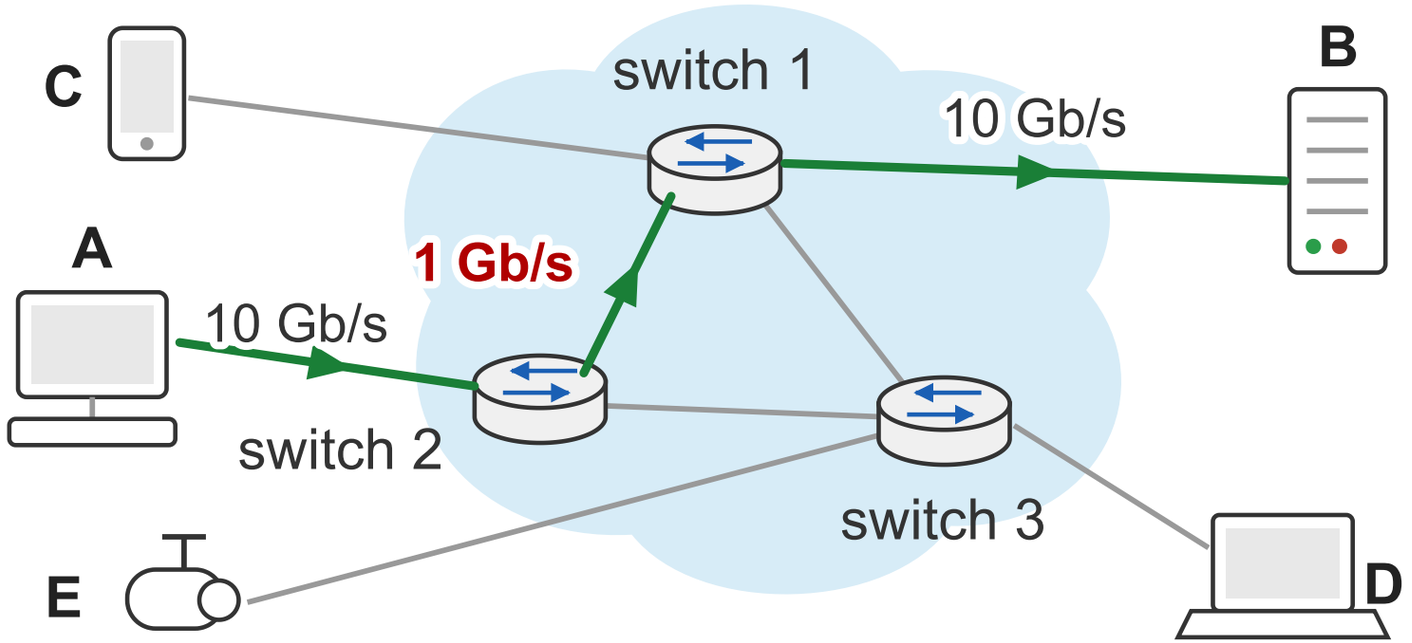
How fast should the sender send?



Consider a sender and a receiver:

- **The sender knows exactly what path its traffic will take to the destination**, and the properties (rate, propagation delay, etc.) of each link
- The sender always has more data to send

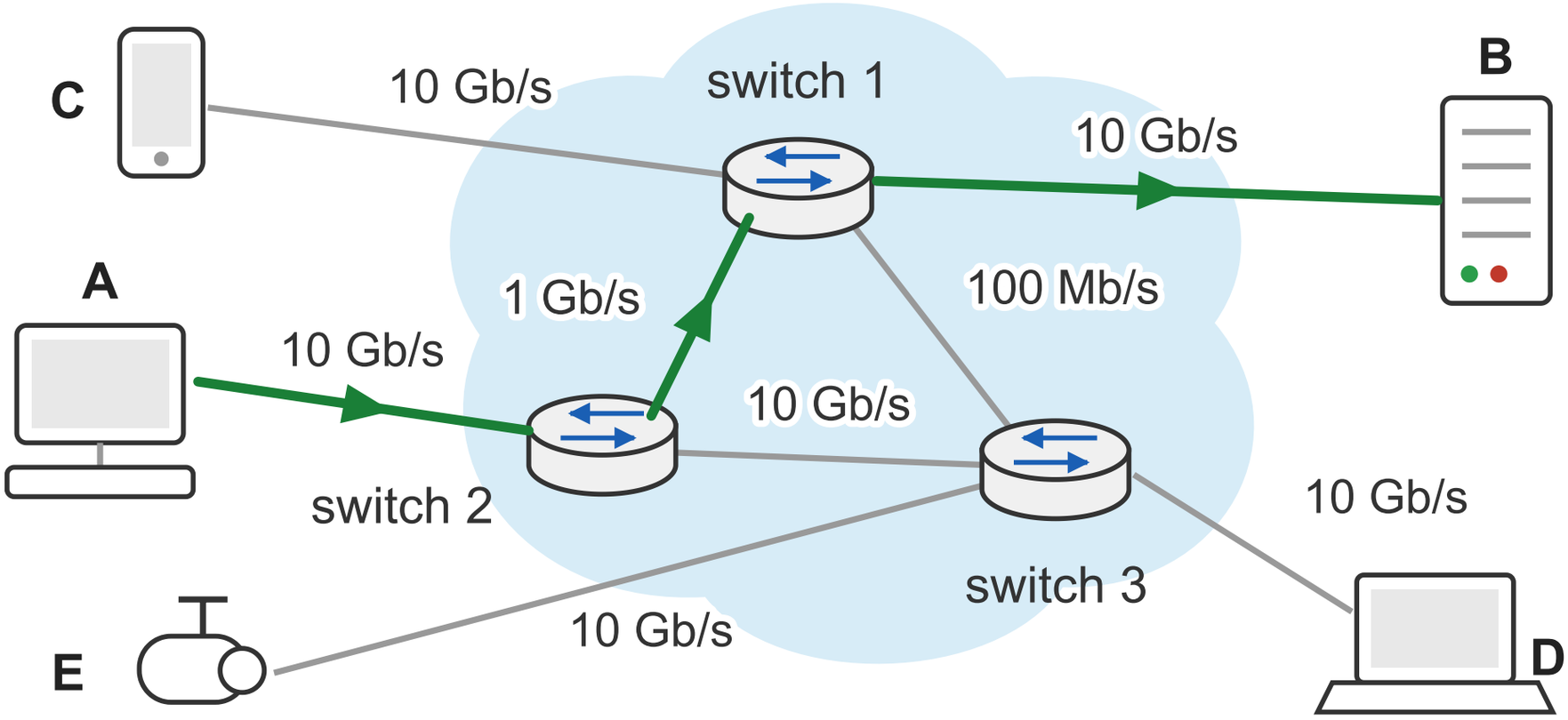
How fast should the sender send?



Consider a sender and a receiver:

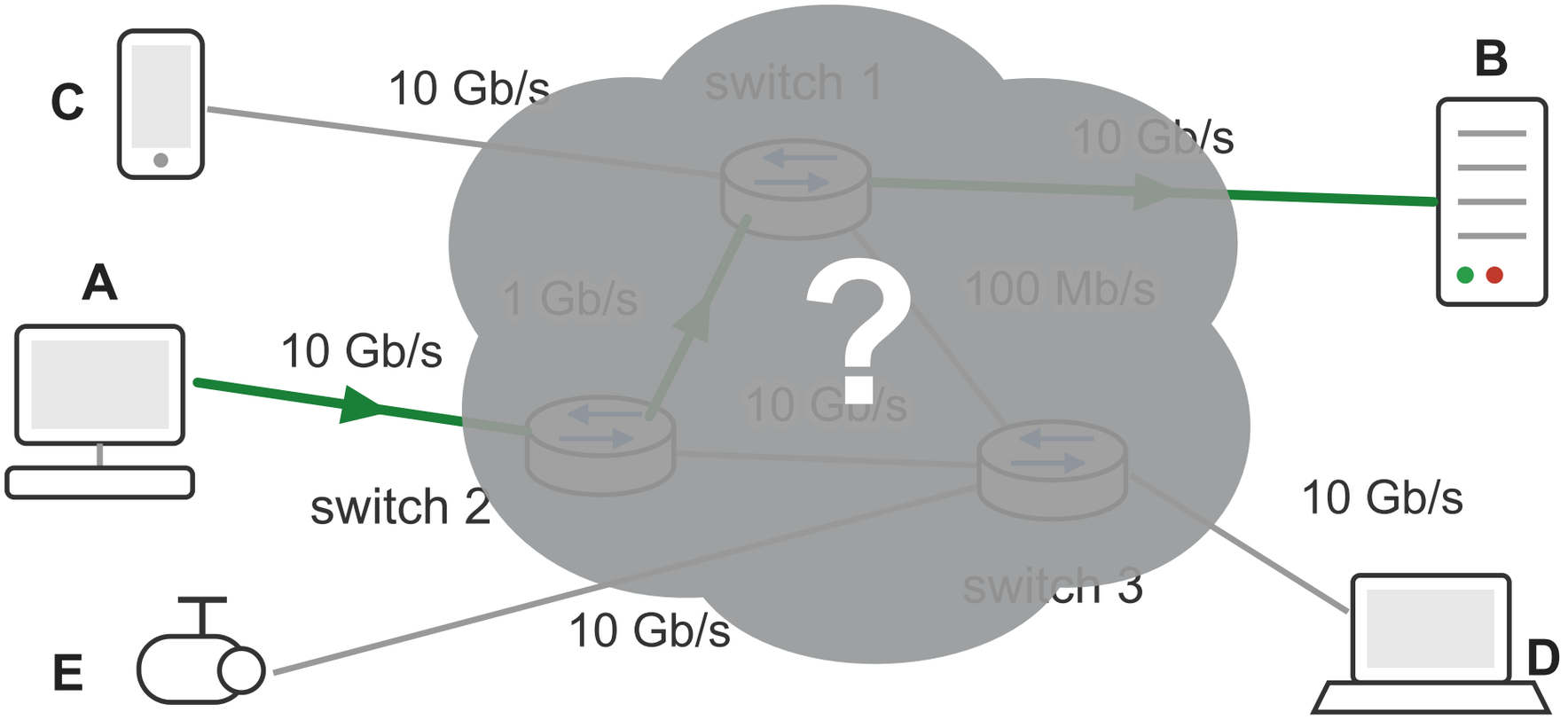
- **The sender knows exactly what path its traffic will take to the destination**, and the properties (rate, propagation delay, etc.) of each link
- The sender always has more data to send

In reality



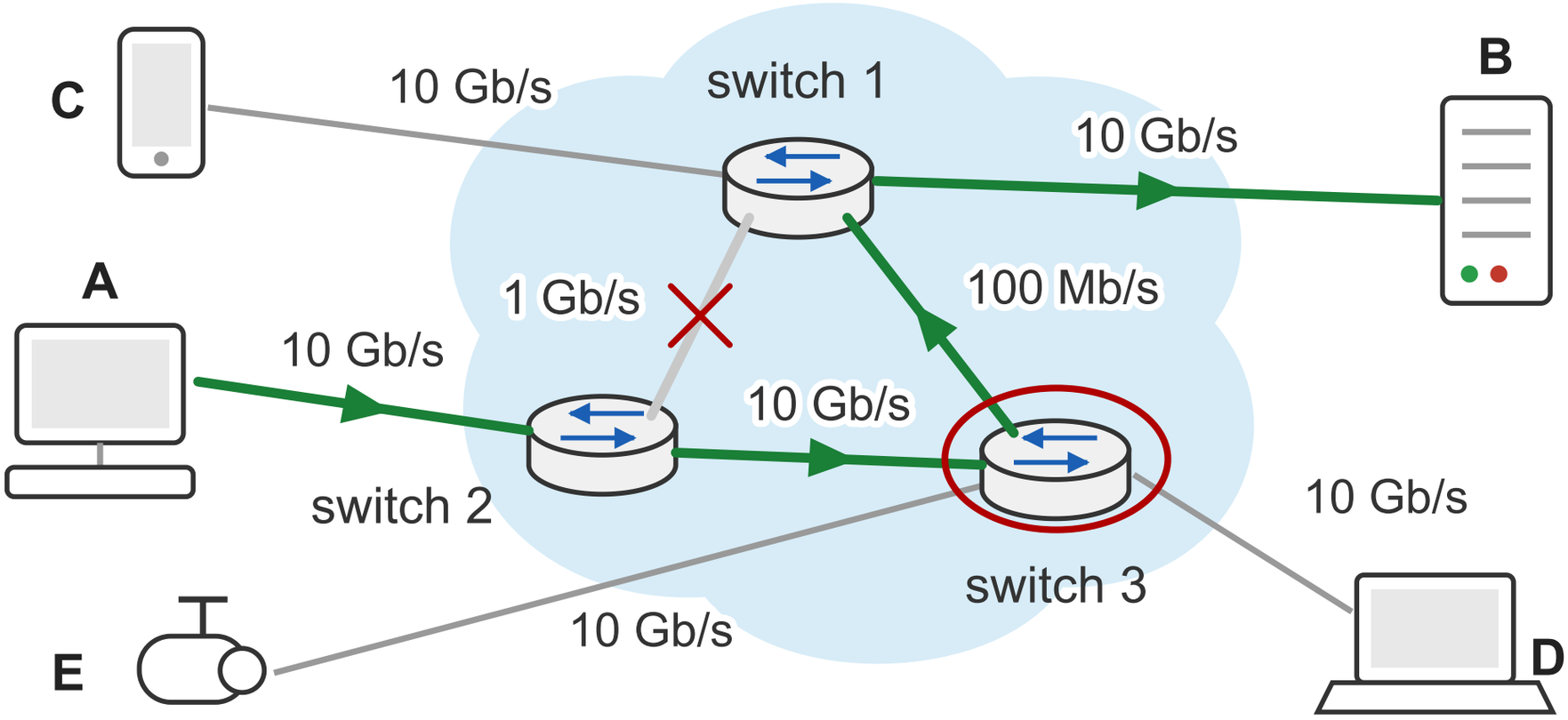
So far: A knew its path (green) and every link's rate

In reality



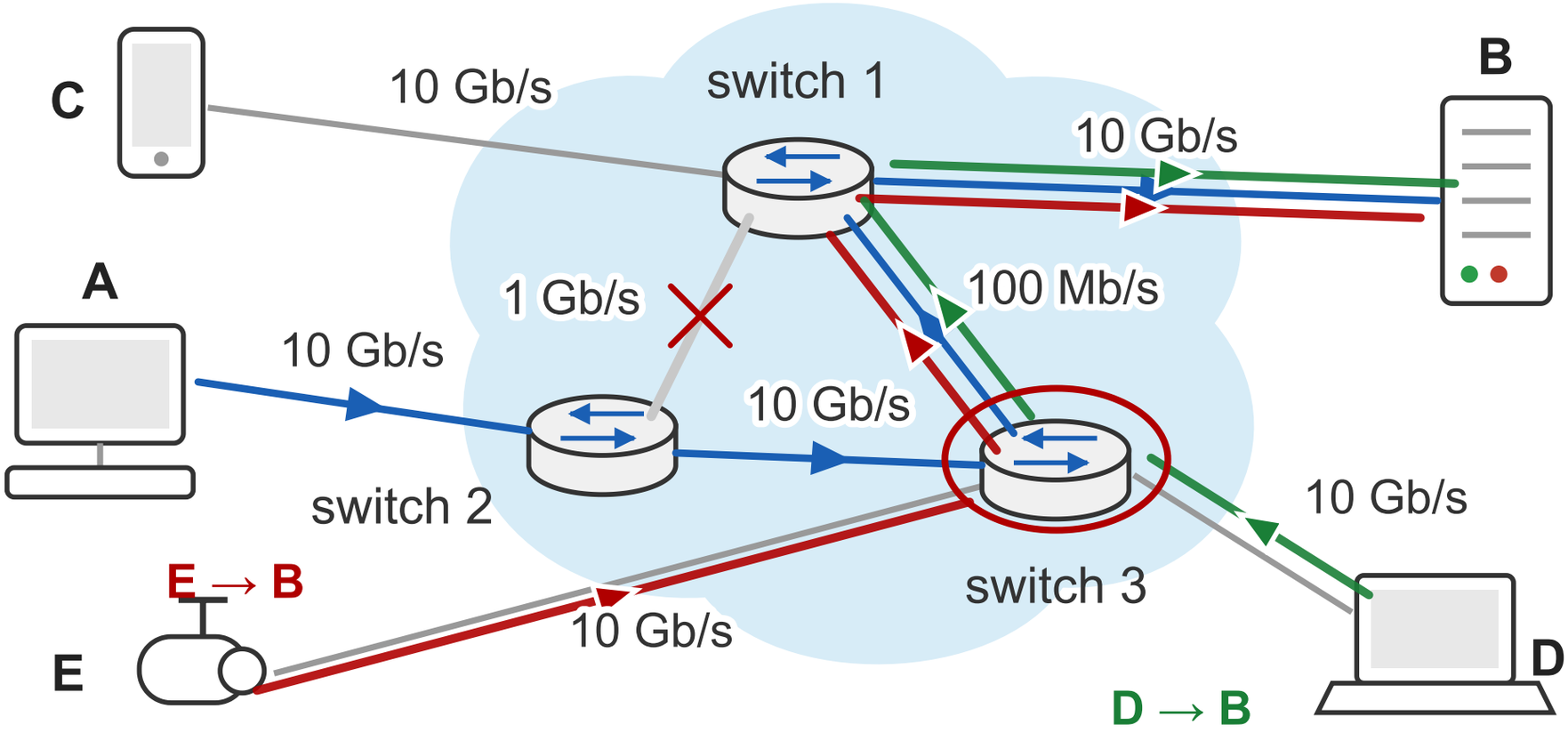
1. A does not know its path, or the rates on it

In reality



2. Even if A somehow found out once, that path might change (link failures, traffic changes, …)

In reality



3. Other flows may start sharing the bottleneck, and reduce A's share

How would you find the right share? (in-class)

(in-class discussion)

- You are a sender communicating with a receiver. You don’t get any extra information from the network or other senders and receivers that share the network with you.
- How can you figure out your “right” share, and notice when it changes?

Reset

Congestion control

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows
  - React quickly when things change

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows
  - React quickly when things change
  - Without much direct input from the network

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows
  - React quickly when things change
  - Without much direct input from the network
  - Without coordination with other flows

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows
  - React quickly when things change
  - Without much direct input from the network
  - Without coordination with other flows
  - …

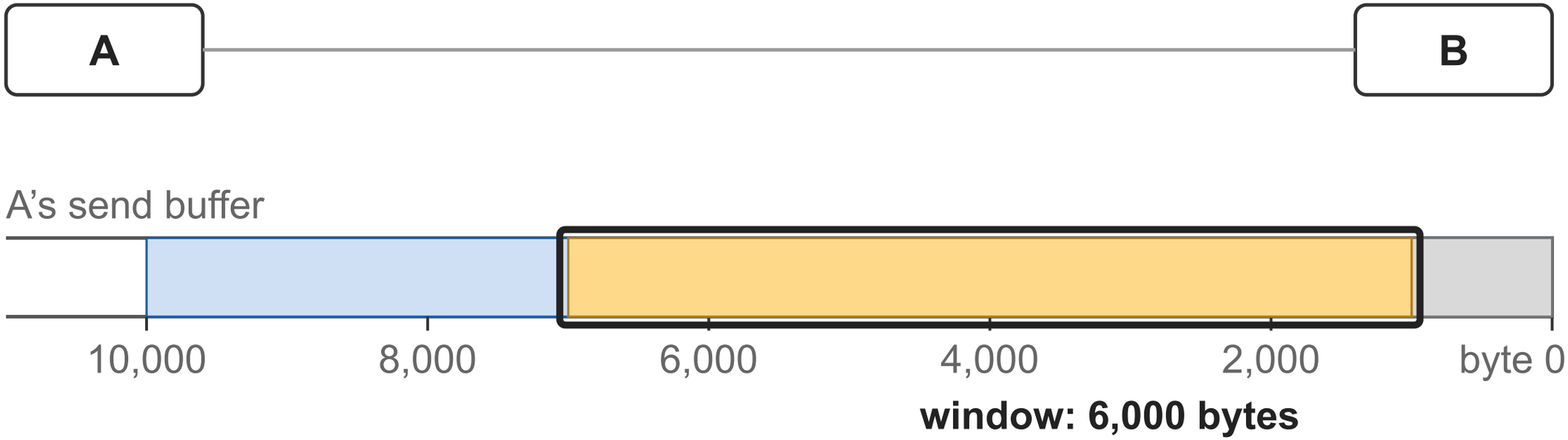
Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows
  - React quickly when things change
  - Without much direct input from the network
  - Without coordination with other flows
  - …
- Four decades of research

Congestion control

- Congestion control algorithms use control packets and measurements to decide when to send more data into the network:
  - Typically part of the transport layer (Why?)
- This is a hard problem:
  - Keep the bottleneck busy
  - Keep the queues short
  - Share fairly with other flows
  - React quickly when things change
  - Without much direct input from the network
  - Without coordination with other flows
  - …
- Four decades of research
- → Let’s see how TCP classically does congestion control

Recap: the sliding window

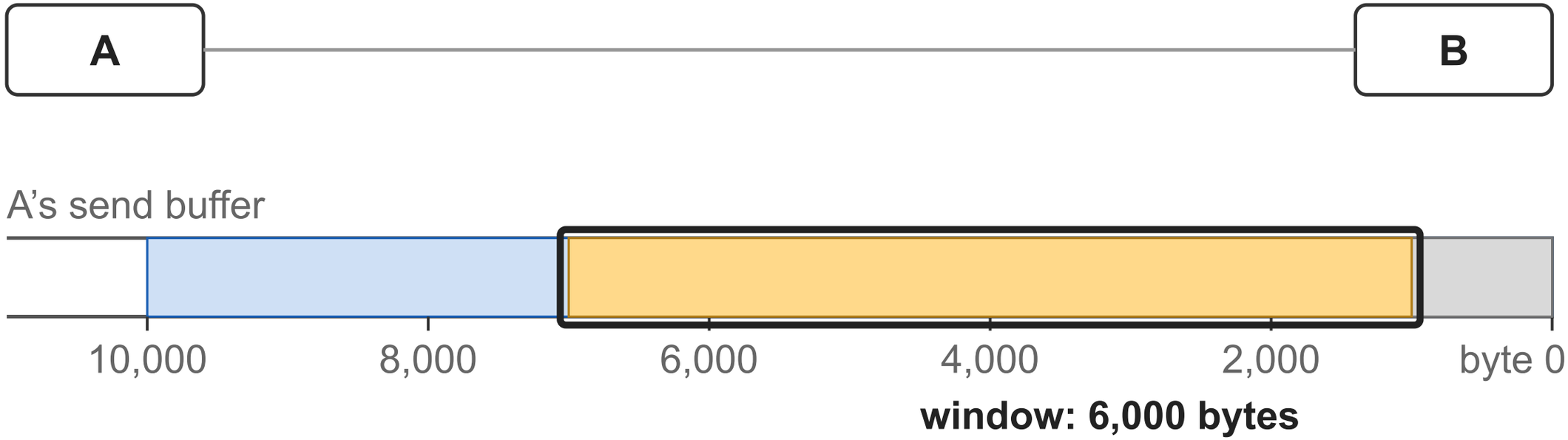


A only sends bytes inside the window

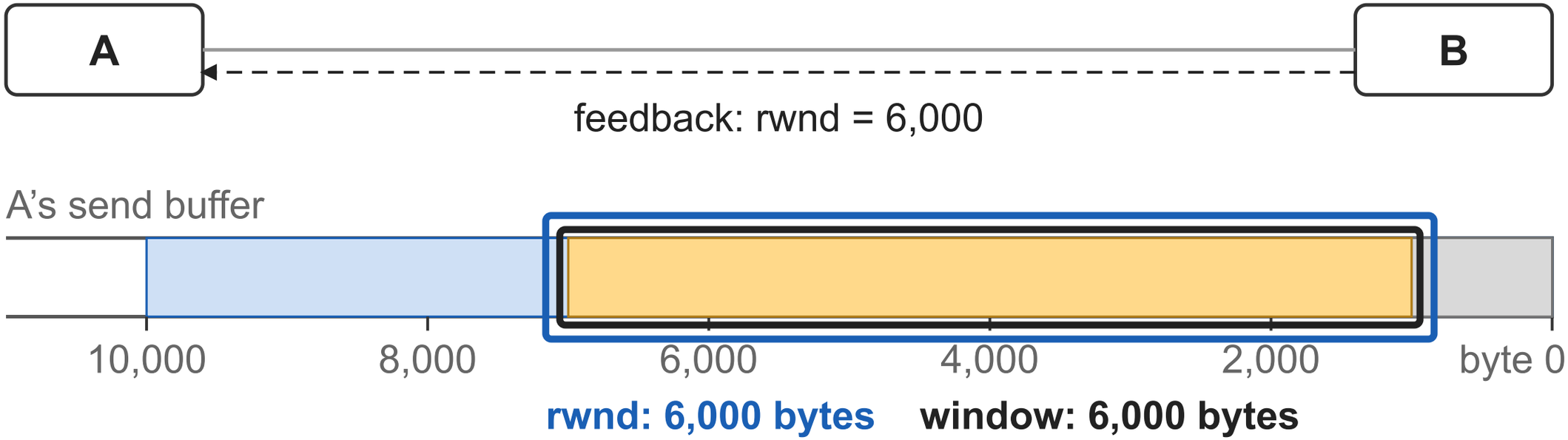
The window starts at the oldest byte without feedback

feedback received sent, waiting for feedback not sent yet

So far, sliding window = receive window



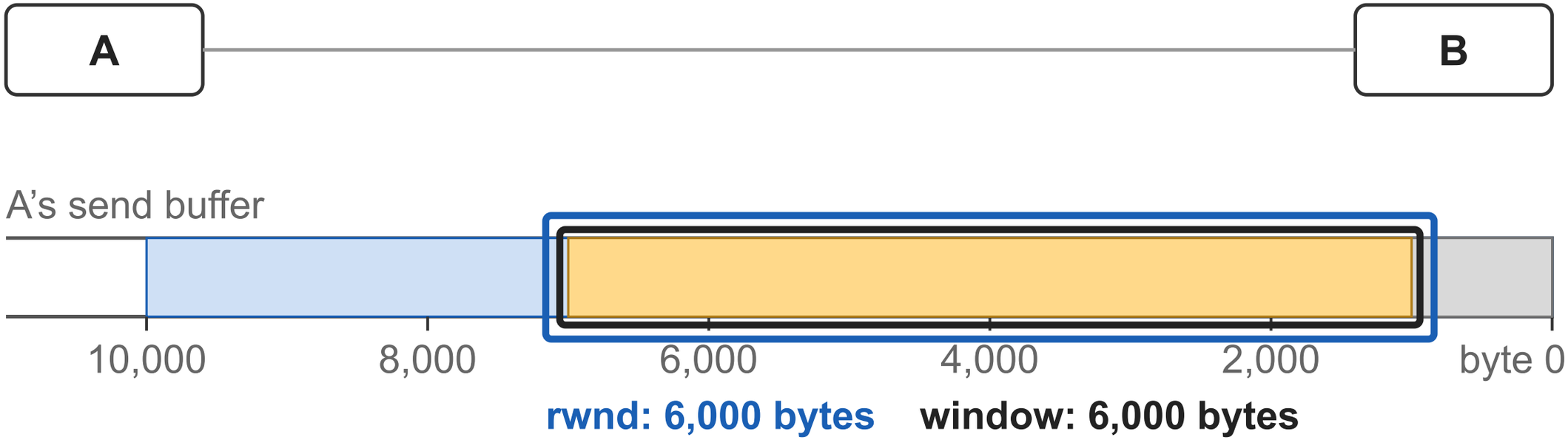
So far, sliding window = receive window



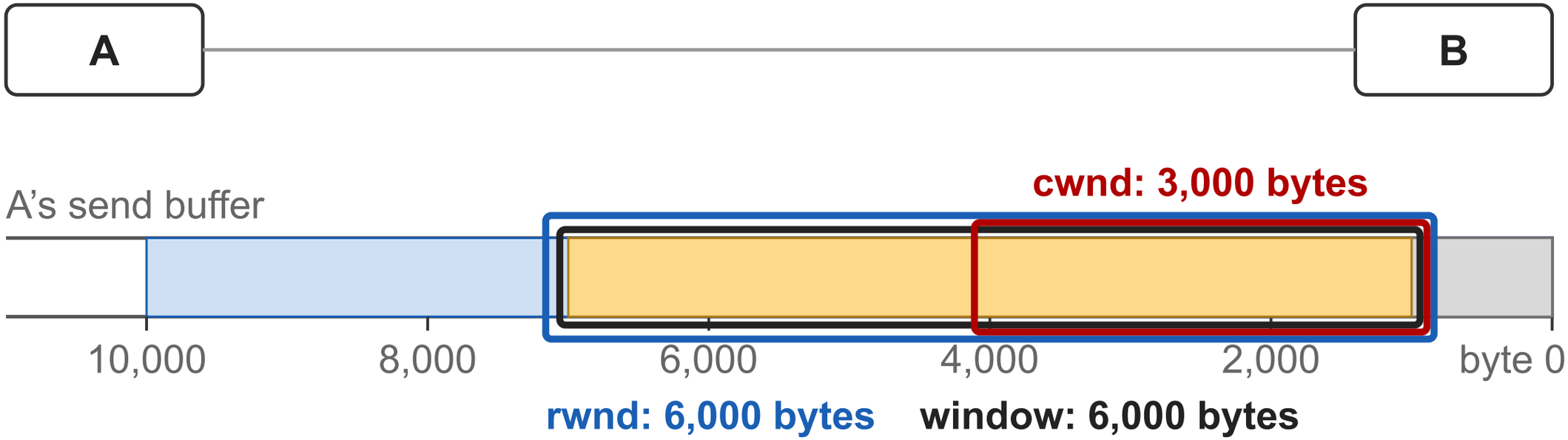
Receive window (rwnd): how far A can go beyond the oldest unacked byte, so B's receive buffer doesn't overflow

Sliding window size = rwnd

Congestion window

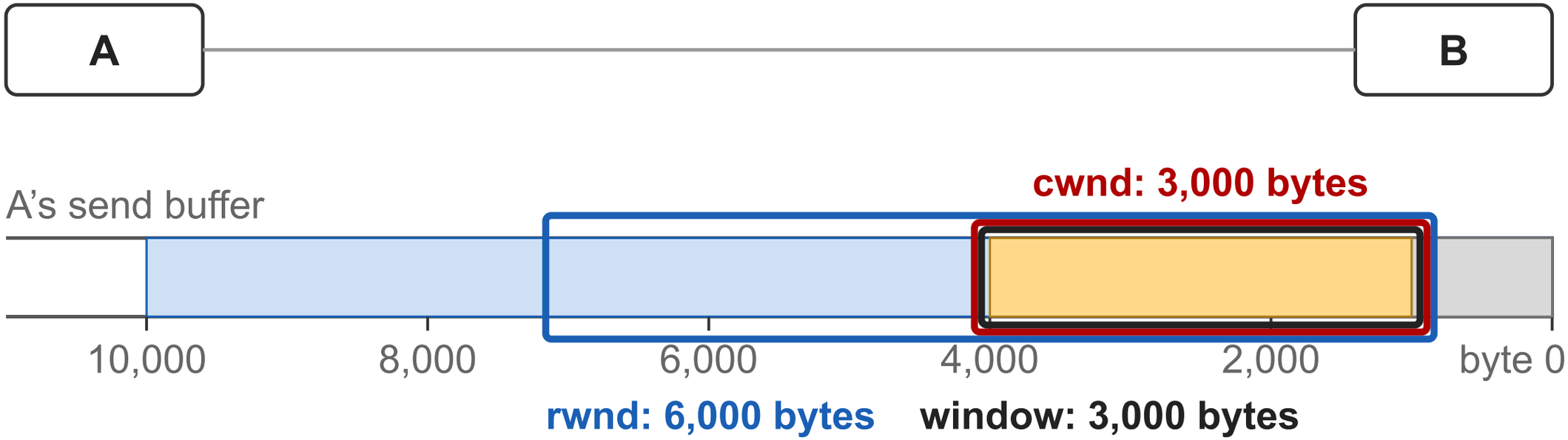


Congestion window



Congestion window (cwnd): how much capacity the network has for this flow at the moment

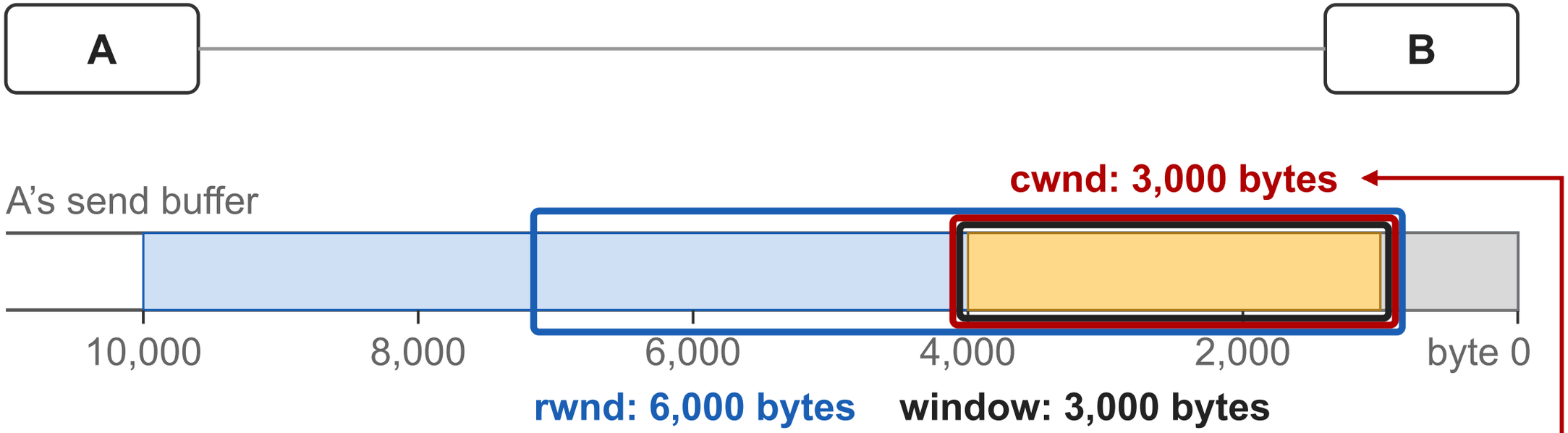
Congestion window



Congestion window (cwnd): how much capacity the network has for this flow at the moment

Sliding window size = the smaller of cwnd and rwnd

Congestion window



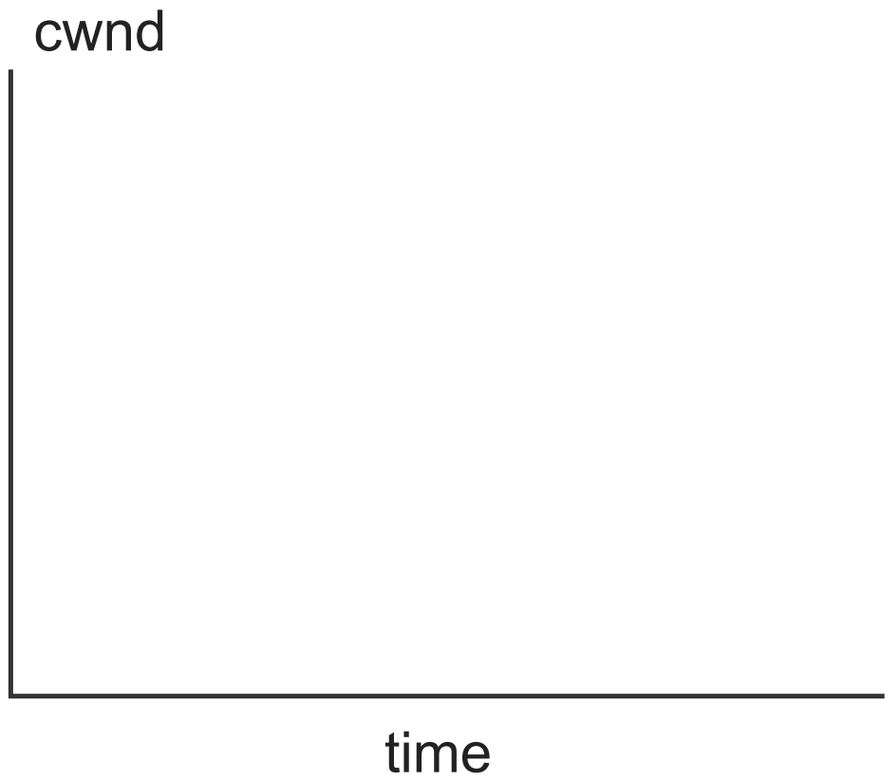
Congestion window (cwnd): how much capacity the network has for this flow at the moment

Sliding window size = the smaller of cwnd and rwnd

A needs to adjust cwnd based on signals from the network

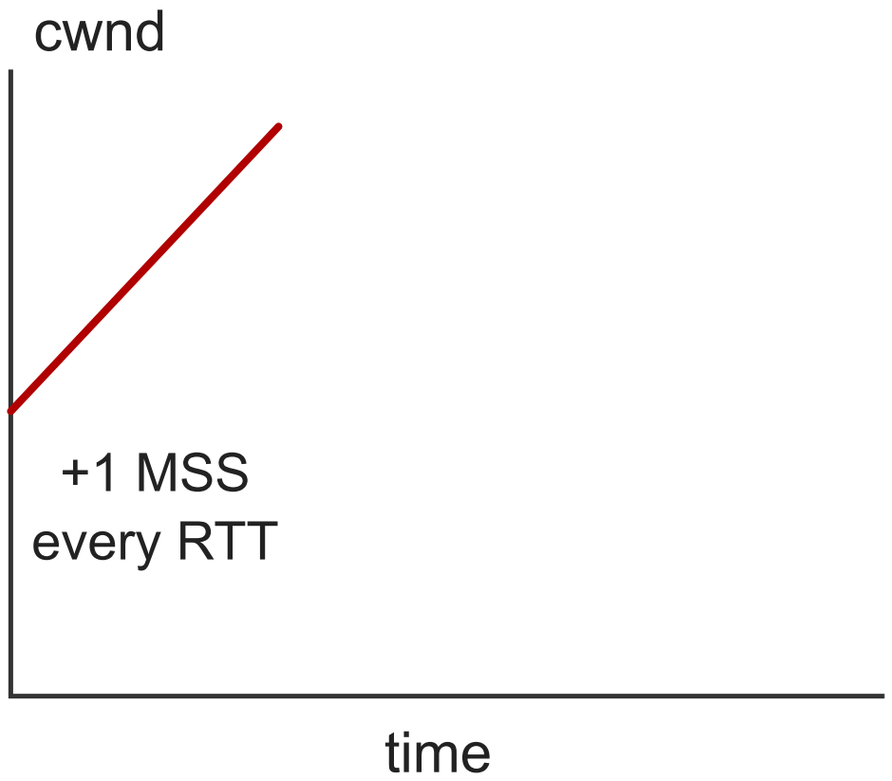
TCP congestion control - Basic AIMD

- Increase cwnd until a loss, then decrease it



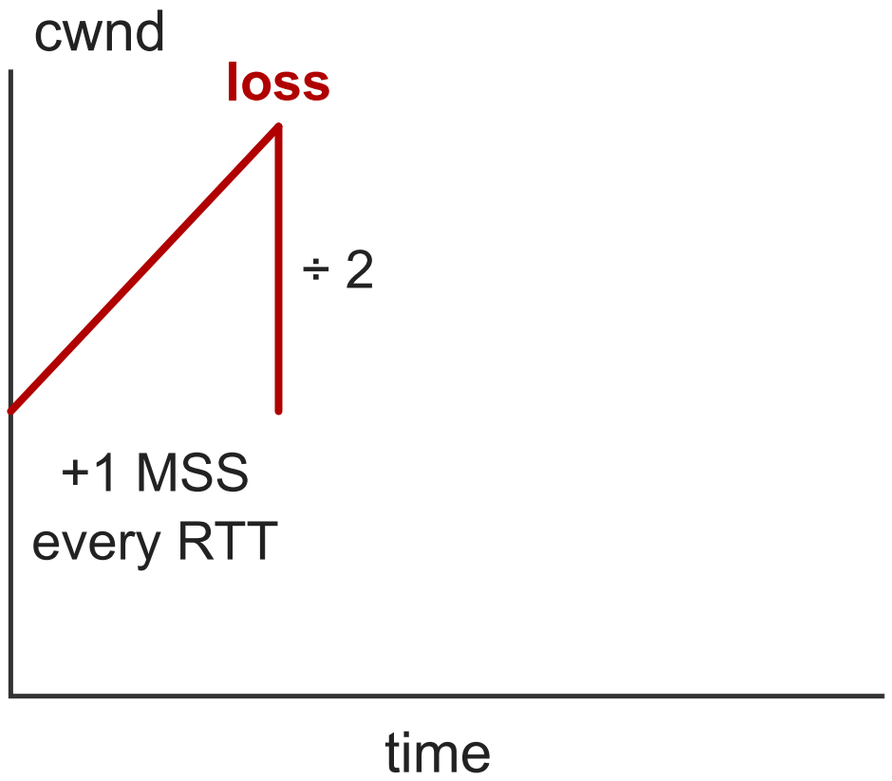
TCP congestion control - Basic AIMD

- Increase cwnd until a loss, then decrease it
- **Additive increase (AI):**
  - Increase cwnd by one MSS every RTT



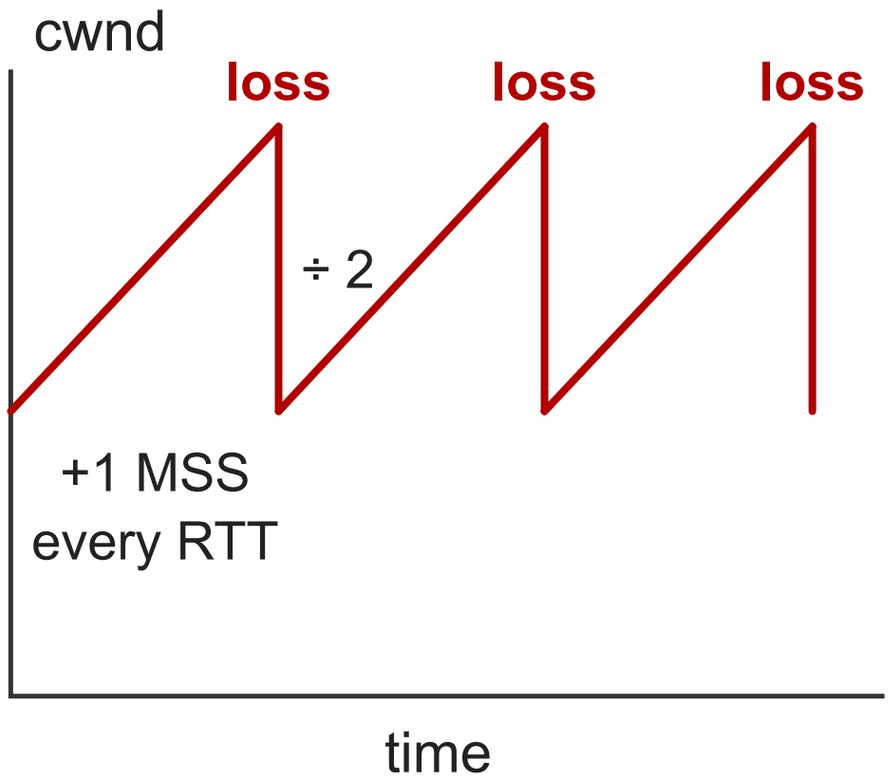
TCP congestion control - Basic AIMD

- Increase cwnd until a loss, then decrease it
- **Additive increase (AI):**
  - Increase cwnd by one MSS every RTT
- **Multiplicative decrease (MD):**
  - On a loss: cut cwnd in half

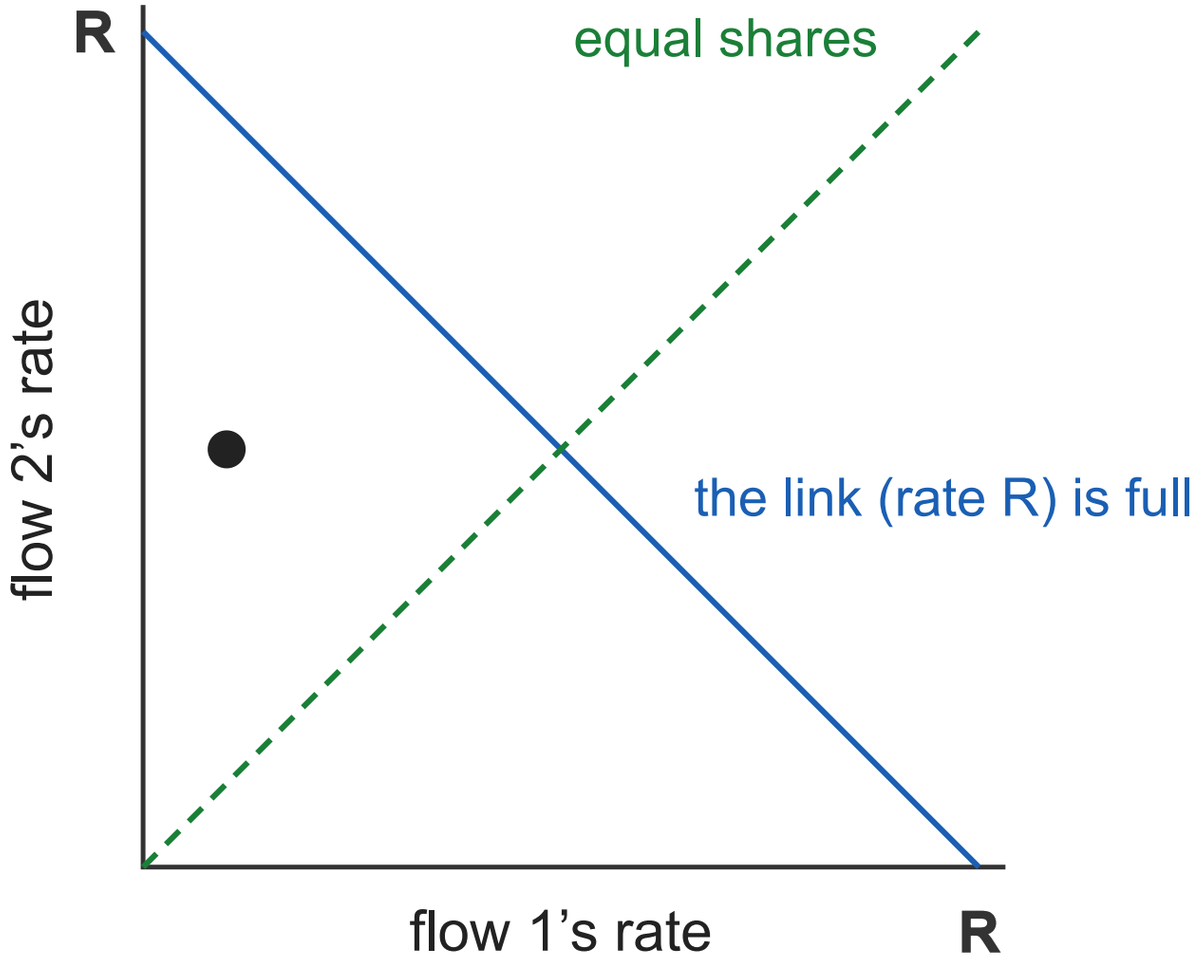


TCP congestion control - Basic AIMD

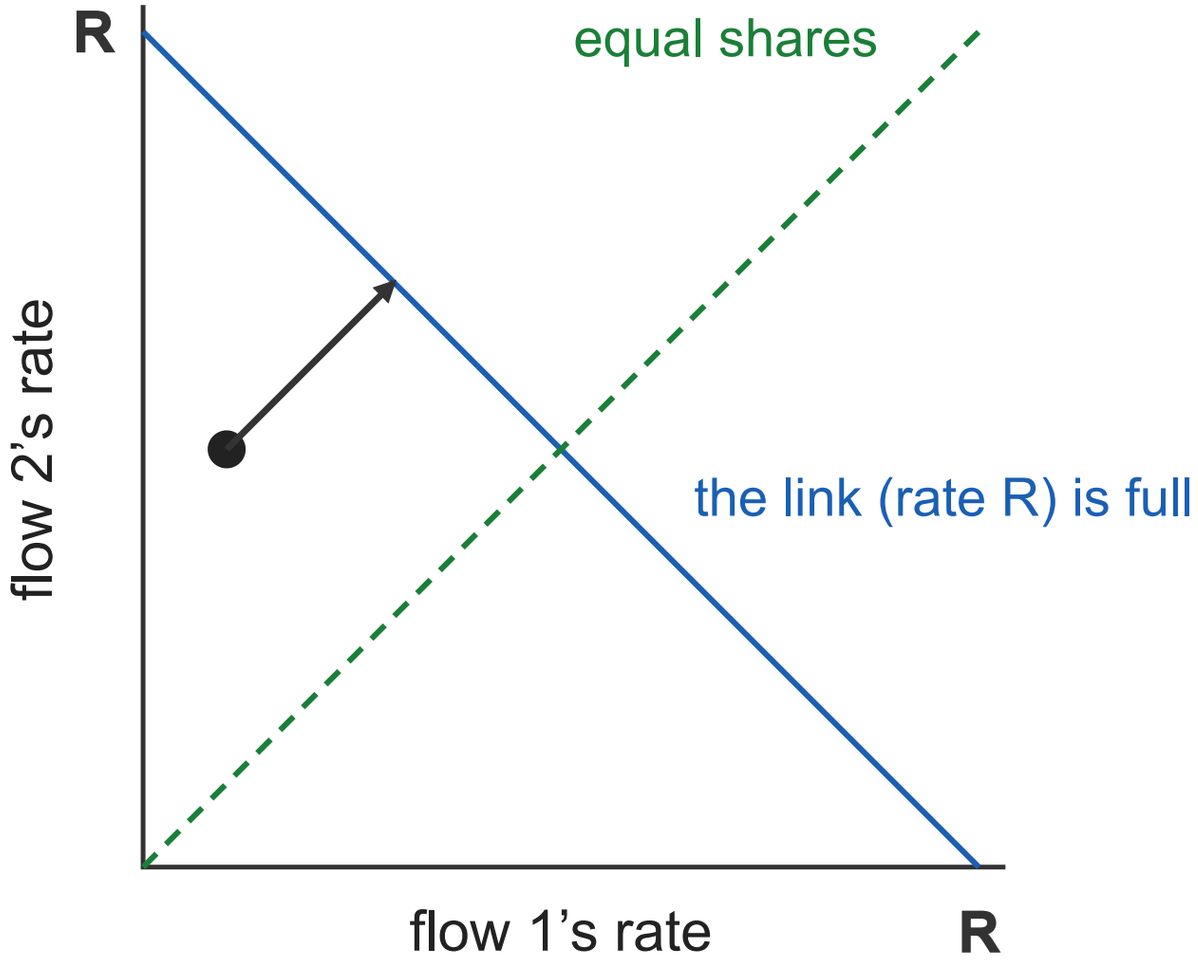
- Increase cwnd until a loss, then decrease it
- **Additive increase (AI):**
  - Increase cwnd by one MSS every RTT
- **Multiplicative decrease (MD):**
  - On a loss: cut cwnd in half
- Cautious probing, aggressive backing off on loss:
  - “the TCP sawtooth”



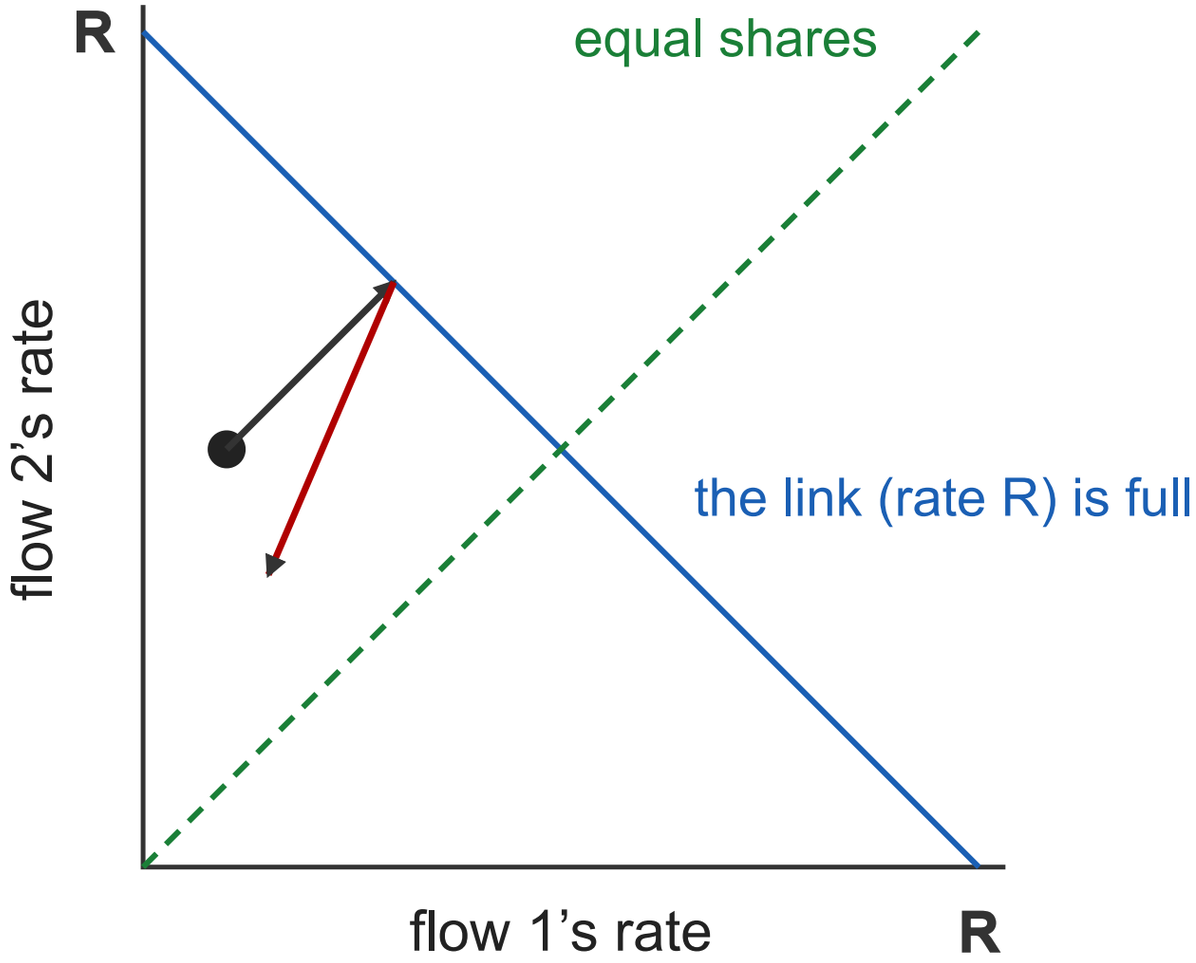
Two flows sharing a link under Basic AIMD



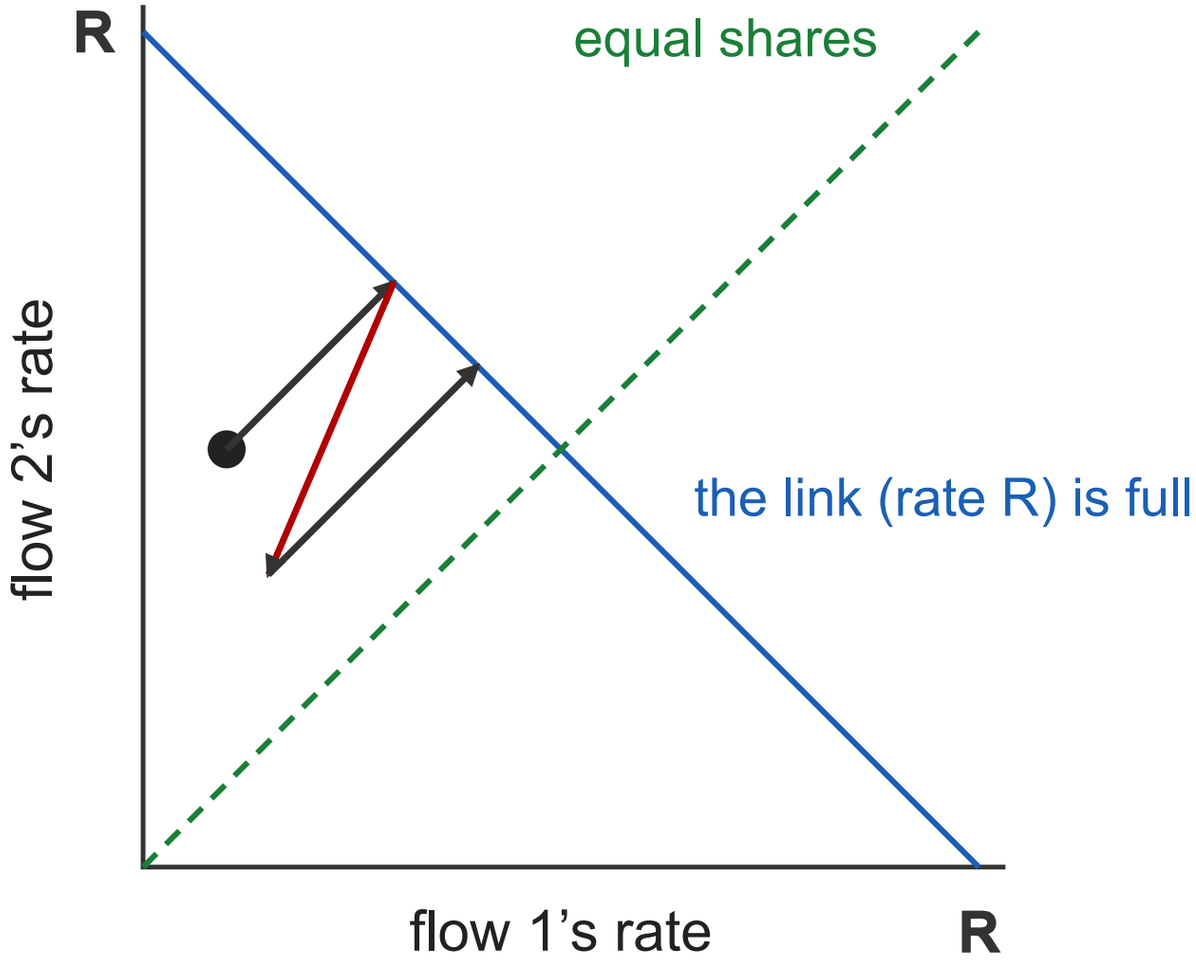
Two flows sharing a link under Basic AIMD



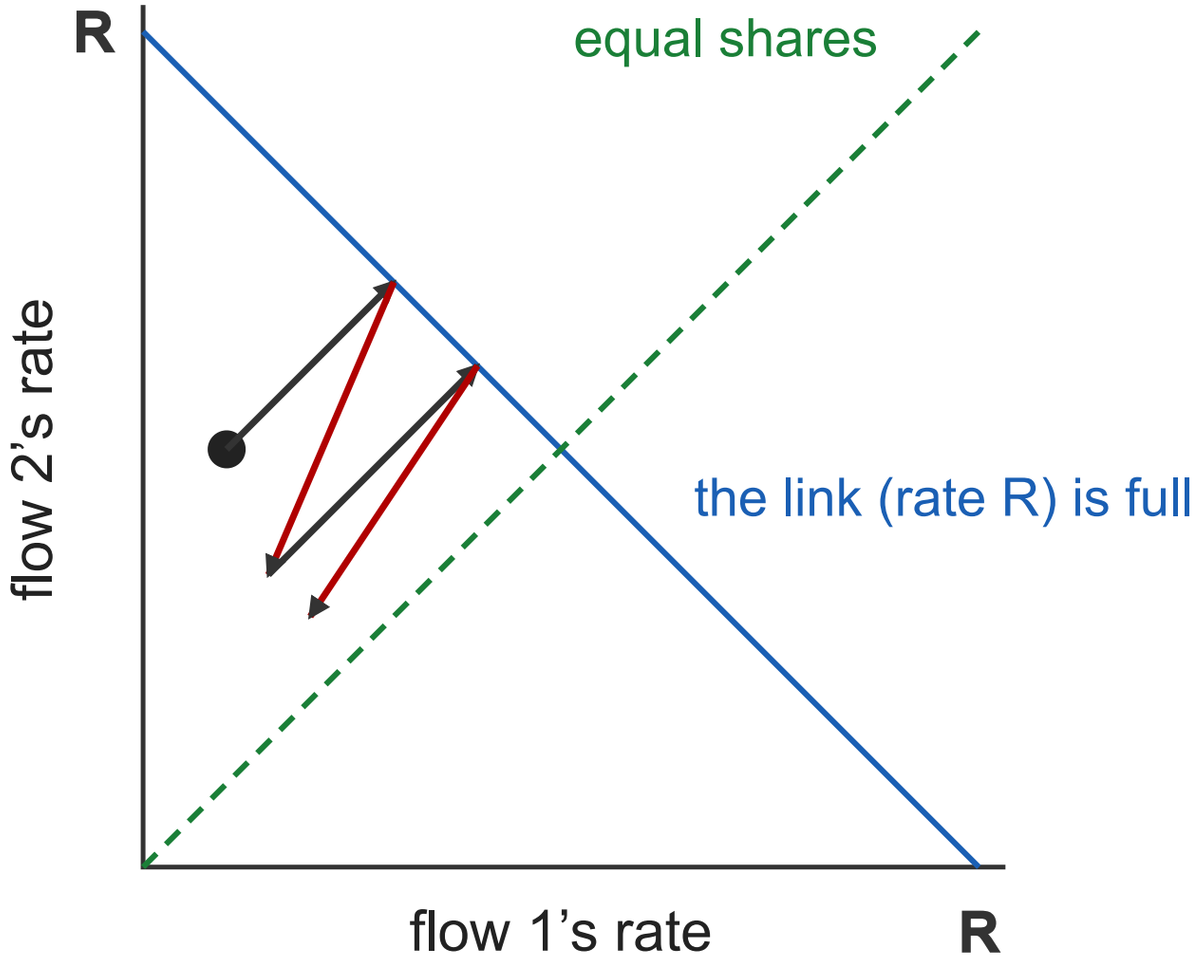
Two flows sharing a link under Basic AIMD



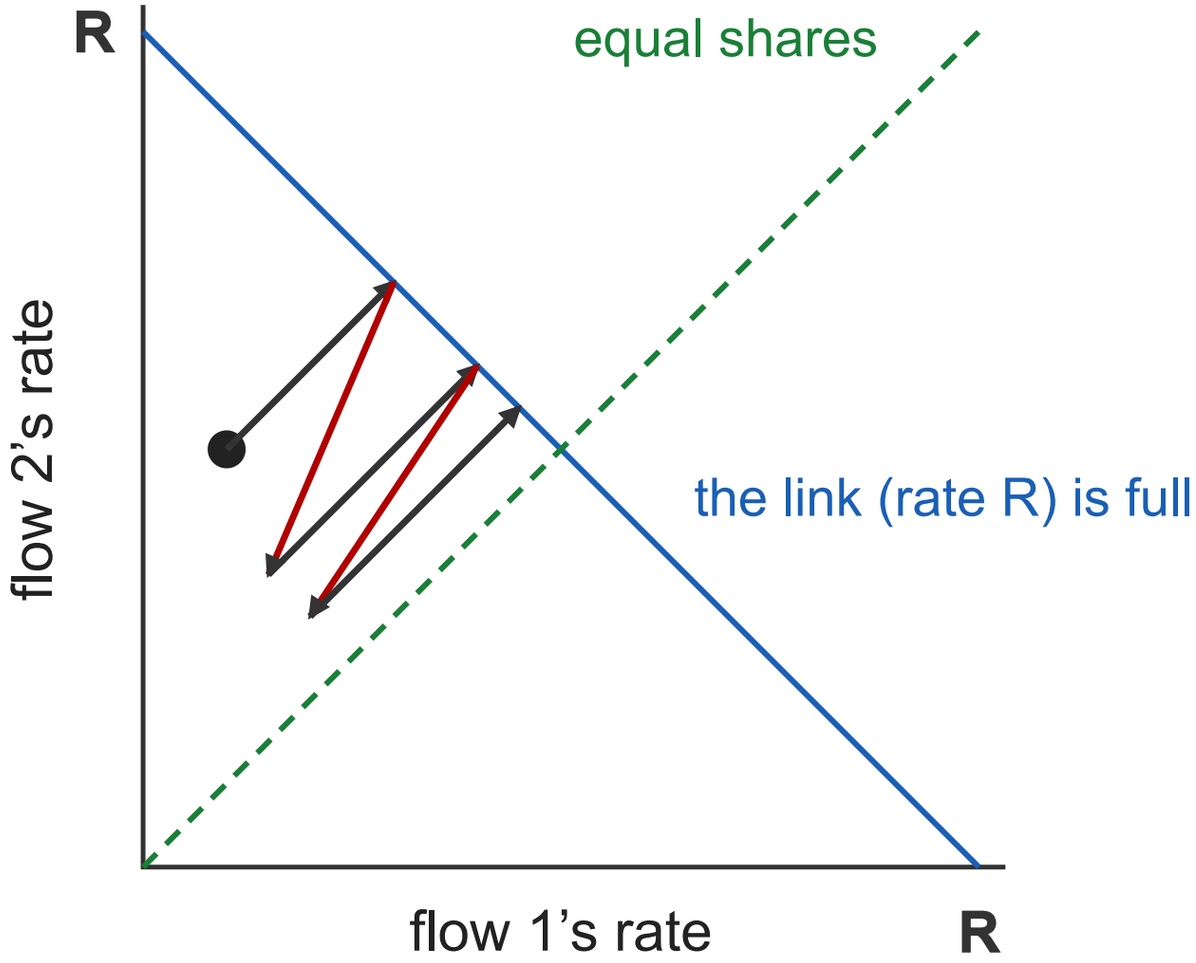
Two flows sharing a link under Basic AIMD



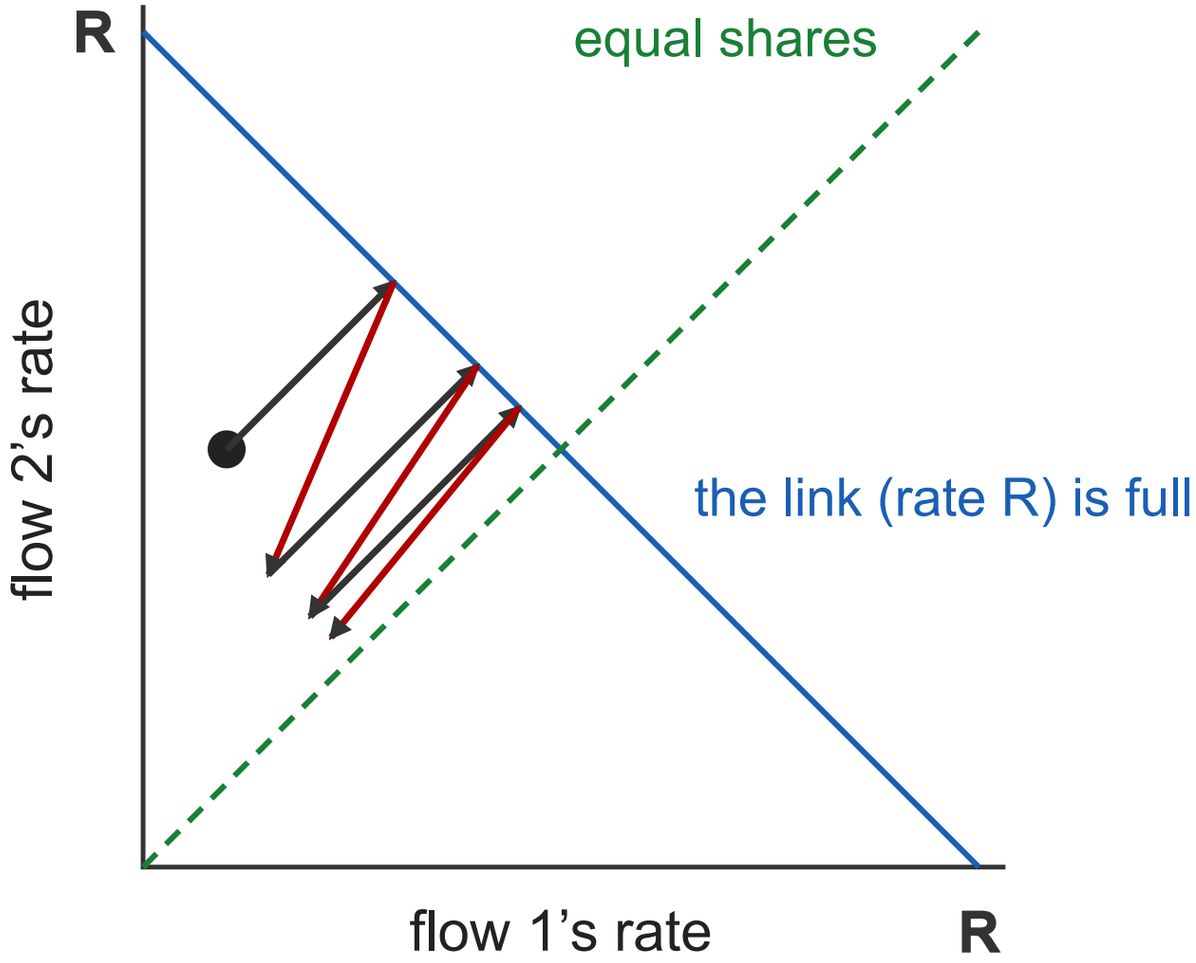
Two flows sharing a link under Basic AIMD



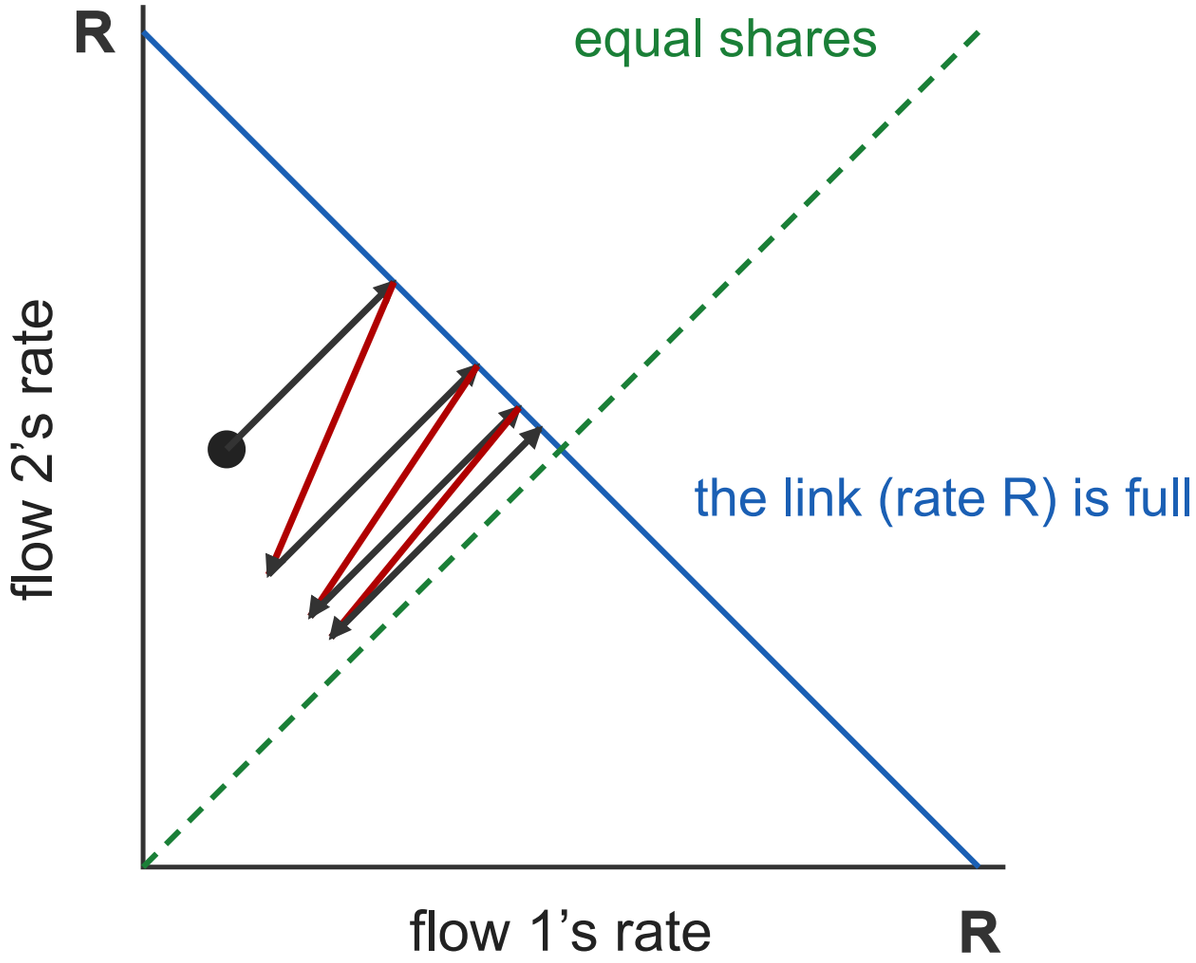
Two flows sharing a link under Basic AIMD



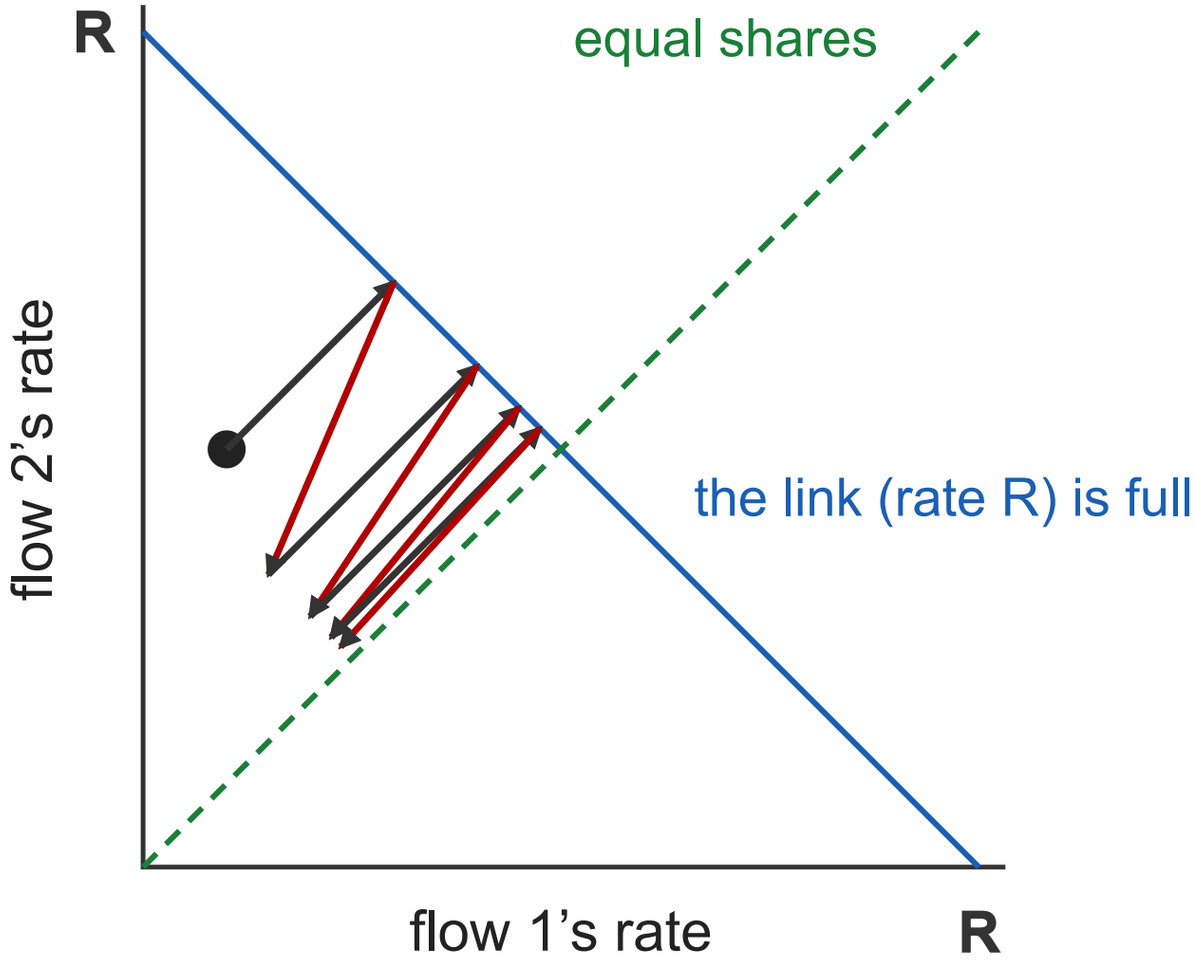
Two flows sharing a link under Basic AIMD



Two flows sharing a link under Basic AIMD



Two flows sharing a link under Basic AIMD



Under idealized assumptions, they share the link fairly: each gets R/2

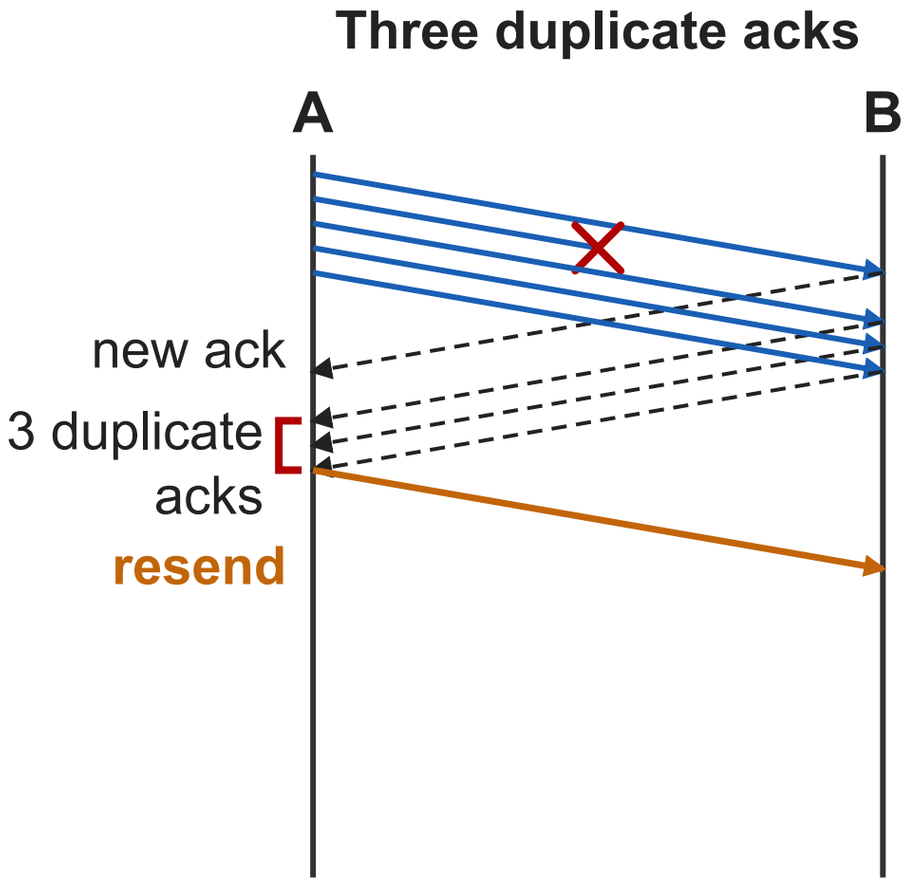
– Same RTT

– Fixed number of flows

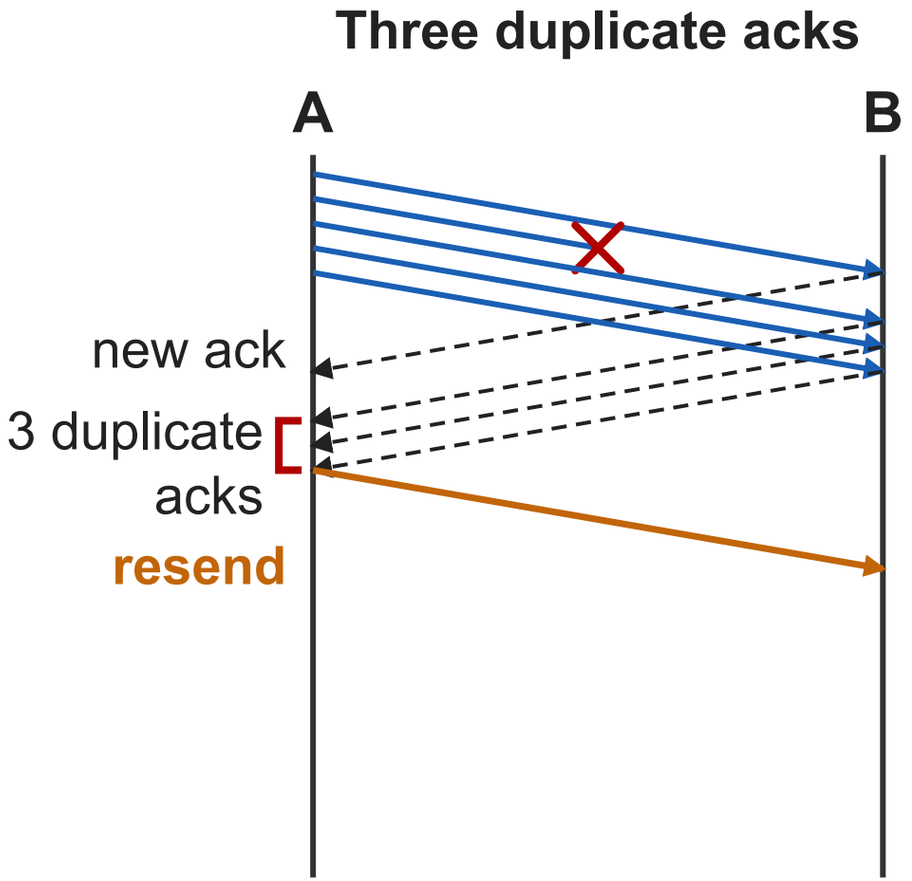
– Both only doing AIMD

– …

Classic TCP: two kinds of loss signal

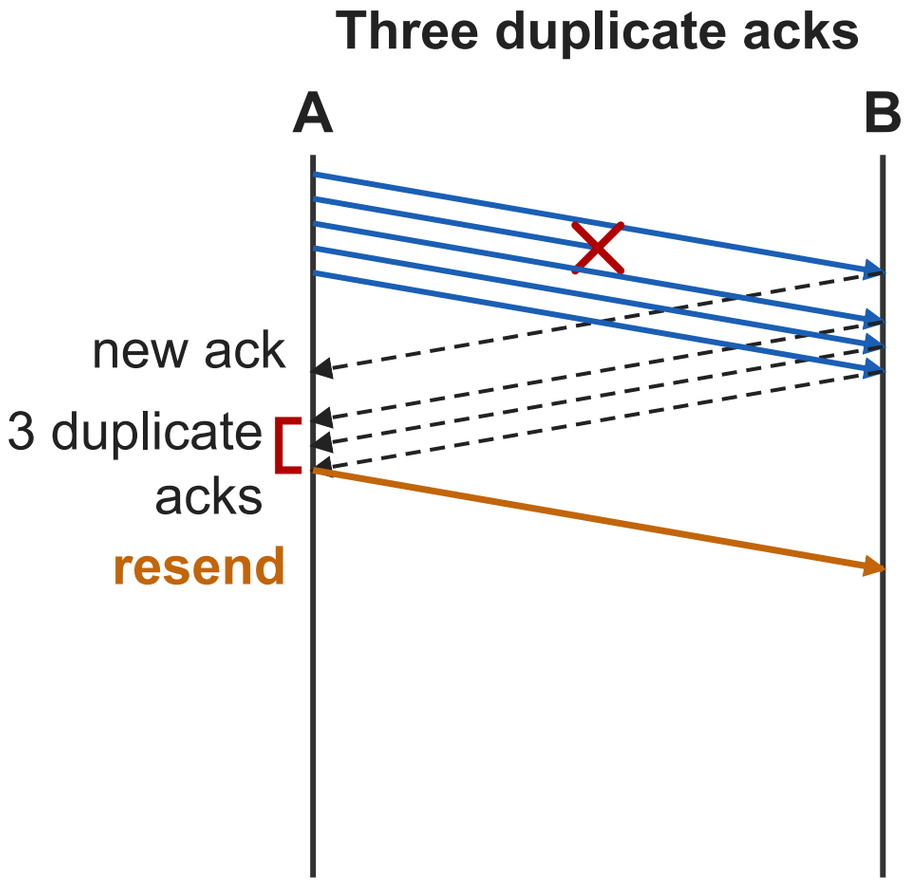


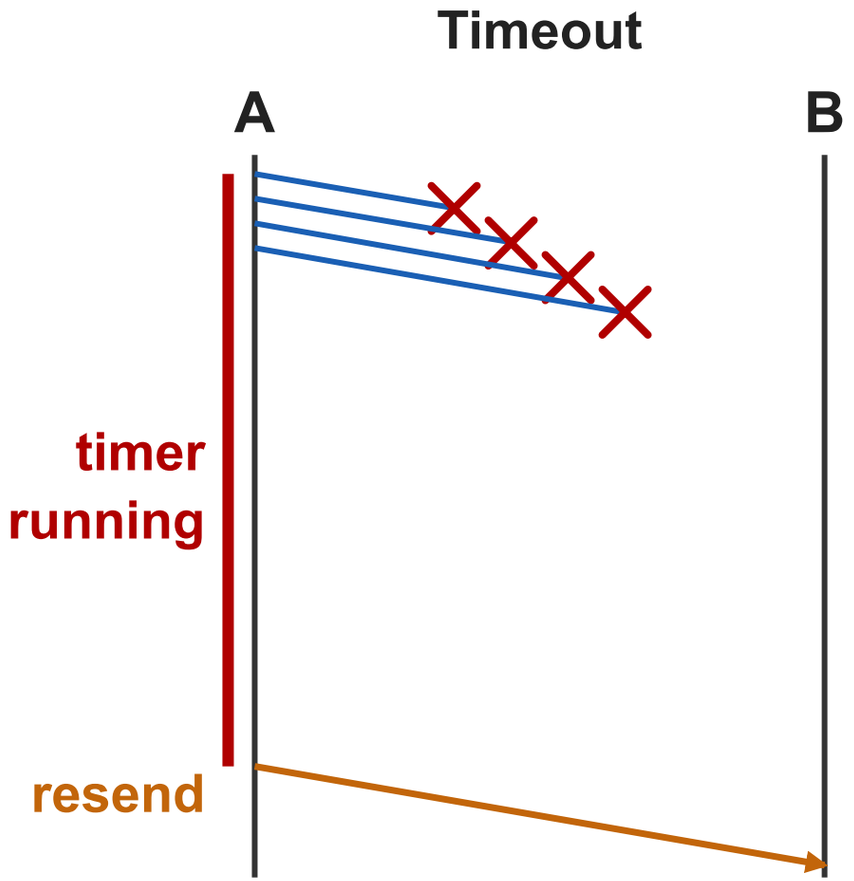
Classic TCP: two kinds of loss signal



Later packets still get through: **mild congestion**. Resend at once (fast retransmit) and cut cwnd in half

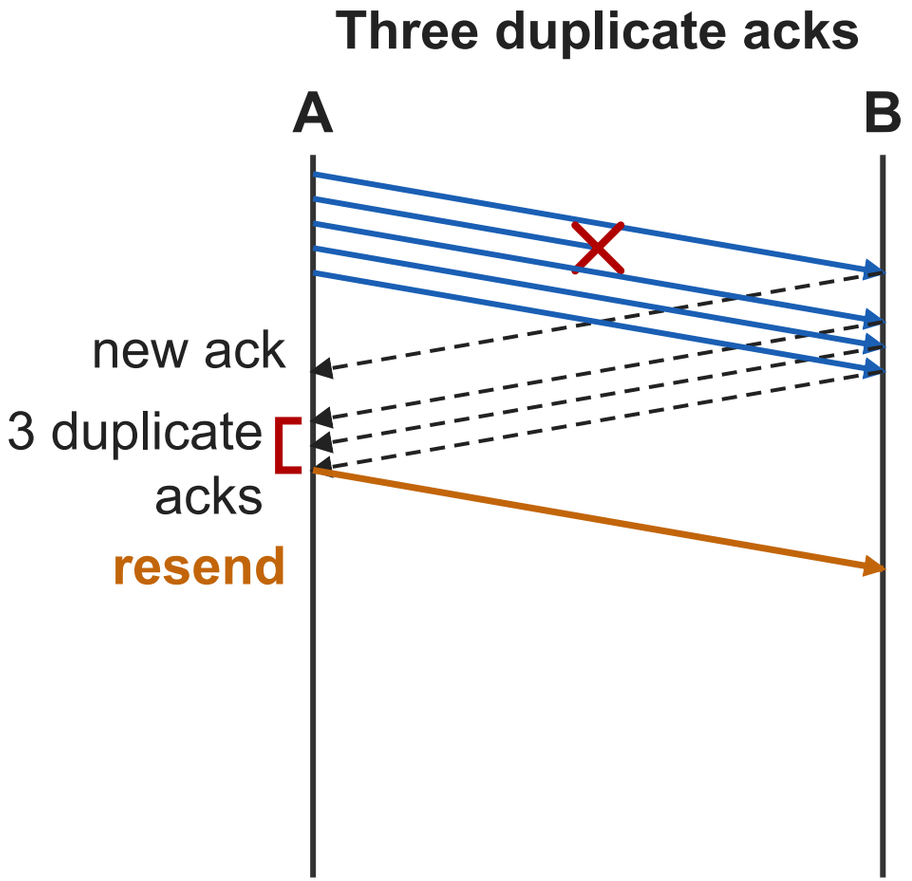
Classic TCP: two kinds of loss signal



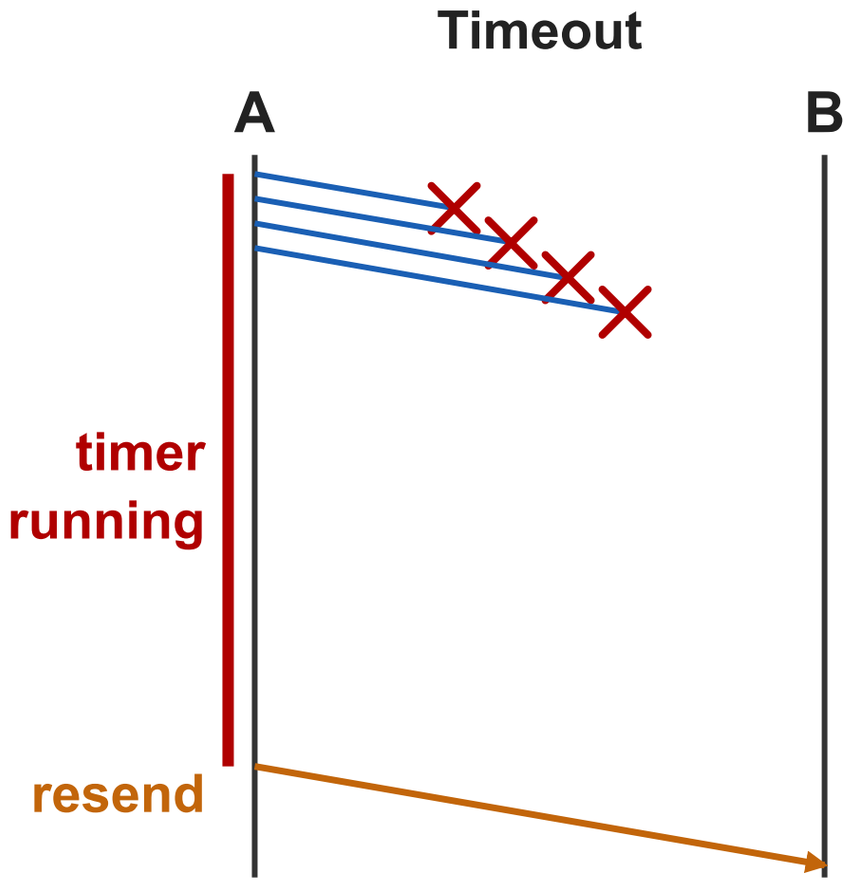


Later packets still get through: **mild congestion**. Resend at once (fast retransmit) and cut cwnd in half

Classic TCP: two kinds of loss signal



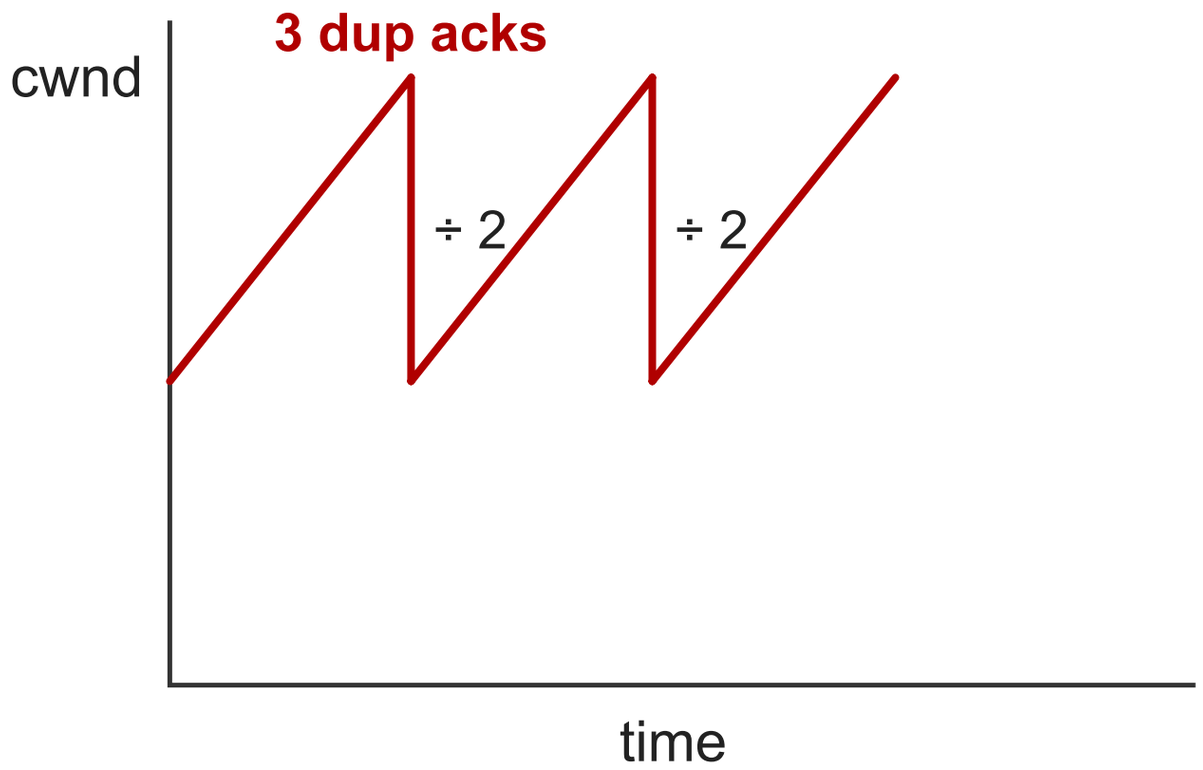
Later packets still get through: **mild congestion**. Resend at once (fast retransmit) and cut cwnd in half



Nothing is getting through: **severe congestion**. cwnd goes back to 1 MSS

Classic TCP: AIMD, with both loss signals

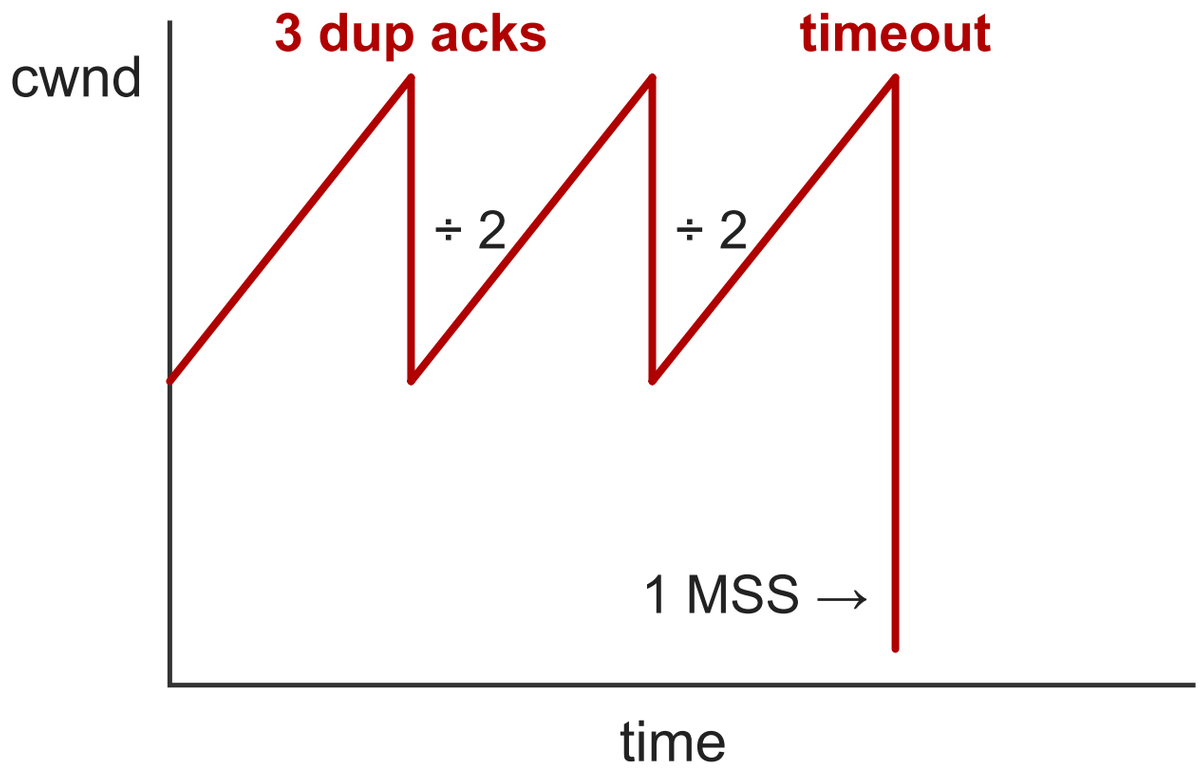
3 duplicate acks:
cut cwnd in half



Classic TCP: AIMD, with both loss signals

3 duplicate acks:
cut cwnd in half

Timeout:
cwnd = 1 MSS

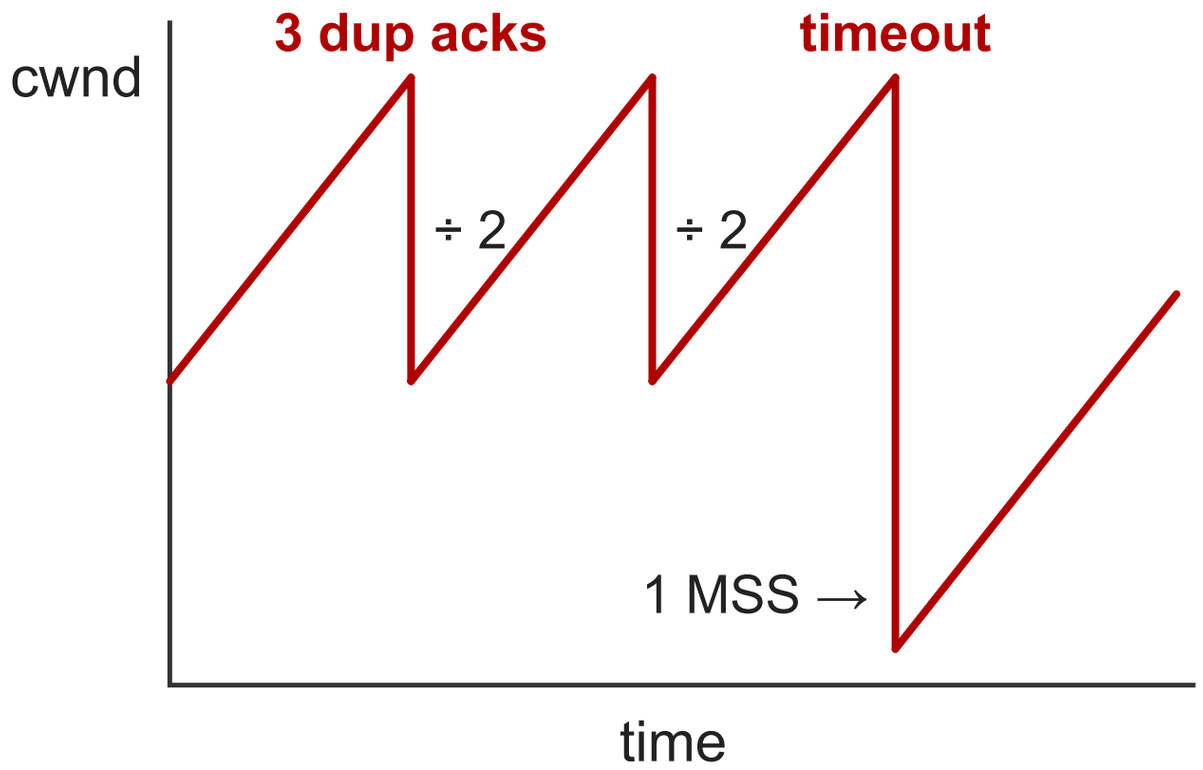


Classic TCP: AIMD, with both loss signals

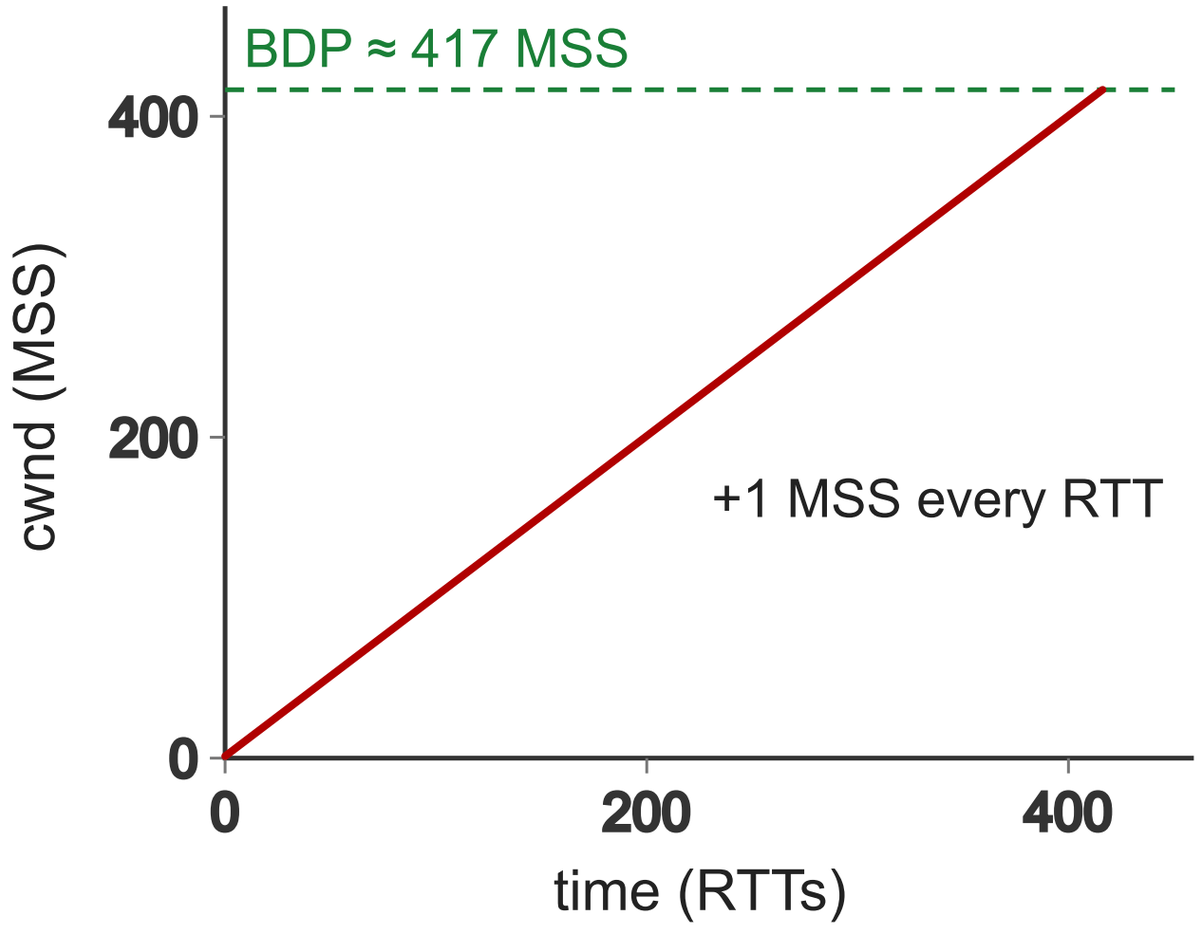
3 duplicate acks:
cut cwnd in half

Timeout:
cwnd = 1 MSS

Then additive increase again,
from 1 MSS, one MSS every RTT



Additive increase is, sometimes, too slow

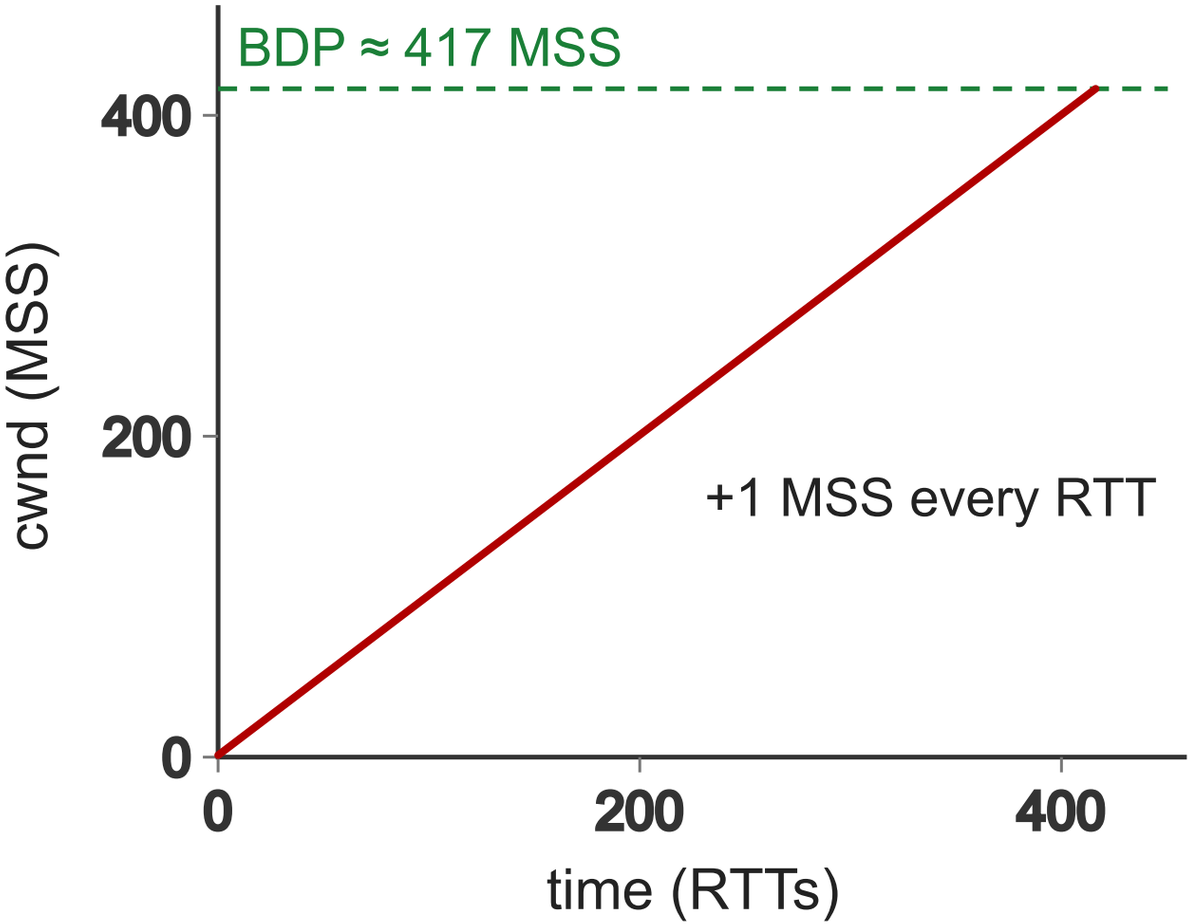


Example: 100 Mb/s, RTT 50 ms, MSS = 1,500 bytes: BDP = 625,000 bytes ≈ 417 MSS

At the start, or after a timeout: cwnd = 1 MSS

+1 MSS every RTT: 416 RTTs ≈ 21 seconds to use the available network capacity

Additive increase is, sometimes, too slow



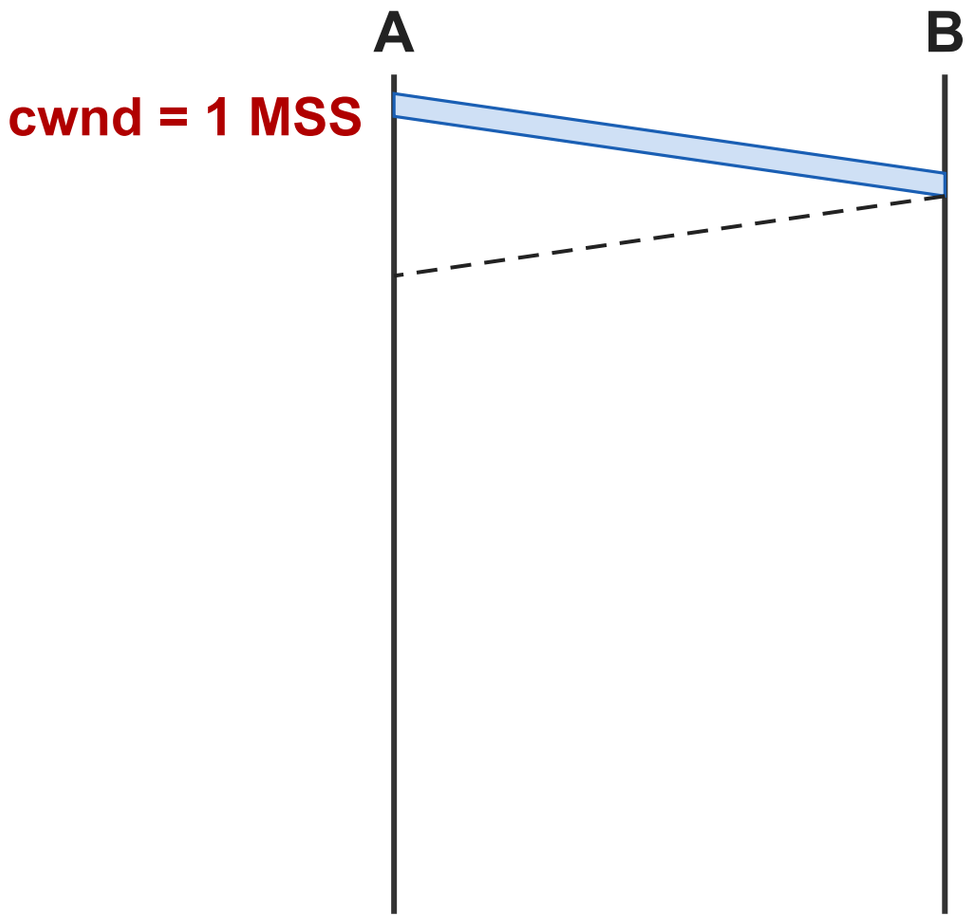
Example: 100 Mb/s, RTT 50 ms, MSS = 1,500 bytes: BDP = 625,000 bytes ≈ 417 MSS

At the start, or after a timeout: cwnd = 1 MSS

+1 MSS every RTT: 416 RTTs ≈ 21 seconds to use the available network capacity

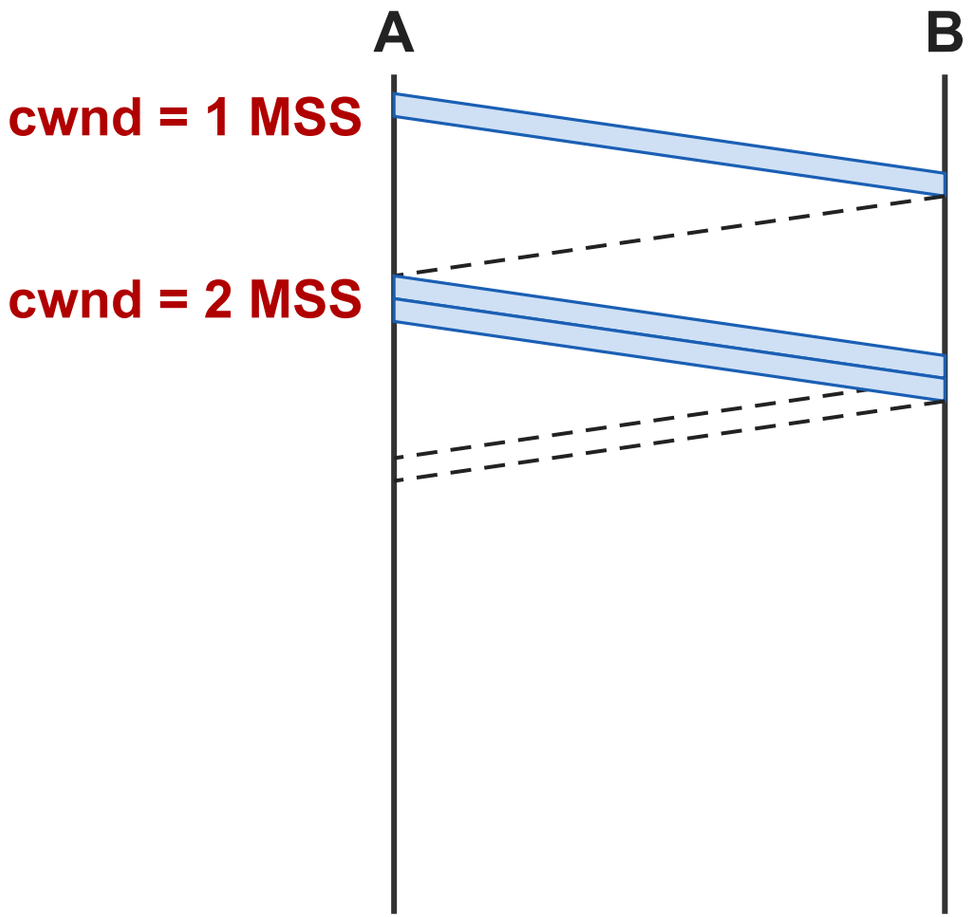
Until then the bottleneck is partly idle

“Slow” Start



Start: cwnd = 1 MSS

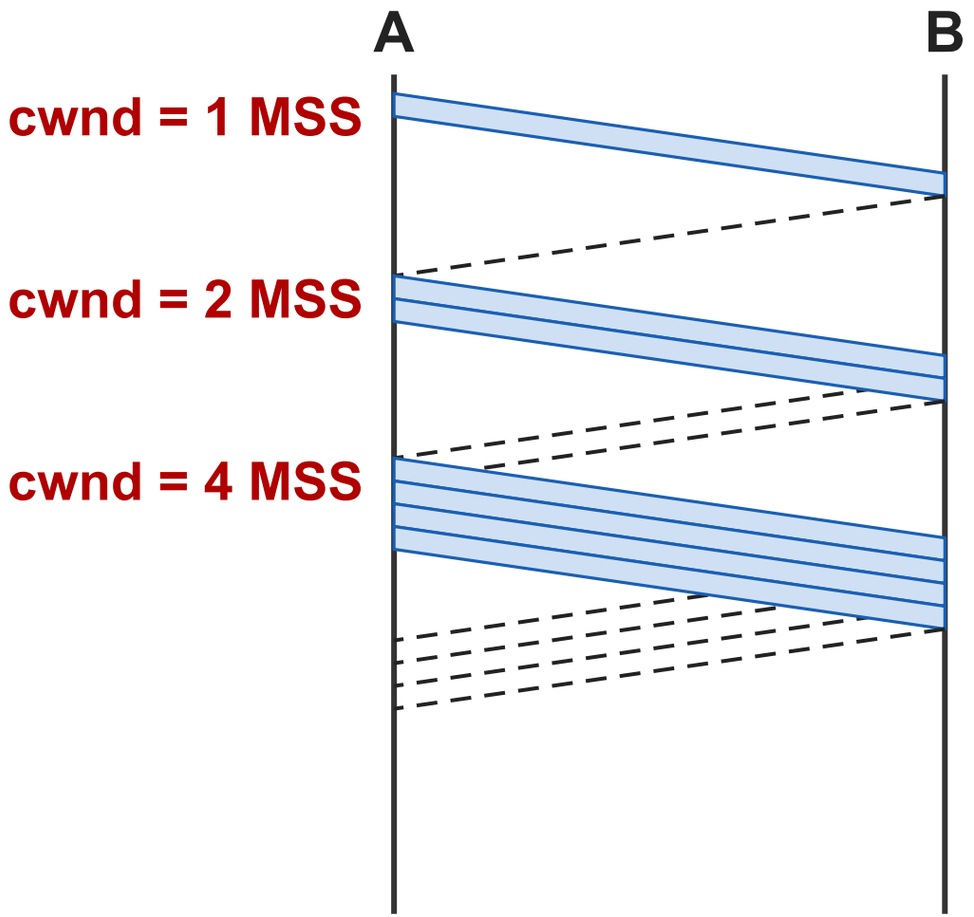
“Slow” Start



Start: cwnd = 1 MSS

+1 MSS to cwnd for every ack:
cwnd doubles every RTT

“Slow” Start

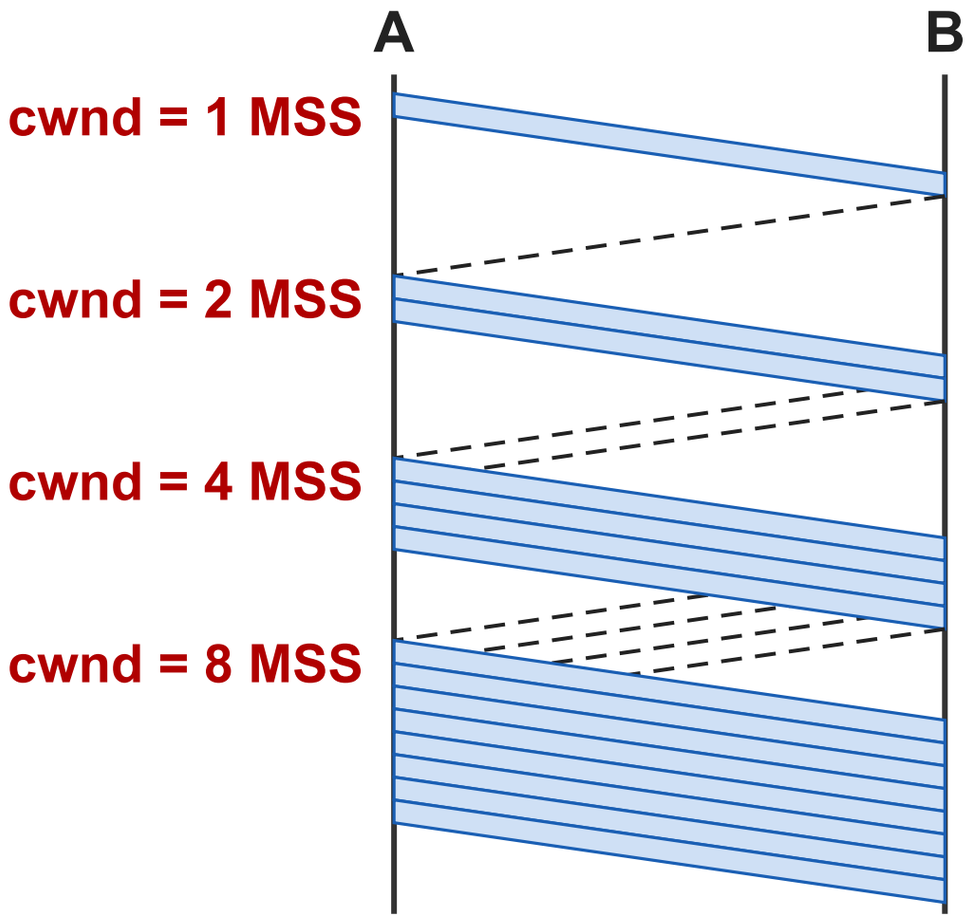


Start: cwnd = 1 MSS

+1 MSS to cwnd for every ack:
cwnd doubles every RTT

From 1 to 417 MSS in about
9 RTTs (0.45 s), not 416

“Slow” Start



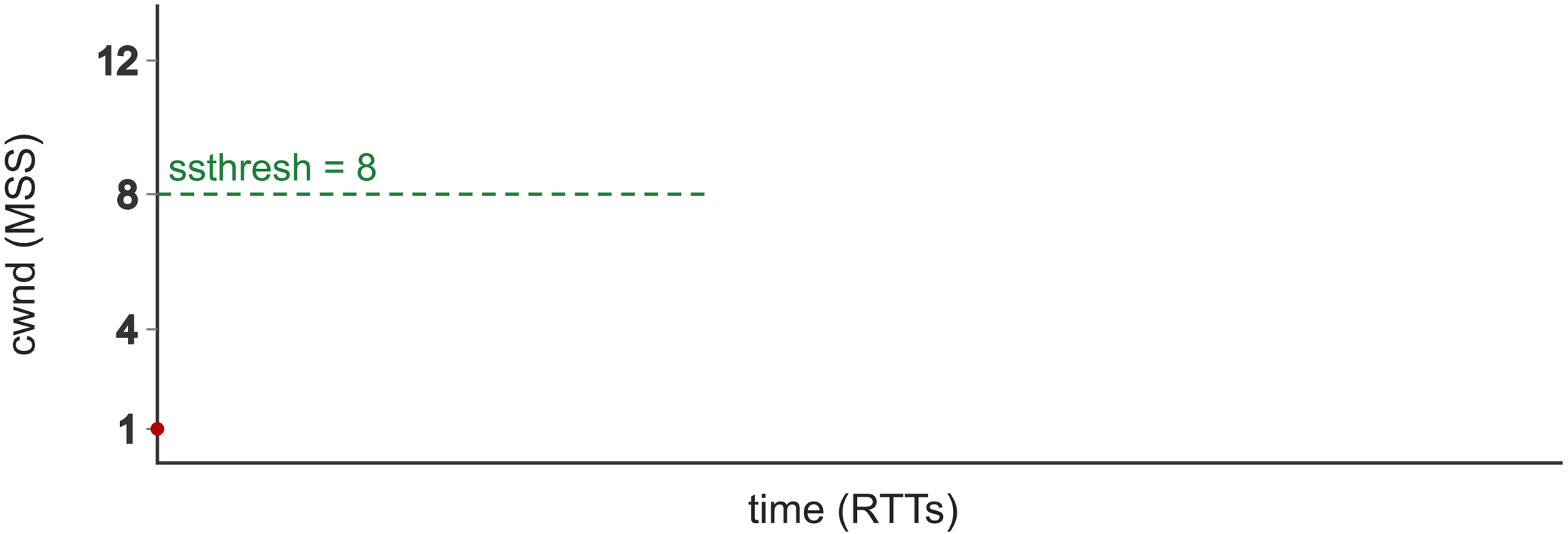
Start: cwnd = 1 MSS

+1 MSS to cwnd for every ack:
cwnd doubles every RTT

From 1 to 417 MSS in about
9 RTTs (0.45 s), not 416

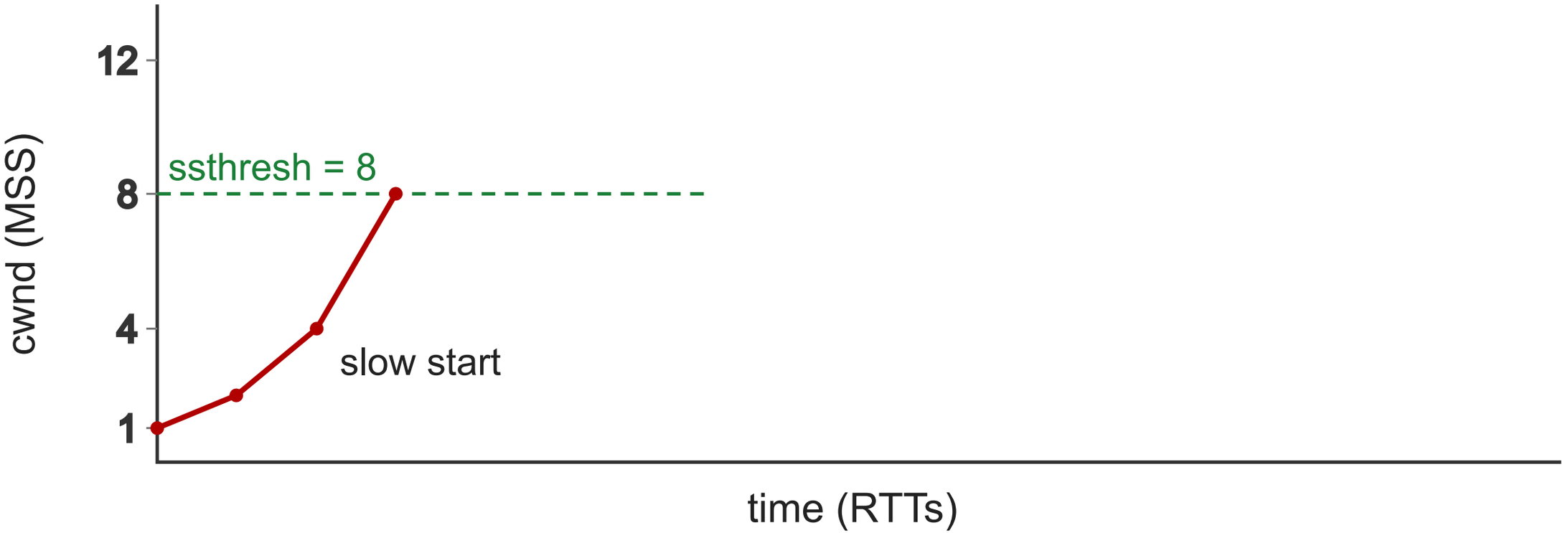
Doubles until the slow start
threshold (ssthresh)

Classic TCP Congestion control



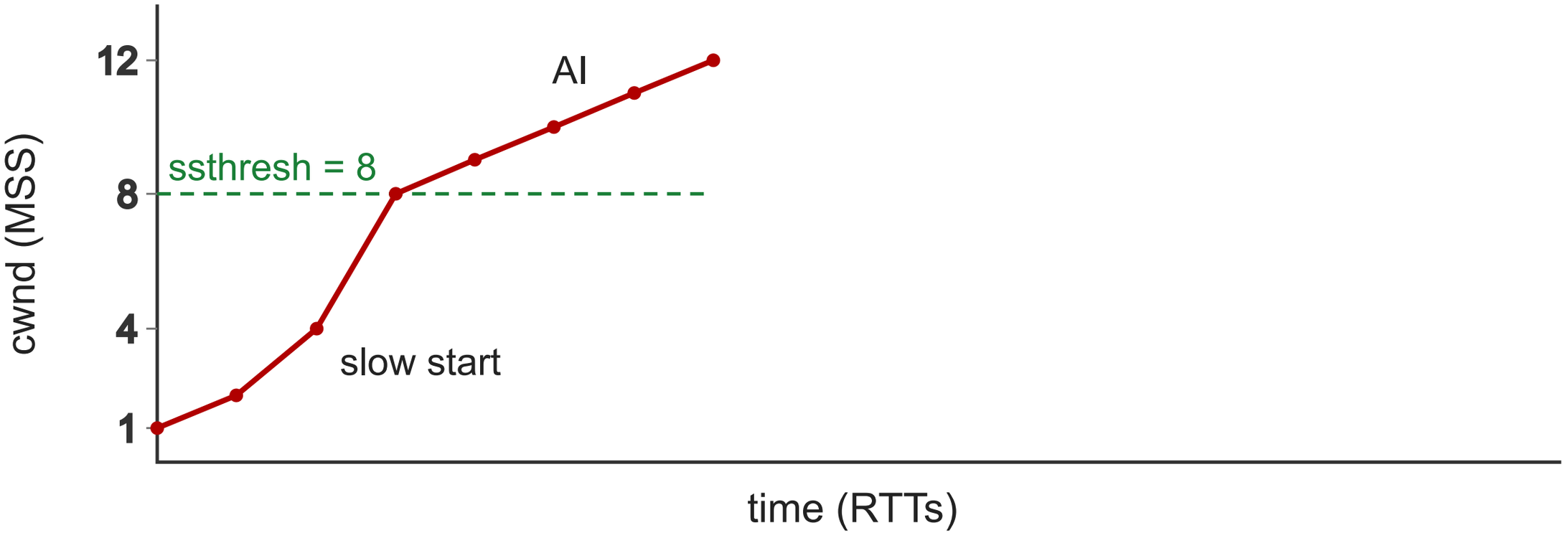
Start: cwnd = 1 MSS, below ssthresh

Classic TCP Congestion control



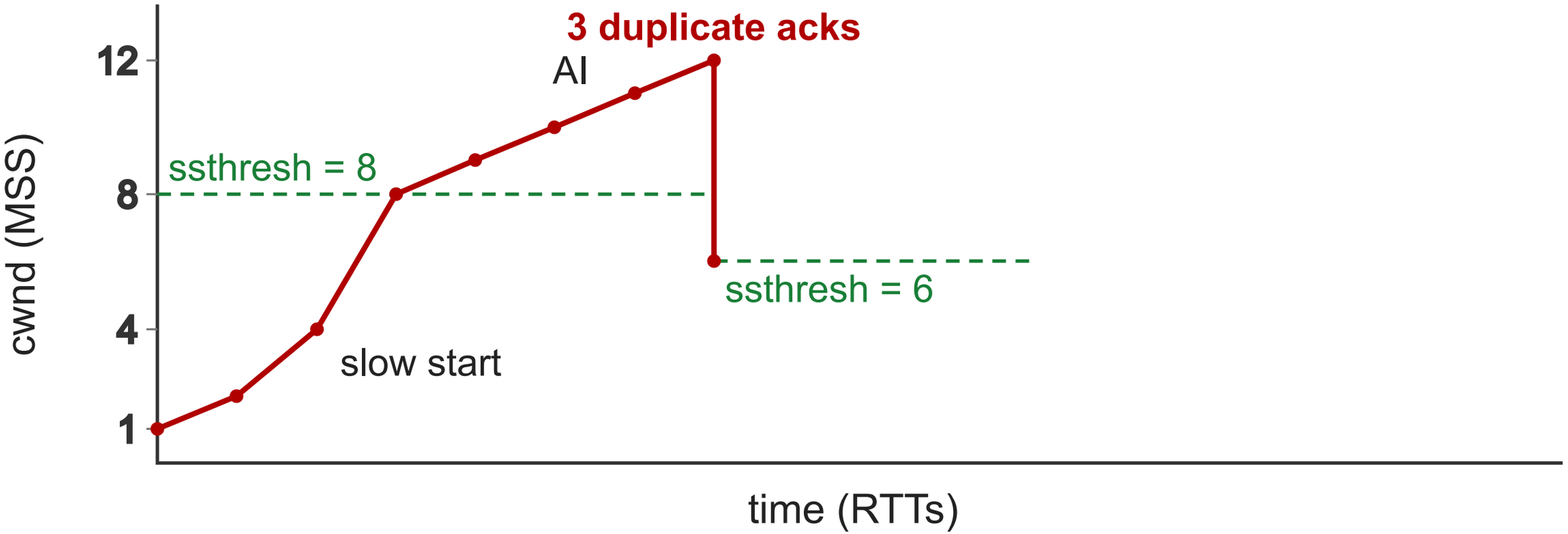
Slow start: cwnd doubles every RTT, up to ssthresh

Classic TCP Congestion control



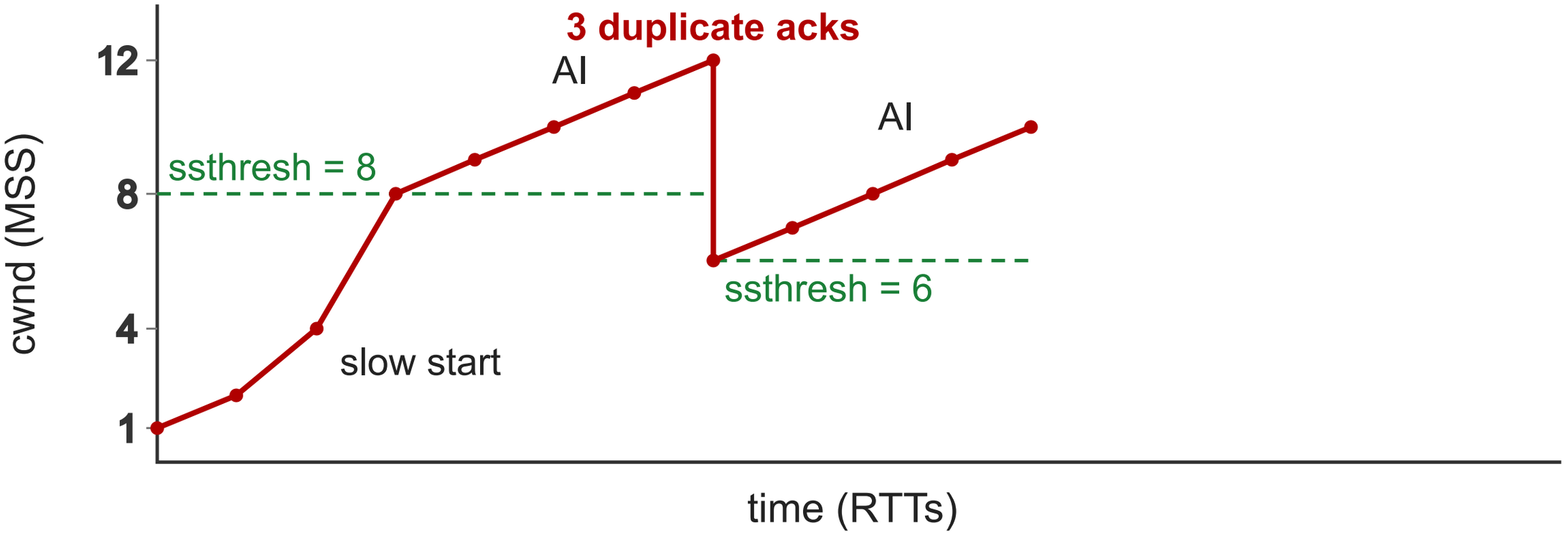
Above ssthresh: additive increase, +1 MSS every RTT

Classic TCP Congestion control



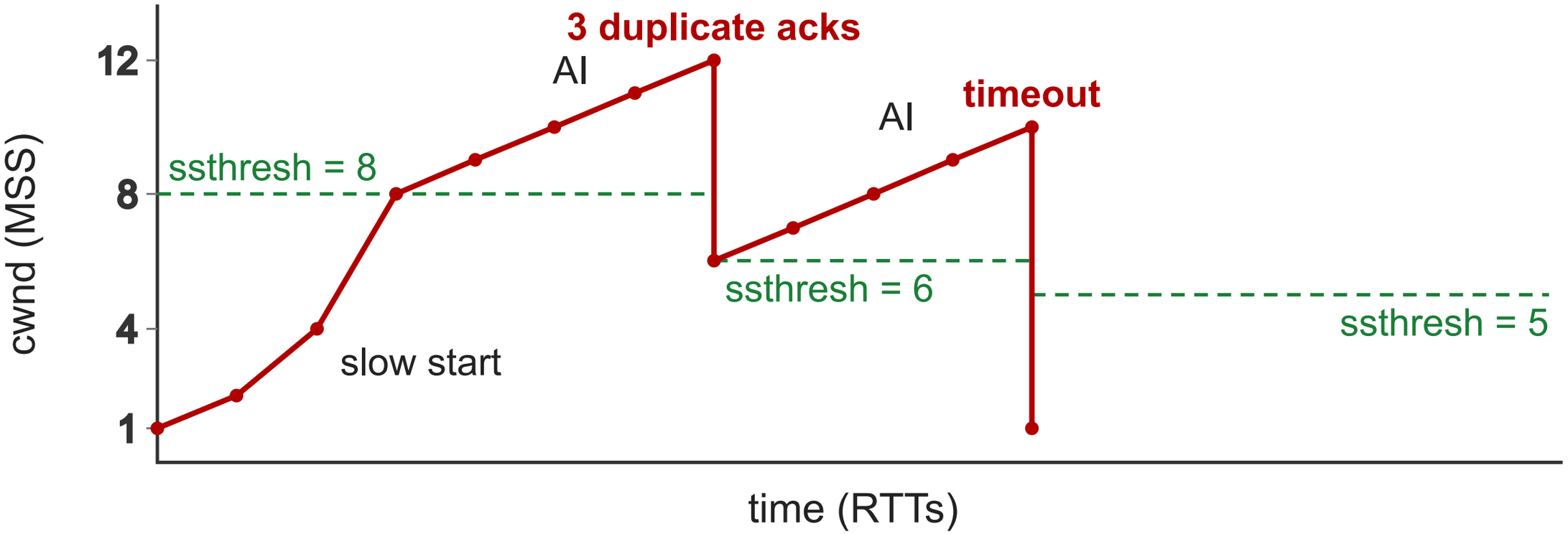
3 duplicate acks: ssthresh = cwnd / 2, cwnd = ssthresh

Classic TCP Congestion control



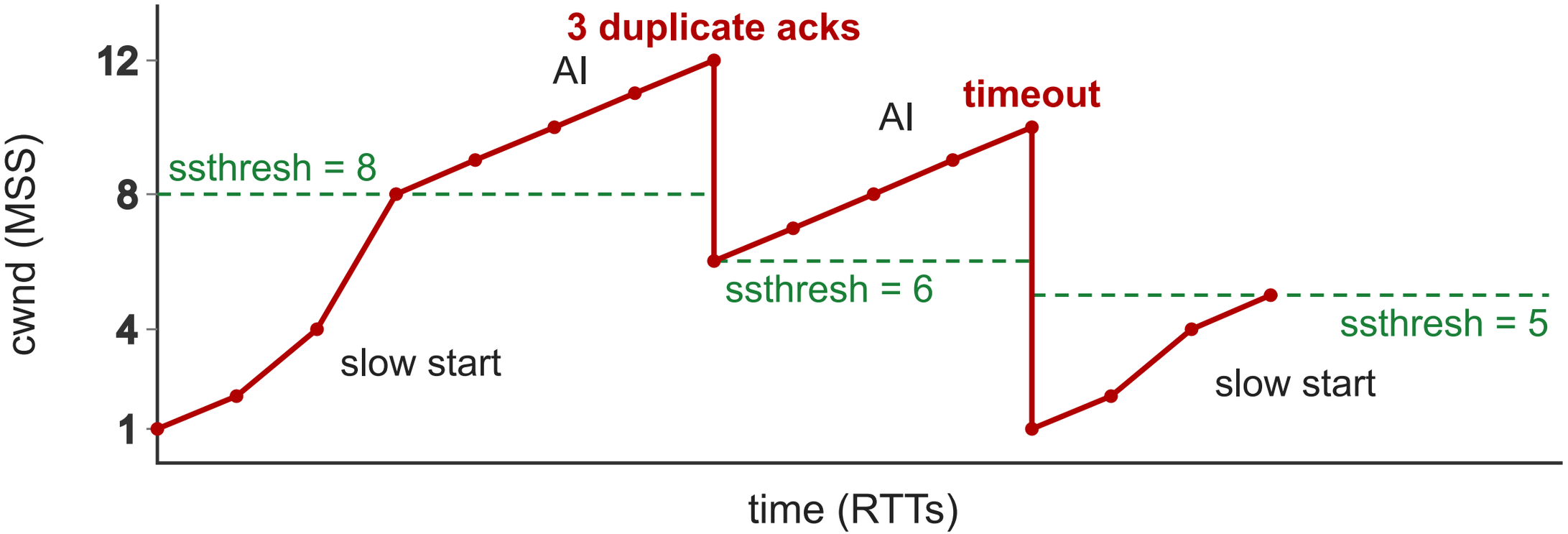
Additive increase again

Classic TCP Congestion control



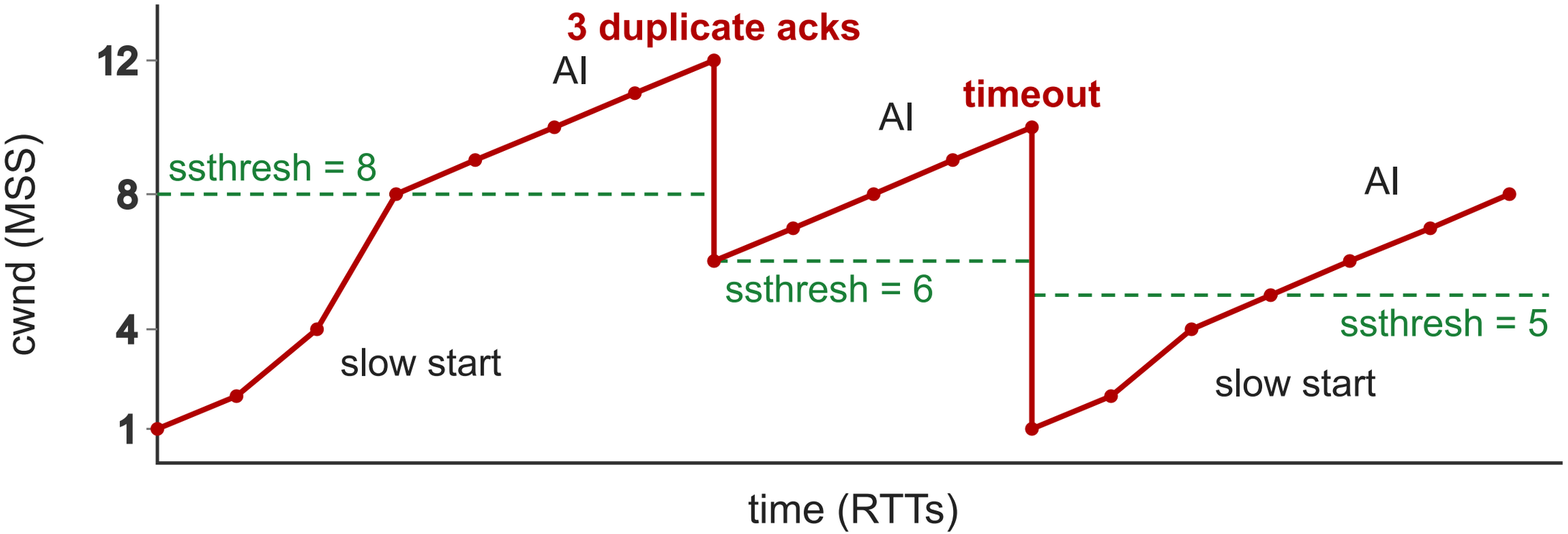
Timeout: ssthresh = cwnd / 2, cwnd = 1 MSS

Classic TCP Congestion control



Slow start again, up to the new ssthresh

Classic TCP Congestion control



Then additive increase

There are even more details (e.g., inflating cwnd with duplicate acks), but we won't get into them in this course